



RS6x_7x HIF

参考手册

文档页数：138
文档版本号：1.1
发布时间：2026-08-14

版本历史

版本	版本描述	日期
1.1	1、修正 C1、C2 数据格式； 2、推荐使用函数 DevHw_Set_DefaultParam 配置 DevHw_t 参数； 3、Host_General_Command 增加参数 data_len 用于获取 resp payload 长度； 4、增加 HIF 帧示例说明。	2026-08-14
1.0	文档初版。	2026-06-30

目录

版本历史 ii

目录 iii

图片目录viii

表格目录 x

1 简介 2

2 硬件设计 3

 2.1 框架 3

 2.2 IO 选型 3

 2.2.1 SPI 3

 2.2.2 UART 3

 2.2.3 IIC 4

 2.2.4 唤醒 IO 4

 2.2.5 多从机 4

 2.2.5.1 SPI 4

 2.2.5.2 Uart 5

 2.2.5.3 IIC 5

 2.3 硬件设计注意事项 6

 2.3.1 参考原理图 6

 2.3.2 Layout 说明 6

3 基本通信流程 7

 3.1 HIF 协议帧格式 7

 3.1.1 Header 7

 3.1.2 CheckSum 9

 3.1.3 Payload 9

 3.1.3.1 Command-Response 10

 3.1.3.2 上报数据 11

 3.1.3.3 分片 17

 3.1.4 HIF 协议帧示例 18

 3.2 Host 与 Device 交互 19

 3.2.1 唤醒与休眠 19

 3.2.1.1 唤醒 20

 3.2.1.2 休眠 20

 3.2.1.3 SPI 21

 3.2.1.4 UART 23

 3.2.1.5 IIC 通信 24

 3.2.2 Command-Response 24

 3.2.2.1 SPI 24

 3.2.2.2 UART 24

 3.2.2.3 IIC 24

3.2.3 数据上报	25
3.2.3.1 SPI	26
3.2.3.2 UART	27
3.2.3.3 IIC	27
3.2.3.4 传输层重传	27
3.2.4 异常恢复	27
3.2.4.1 Host 接收上报数据时收不到消息头	27
4 软件框架	29
4.1 协议架构	29
4.1.1 消息层	29
4.1.2 传输层	30
4.1.3 链路层	30
4.1.4 物理层	30
4.2 软件架构	30
4.2.1 Host	30
4.2.2 Device	31
5 Host 开发	32
5.1 Host Driver 移植	32
5.1.1 文件结构介绍	32
5.1.2 移植内容	34
5.1.2.1 系统接口	34
5.1.2.2 IO 接口	41
5.1.2.3 存储介质接口	44
5.1.2.4 日志接口	46
5.1.2.5 COM 接口	46
5.2 Host Driver 使用流程	56
5.2.1 驱动和设备初始化	56
5.2.1.1 初始化流程	56
5.2.1.2 设备参数配置	57
5.2.1.3 API 使用说明	58
5.2.2 数据上报	61
5.2.2.1 C1/C2/C3 数据使用流程	61
5.2.2.2 其他数据处理流程	63
5.2.2.3 API 使用说明	65
5.2.3 命令下发	68
5.2.3.1 使用流程	68
5.2.3.2 API 使用说明	69
5.2.4 固件下载	79

5.2.4.1 使用流程	79
5.2.4.2 API 使用说明	80
5.3 配置和编译	81
5.3.1 功能裁剪	81
5.3.2 适配层功能开关	81
5.3.3 固件下载功能	82
5.3.4 消息层功能	83
5.3.5 其它	83
5.4 简单的移植测试代码	84
5.4.1 数据上报	84
5.4.1.1 UART	84
5.4.1.2 SPI	85
5.4.2 配置下发	88
5.4.2.1 UART	88
5.5 典型调试过程	89
5.5.1 数据通路	89
5.5.1.1 初始化流程	89
5.5.1.2 LLC_Send 和 LLC_Recv 通路	89
5.5.2 数据上报	90
5.5.2.1 HIF_TL_DataNotify_CB	90
5.5.2.2 com_ops.interrupt_ctrl	90
5.5.2.3 协议逻辑	90
5.5.3 命令下发	90
5.5.3.1 命令发送请求到 HIF_TL_Transport_Task 线程的通路	90
5.5.3.2 协议逻辑	90
5.5.4 固件下载	90
5.5.4.1 同步失败	90
5.6 注意事项	91
5.6.1 应用层的 C1、C2、C3 自动组包	91
5.6.2 休眠功能	91
6 Device 开发	92
6.1 代码目录结构	92
6.2 使用流程	93
6.3 配置说明	94
6.3.1 配置结构体和全局变量	94
6.3.2 默认配置	96
6.3.2.1 基本配置	96
6.3.2.2 功能配置	97

6.3.3 其他配置	97
6.3.4 配置与使用方法	98
6.3.4.1 使用默认配置	98
6.3.4.2 工程静态配置	98
6.3.4.3 动态配置	99
6.4 API 使用说明	100
6.4.1 状态码	100
6.4.2 API	100
6.4.2.1 HIF_DefaultInitParam	100
6.4.2.2 HIF_Init	101
6.4.2.3 HIF_ExtControl	101
6.4.2.4 HIF_MsgReport_ListPushData	102
6.4.2.5 HIF_MsgReport_ListPopData	102
6.4.2.6 HIF_MsgReport_ListStart	103
6.4.2.7 HIF_MsgReport	103
6.4.2.8 HIF_MsgHdl_Regist	104
6.4.2.9 HIF_MsgResp	104
6.4.2.10 HIF_InitStatusReport	105
6.4.2.11 HIF_MsgReport_Lock	106
6.4.2.12 HIF_MsgReport_Unlock	106
6.4.2.13 HIF_PM_WakeupDevice	106
6.4.2.14 HIF_PM_SuspendDevice	107
6.4.3 数据上报	107
6.4.3.1 链表阻塞式发送	108
6.4.3.2 链表回调式发送	109
6.4.3.3 非链表阻塞式发送	111
6.4.3.4 非链表回调式发送	111
6.4.4 Command-Response	111
6.4.4.1 消息处理回调函数原型	112
6.4.4.2 Response 说明	112
6.4.4.3 接收与回复示例	112
6.5 雷达数据上报流程	113
6.5.1 0xC2 雷达 FFT 数据	114
6.5.2 0xC3 雷达点云数据	114
6.5.3 0xC6 “PSIC-调试协议”数据	114
7 自定义 Msg ID	116
7.1 允许自定义的 Msg ID 范围	116
7.2 自定义命令	116
7.2.1 Host 端开发步骤	116
7.2.2 Device 端开发步骤	117
7.3 自定义上报数据	117

7.3.1 Host 端开发步骤	118
7.3.2 Device 端开发步骤	118
8 Q&A	119
8.1 通信问题	119
8.1.1 HOST 发送 payload 大于等于 1024 字节的消息时出现 checksum 校验问题	119
8.1.2 为什么使用 host driver 主动上报 COM_IRQ_THREAD 通知时会接收一包数据但连续遗漏多包数据	119
8.1.3 无法接收到通过 checksum 校验的上报数据包	119
8.1.4 指令与回复	119
8.2 配置问题	120
8.2.1 通过 Host 修改工具/上位机对应的配置	120
8.3 雷达工作异常	120
8.3.1 radar analysis sample 的帧周期设置得很小时，雷达突然不上报了	120
附录	122
现有 Msg ID 定义	122
常用 Command-Response 数据格式	125

图片目录

图 2-1 Device 和 Host 通过 HIF 通信的硬件连接图	3
图 3-1 HIF 帧格式	7
图 3-2 HIF 帧 Header 格式（小端）	7
图 3-3 HIF checksum 计算区域	9
图 3-4 Command Frame MsgHeader	10
图 3-5 Response Frame MsgHeader	10
图 3-6 Response Frame Payload 字段定义	10
图 3-7 C1 FFT 数据消息	12
图 3-8 C2 FFT 数据消息	13
图 3-9 点云数据消息	15
图 3-10 PSIC-调试协议 0xC6	16
图 3-11 分片 Frame 的 Payload 结构	17
图 3-12 待发送数据分片示例	18
图 3-13 分片 Frame 的 Payload 示例	18
图 3-14 HIF Header 示例（小端）	19
图 3-15 唤醒与休眠交互流程	20
图 3-16 唤醒的 SPI 时序	21
图 3-17 数据 55 FF 55 FF 的 SPI 时序图	21
图 3-18 数据 79 79 79 79 的 SPI 时序图	22
图 3-19 Command-Response 的 SPI 时序图	22
图 3-20 休眠 Command 数据的 SPI 时序	22
图 3-21 Response 数据 Header 的 SPI 时序	23
图 3-22 Response 数据 Payload+CheckSum 的 SPI 时序	23
图 3-23 唤醒的 UART 时序	23
图 3-24 Command-Response 的 UART 时序	24
图 3-25 Command-Response 的 SPI 时序	24
图 3-26 数据上报交互流程	26
图 3-27 SPI 上报数据时序	27
图 3-28 UART 上报数据时序	27
图 3-29 Host 与 Device 进行状态同步	28
图 4-1 协议架构	29
图 4-2 Host 软件架构	31
图 4-3 Device 软件架构	31

图 5-1 EDGE 通知类型接收上报数据的流程示意图 47

图 5-2 COM_ISR 通知类型接收上报数据的流程示意图（线程在 cache 满之前调用 LLC_Recv）48

图 5-3 COM_ISR 通知类型接收上报数据的流程示意图（线程在 cache 满之后调用 LLC_Recv）49

图 5-4 COM_IRQ_THREAD 通知类型接收上报数据的流程示意图 49

图 5-5 HostDriver 总体使用流程 56

图 5-6 Host Driver 驱动和设备初始化流程 57

图 5-7 C1/C2/C3 上报数据回调设置流程 61

图 5-8 自定义数据处理方案 64

图 5-9 命令下发流程 68

图 5-10 下载固件流程 79

图 6-1 链表式数据上报示例场景一 107

图 6-2 链表式数据上报示例场景二 107

表格目录

表 2-1	SPI IO 定义	3
表 2-2	Uart IO 定义	4
表 2-3	IIC IO 定义	4
表 2-4	Notify IO 定义	4
表 2-5	SPI 通信 N 从机的 Host IO 需求	4
表 2-6	Uart 通信 N 从机的 Host IO 需求	5
表 2-7	IIC 通信 N 从机的 Host IO 需求	5
表 3-1	HIF-Header 字段定义	7
表 3-2	HIF Response 和 Report status 定义	10
表 3-3	HIF Command-Response 常用 MsgID 定义	11
表 3-4	C1 FFT 数据消息 Payload 说明	12
表 3-5	C2 FFT 数据消息 Payload 说明	13
表 3-6	点云数据消息 Payload 说明	15
表 3-7	PSIC-调试协议 0xC6 Payload 字段说明	16
表 3-8	Frag Header 说明	17
表 5-1	host_os_enter_critical	34
表 5-2	host_os_exit_critical	34
表 5-3	host_os_timestamp_us	35
表 5-4	host_os_malloc	35
表 5-5	host_os_free	35
表 5-6	host_os_memset	36
表 5-7	host_os_memcpy	36
表 5-8	host_os_memcmp	36
表 5-9	host_os_thread_create	37
表 5-10	host_os_thread_delete	38
表 5-11	host_os_thread_is_valid	38
表 5-12	host_os_sem_create	38
表 5-13	host_os_sem_delete	39
表 5-14	host_os_sem_is_valid	39
表 5-15	host_os_sem_take	39
表 5-16	host_os_sem_give	40
表 5-17	host_os_delayms	40
表 5-18	host_os_delayus	40

表 5-19	host_os_pm_lock	40
表 5-20	host_os_pm_unlock	41
表 5-21	host_io_mode_set	41
表 5-22	host_io_irq_cfg	42
表 5-23	host_io_irq_enable	43
表 5-24	host_io_irq_disable	43
表 5-25	host_io_write	43
表 5-26	host_io_read	44
表 5-27	host_store_open	44
表 5-28	host_store_close	45
表 5-29	host_store_read	45
表 5-30	host_log_print	46
表 5-31	LLC 不同通知类型的接口需求	49
表 5-32	host_com_init_t	50
表 5-33	host_com_deinit_t	51
表 5-34	host_com_open_t	51
表 5-35	host_com_close_t	51
表 5-36	host_com_write_t	52
表 5-37	host_com_read_t	52
表 5-38	host_com_interrupt_ctrl_t	53
表 5-39	host_com_interrupt_handle_regist_t	53
表 5-40	host_com_read_finish_t	54
表 5-41	Host_Driver_Init	58
表 5-42	Host_Driver_Deinit	59
表 5-43	Host_Device_Regist	59
表 5-44	Host_Device_Unregist	60
表 5-45	Host_Device_Open	60
表 5-46	Host_Device_Close	60
表 5-47	Report_CB	65
表 5-48	Mmw_Cx_FUNCTION_Handle_Set	65
表 5-49	Cx 与 FUNCTION 的对应关系	66
表 5-50	Mmw_Report_Handle_Set_General	66
表 5-51	Mmw_Report_Handle_FreeAll	68
表 5-52	HostCmd_Version_Get	69
表 5-53	HostCmd_Device_Reboot	69
表 5-54	Host_General_Command	70

表 5-55	MmwCmd_General_StartCtrl_Cfg	71
表 5-56	MmwCmd_General_MimoMode_Cfg	71
表 5-57	MmwCmd_General_FrameType_Cfg	72
表 5-58	MmwCmd_General_StartFreq_Cfg	72
表 5-59	MmwCmd_General_TriggerRange_Cfg	73
表 5-60	MmwCmd_General_RangeResolution_Cfg	73
表 5-61	MmwCmd_General_MaxVelocity_Cfg	73
表 5-62	MmwCmd_General_VelResolution_Cfg	74
表 5-63	MmwCmd_General_FramePeriod_Cfg	74
表 5-64	MmwCmd_RadarAnalysis_Start	74
表 5-65	MmwCmd_RadarAnalysis_Stop	75
表 5-66	MmwCmd_RadarAnalysis_Mode_Cfg	75
表 5-67	MmwCmd_RadarAnalysis_Freq_Cfg	75
表 5-68	MmwCmd_RadarAnalysis_Range_Cfg	76
表 5-69	MmwCmd_RadarAnalysis_Veloc_Cfg	76
表 5-70	MmwCmd_RadarAnalysis_Frame_Cfg	77
表 5-71	MmwCmd_RadarAnalysis_Intv_Cfg	77
表 5-72	MmwCmd_RadarAnalysis_ChirpNum_Cfg	78
表 5-73	MmwCmd_RadarAnalysis_Report_Cfg	78
表 5-74	MmwCmd_RadarAnalysis_DopFft_Cfg	78
表 5-75	Host_BurnMode_Enter	80
表 5-76	Host_Burn_Image	80
表 5-77	Host_BurnMode_Exit	81
表 5-78	用于裁剪功能的全局宏	81
表 5-79	用于开关适配层功能的全局宏	82
表 5-80	固件下载功能的详细配置	82
表 5-81	用于配置消息层的全局宏	83
表 5-82	其它配置宏	83
表 6-1	HIF 基本配置	96
表 6-2	HIF 功能配置项	97
表 6-3	状态码	100
表 8-1	radar 配置 TLV (部分)	128

1 简介

PSIC 作为独立的智能传感器，在满足应用需求时，例如：检测到目标时，会通过通信口上报被检测物的坐标给 Host。Host 在面对不同的应用场景时也会通过通信口下发雷达配置参数。若在低功耗场景，Host 和 Device 大部分时间处于休眠状态，突发的通信需要先唤醒对方，因此需要唤醒通知 IO。

Device 主动发送数据给 Host，本文档称为“Report”，Host 主动发送数据给 Device 称为“Command”，Device 对“Command”的回复称为“Response”。协议约定，除几个特殊“Command”外，其他“Command”都对应一个“Response”，且仅支持处理单个“Command”，即 Host 收到当前“Command”的“Response”后才能发送下一个“Command”。“Report”支持主动上报和被动上报两种方式。被动上报常用于 SPI/IIC 这种通信方式，Device 作为通信从机，Device 在传输数据给 Host 前需先通过 IO 向 Host 指示 Device 有数据发送，Host 先发送特殊 POLL 帧给 Device，Device 准备好数据后再次通过 IO 指示后，Host 再读取 Device 数据，从而达到 Device 的“Report”数据传输给 Host。主动上报常用于 UART 通信方式，Device 直接通过 UART 的 Tx 发送“Report”数据给 Host。被动上报常用于 SPI 或 IIC 等主从通信方式。

HIF 默认支持 UART/SPI/IIC 三种通信方式，可通过参数动态选择其中的一种通信方式，也可通过配置宏裁剪关闭某些通信方式以节省代码空间。UART 使用简便，SPI 适用大带宽数据传输，IIC 适用多从机传输。SPI/IIC 只支持被动上报，UART 支持主动上报和被动上报。

注意：

1. 雷达应用的通信协议使用 HIF 协议，如雷达上报检测目标位置信息或 Host/主控发送配置参数。
2. 雷达固件升级的通信协议有可能不使用 HIF 协议，需根据具体型号确定。雷达固件升级分为 BURN 和 OTA 两种，BURN 即 Host 与雷达芯片的 BOOT ROM 通信直接进行固件下载烧录，部分型号芯片使用的通信协议不是 HIF；OTA 是应用上先将固件下载到备份区再进行交换升级，使用的协议默认是 HIF。

2 硬件设计

2.1 框架

在 Device 和 Host 之间通过 HIF 通信，硬件上需要的资源：

- 物理通信接口：SPI、UART、I2C 等。
- 通知 IO：用于 Device 通知 Host，指示 Device 有数据需要上报以及 Device 处于 Ready 状态，Host 可以取数。

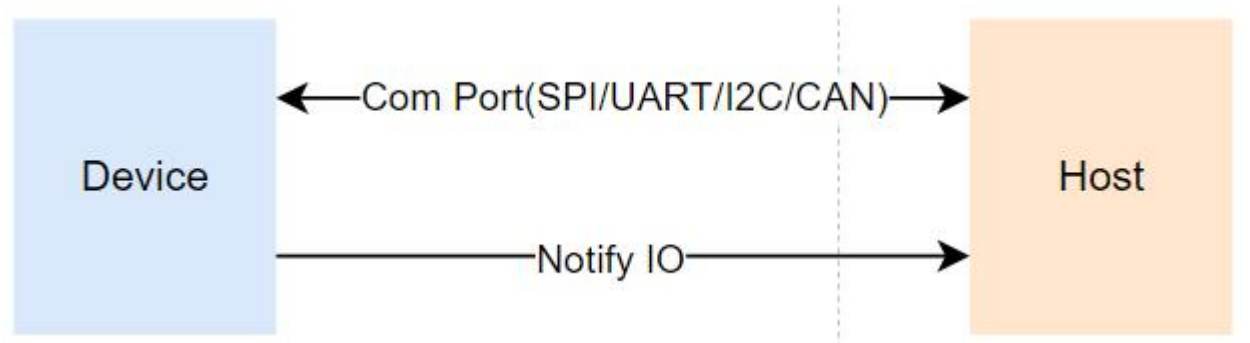


图 2-1 Device 和 Host 通过 HIF 通信的硬件连接图

2.2 IO 选型

2.2.1 SPI

当 SPI 作为物理通信接口时，Device 作为 Slave（默认 SPI0），Host 作为 Master。IO 说明如下：

表 2-1 SPI IO 定义

IO	Device	Host
SPI0-CS	PA0	自定义
SPI0-CLK	PA1	自定义
SPI0-MOSI	PA2	自定义
SPI0-MISO	PA3	自定义

注意：选择 SPI 作为通信端口时，Debug（Uart Print）口建议选择 Uart0（PB0，PB1），同时 PB0 还可以复用为 UPD（烧录控制 IO），可节约 IO 资源。

2.2.2 UART

当 UART 作为物理通信接口时，Device 默认选择 Uart0 或者 Uart1 作为通信接口。由于 PB0 用作 UART0-TX 的同时，也可用作烧录控制 IO，所以建议选择 Uart0，这样可以节约 IO 资源。

IO 说明如下：

表 2-2 Uart IO 定义

IO	Device	Host
UART0-TX	PB0	自定义
UART0-RX	PB1	自定义

注意：选择 UART 作为通信端口时，Debug（Uart Print）口建议选择 Uart1（PA0，PA1）。

2.2.3 IIC

当 IIC 作为物理通信接口时，Device 作为 Slave（默认 IIC0），Host 作为 Master。IO 说明如下：

表 2-3 IIC IO 定义

IO	Device	Host
IIC0-SCL	PA5	自定义
IIC0-SDA	PA4	自定义

注意：选择 IIC 作为通信端口时，Debug（Uart Print）口建议选择 Uart0（PB0，PB1），同时 PB0 还可以复用为 UPD（烧录控制 IO），可节约 IO 资源。

2.2.4 唤醒 IO

表 2-4 Notify IO 定义

IO	Device	Host
Notify IO	PA6（输出）	自定义（输入）

2.2.5 多从机

本章节说明一主（Host）多从（Device）应用场景下，Host 的 IO 资源需求。

主要有唤醒 IO、通信接口 IO（SPI、IIC）。

2.2.5.1 SPI

表 2-5 SPI 通信 N 从机的 Host IO 需求

IO 类别	Host IO 需求	说明
唤醒 IO	N	一个唤醒 IO 对应一个 Device 的唤醒 IO。
SPI-CS	N	
SPI-CLK	1	
SPI-MOSI	1	
SPI-MISO	1	

IO 类别	Host IO 需求	说明
UPD	1	用于烧录 Device。
NRST	N	用于复位 Device。

注:

- UPD 和 NRST 联合使用可使 Device 进入烧录模式：拉低 UPD，再拉低 NRST，之后保持 UDP 低电平最低 50ms，可使 Device 进入烧录模式。
- SPI 多从机设计有使用约束，故：

RS6130 方案请参考《RS6130_硬件设计和 PCB Layout_指导手册_Vxx.pdf》的章节 7.3 SPI 多雷达硬件设计。

RS6240 方案请参考《RS6240_硬件设计和 PCB Layout_指导手册_Vxx.pdf》的章节 7.3 SPI 多雷达硬件设计。

2.2.5.2 Uart

表 2-6 Uart 通信 N 从机的 Host IO 需求

IO 类别	Host IO 需求	说明
唤醒 IO	N	一个唤醒 IO 对应一个 Device 的唤醒 IO。
Uart-Tx	N	
Uart-Rx	N	
UPD	1	用于烧录 Device。
NRST	N	用于复位 Device。

2.2.5.3 IIC

表 2-7 IIC 通信 N 从机的 Host IO 需求

IO 类别	Host IO 需求	说明
唤醒 IO	N	一个唤醒 IO 对应一个 Device 的唤醒 IO。
IIC-SCL	1	
IIC-SDA	1	
UPD	1	用于烧录 Device。
NRST	N	用于复位 Device。

2.3 硬件设计注意事项

2.3.1 参考原理图

RS6130 方案请参考《RS6130_硬件设计和 PCB Layout_指导手册_Vxx.pdf》的章节 7 参考原理图。

RS6240 方案请参考《RS6240_硬件设计和 PCB Layout_指导手册_Vxx.pdf》的章节 7 参考原理图。

2.3.2 Layout 说明

UART/I2C 工作频率较低，按照常规设计即可。

SPI 允许更高工作频率，故若频率大于 5MHz 时，则：

RS6130 方案参考《RS6130_硬件设计和 PCB Layout_指导手册_Vxx.pdf》的章节 8.8 对外 SPI Layout 指南。

RS6240 方案参考《RS6240_硬件设计和 PCB Layout_指导手册_Vxx.pdf》的章节 8.8 对外 SPI Layout 指南。

3 基本通信流程

3.1 HIF 协议帧格式

本章节说明 HIF 的帧（本文档称为 Frame）格式。

Frame 由三部分组成：Header，Payload，Checksum，如下图。

- Header 字段长度固定为 6 Bytes；
- Payload 字段长度在 0 至 4095 Bytes 之间；
- Checksum 字段长度固定为 4 Bytes。



图 3-1 HIF 帧格式

3.1.1 Header

HIF Header 的 6 Bytes 组成如下图。

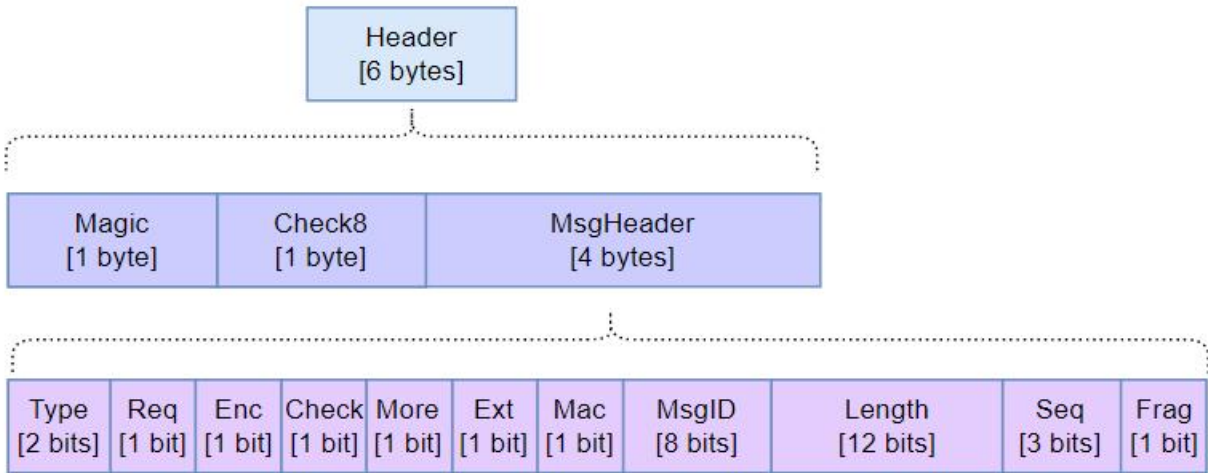


图 3-2 HIF 帧 Header 格式（小端）

Header 的各字段定义如下：

表 3-1 HIF-Header 字段定义

字段	大小	说明	典型值
Magic	8bit	特征码，为固定值 0xA5。	0xA5

字段	大小	说明	典型值
Check8	8bit	头部校验和，校验范围 Magic+MsgHeader；累加单元为 Byte， 校验值为累加和后按位取反，即 check8=~sum8。	N/A
MsgHeader	Type	类型： 0：保留； 1：Host 消息，Host → Device； 2：Device 消息，Device → Host； 3：保留。	Host → Device: 1 Device → Host: 2
	Req	是否需要应答： 0：非请求消息或应答消息； 1：请求消息，需要应答。	Host → Device: 1 Device → Host: 0
	Enc	是否加密： 0：非加密消息； 1：加密消息。	0
	Check	是否有校验字段： 0：无校验值字段 checksum； 1：有校验字段 checksum。	1
	More	待传数据指示： 0：无更多待传消息； 1：有更多待传消息。	0
	Ext	扩展头部指示： 0：无扩展头部； 1：有扩展头部。	0
	Mac	校验字段 Checksum 的校验算法类型： 0：校验算法为 Check32； 1：其他校验算法。	0
	MsgID	消息 ID，范围 0x00~0xFF。	N/A
	Length	数据（payload）长度，范围 0~4095。	N/A

字段		大小	说明	典型值
	Seq	3bit	序列号，范围 0~7，从 0 开始，传输时，每个 Frame 增加 1，循环计数。	0~7
	Frag	1bit	分片标志，定义如下： 0: 当前 Frame 为非分片 Frame； 1: 当前 Frame 为分片 Frame。	0
Check32		32bit	32 位校验值，校验范围 Msg Header[2 ~ 5] + Payload。 1) 当类型为 check32 时，累加单元为 DWORD（32bit），尾部不足 DWORD 时补零计算，校验值为累加和的位取反，即 $Check32 = \sim sum32$ ； 2) 当类型为 MAC32（数据完整性校验）时，校验算法为各平台自定义。	校验和

3.1.2 CheckSum

32 位校验值，校验范围 MsgHeader + Payload。

- 当 MsgHeader 中的 Mac 为 0 时，校验算法为：累加和后按位取反。具体计算方法：以 DWORD(32bit) 为累加单元，从 MsgHeader 到 Payload 进行累加（尾部不足 DWORD 时，高位补零计算），得到 32 位累加和 sum32，则最终校验值 CheckSum = $\sim sum32$ 。
- 当 MsgHeader 中的 Mac 为 1 时，保留。

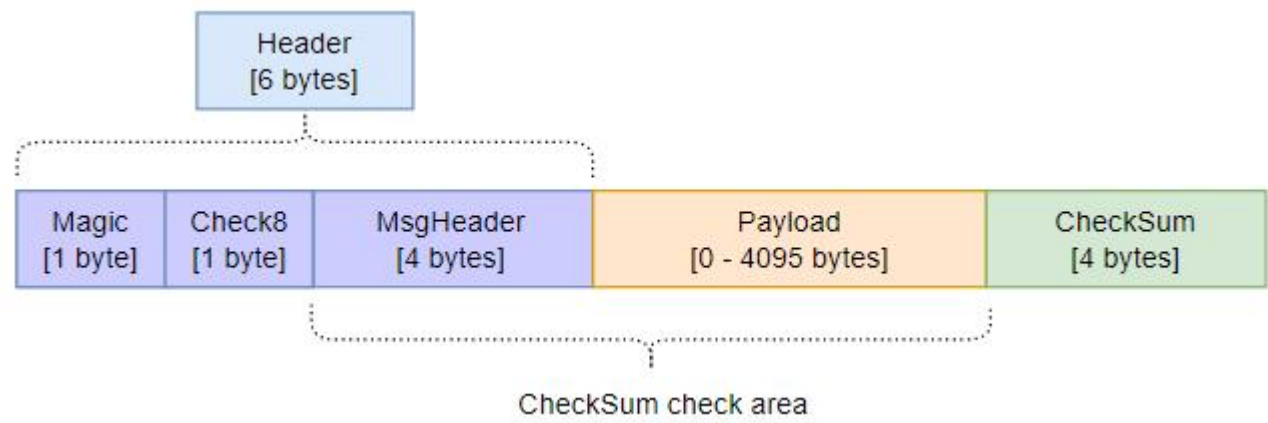


图 3-3 HIF checksum 计算区域

3.1.3 Payload

这部分主要是已定义的应用数据的格式说明，即 payload 字段。

3.1.3.1 Command-Response

Command，指 Host 发送给 Device 的命令。

Response，指的是 Device 收到 Host 的命令后，对该命令的回复，ID 与 Command 相同，回复内容有：执行状态、数据等。

Command 与 Response 一般成对出现。

3.1.3.1.1 Command 格式

遵循上述 HIF Frame 格式。其中，MsgHeader 如下图。

Type [2 bits]	Req [1 bit]	Enc [1 bit]	Check [1 bit]	More [1 bit]	Ext [1 bit]	Mac [1 bit]	MsgID [8 bits]	Length [12 bits]	Seq [3 bits]	Frag [1 bit]
1	1	0	1	0	N/A	0	Cmd ID	Length of Cmd Data	NA	0

图 3-4 Command Frame MsgHeader

3.1.3.1.2 Response 格式

Response Frame 遵循 HIF 帧格式，其中：

- MsgID 与对应 Command 相同；
- MsgHeader 各字段信息如下图：

Type [2 bits]	Req [1 bit]	Enc [1 bit]	Check [1 bit]	More [1 bit]	Ext [1 bit]	Mac [1 bit]	MsgID [8 bits]	Length [12 bits]	Seq [3 bits]	Frag [1 bit]
2	0	0	1	0	N/A	0	Cmd ID	Length of Payload	NA	0

图 3-5 Response Frame MsgHeader

- Payload 的第一个 Byte 为状态信息 Status，表示 Command 的处理状态，Status 后是回复数据（可选），如下图。

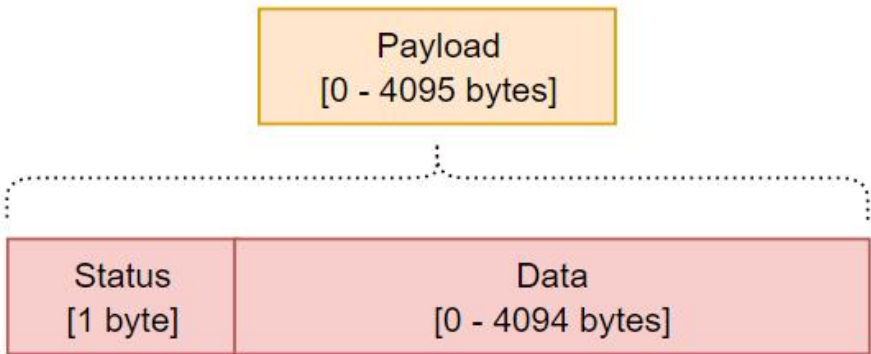


图 3-6 Response Frame Payload 字段定义

Response 的 status 定义如下：

表 3-2 HIF Response 和 Report status 定义

Status	值	含义
HIF_CMD_STATUS_SUCCESS	0	命令执行成功

Status	值	含义
HIF_CMD_STATUS_UNSUPPORT	1	不支持的命令
HIF_CMD_STATUS_VERSION	2	版本不匹配
HIF_CMD_STATUS_TOOLONG	3	消息长度不匹配
HIF_CMD_STATUS_CHECK	4	头部或者数据校验失败
HIF_CMD_STATUS_PARAM	5	参数无效
HIF_CMD_STATUS_REPLAY	6	重复的消息
HIF_CMD_STATUS_AUTH	7	认证失败
HIF_CMD_STATUS_LATE	8	需要延迟上报
HIF_CMD_STATUS_BUSY	9	设备处于忙状态
HIF_CMD_STATUS_STATE	10	当前设备状态不能处理
HIF_CMD_STATUS_SYSERR	11	系统错误/Flash 写入错误
HIF_CMD_STATUS_IO	12	通用错误

3.1.3.1.3 常用 Command

表 3-3 HIF Command-Response 常用 MsgID 定义

MsgID	Description
0x00	获取 Device 版本号。
0x01	获取 Device Tick 时钟，用于防呆检测 Device Active 状态。
0x04	设置雷达复位重启
0x05	设置 Device 休眠状态。
0x07	设置雷达开始或停止检测
0x0C	被动上报时，Host 通知 Device 的 POLL 指令
0x42	动态获取 Device 配置信息，部分 sample 不使用此 ID。
0x60	动态配置 Device 雷达或 IO 参数，部分 sample 不使用此 ID。

各 Command 的 Payload 具体定义见附录：常用 Command-Response 数据格式。

3.1.3.2 上报数据

本章节说明部分已定义的 MsgID 的 Payload 格式，主要是雷达相关的数据。

3.1.3.2.1 Cube 数据（0xC1）

雷达 FFT 数据上报使用 MsgId 为 0xC1 的 hif 消息，该消息的 Payload 定义如下图所示。图中右边为雷达一帧完整的 FFT 数据，其可能被拆分成多个数据分片进行传输，数据分片传输的顺序为从上到下，每个 FFT 数据分片会添加上报数据头部（灰色部分）后，作为消息的 Payload 进行上报，如图中左边所示。

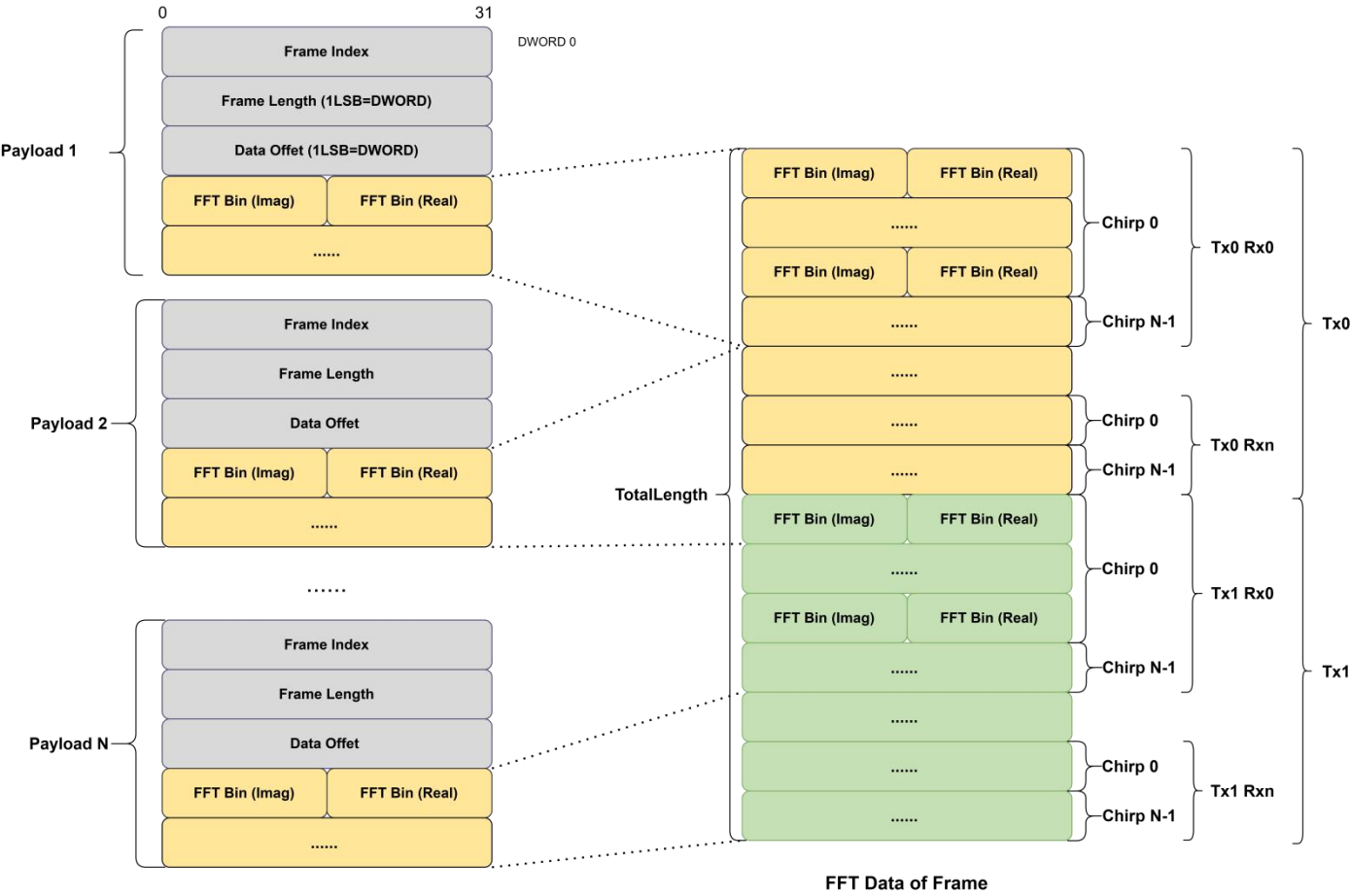


图 3-7 C1 FFT 数据消息

表 3-4 C1 FFT 数据消息 Payload 说明

Frame 参数	参数说明
Frame Index	当前帧序号，同一 Frame 内的多个数据 Payload 的帧序号相同，类型为 32 位无符号整型值。
Frame Length	雷达一帧完整的 FFT 数据总长度，单位 DWORD，当相同帧序号的消息 Payload 中的 FFT 数据长度总和（不包括上报数据头部）与该值相等时表示该帧的所有雷达上报数据传输完毕。
Data Offset	当前数据包的 FFT 数据在总数据中的偏移，单位 DWORD。
FFT Bin	FFT bin 数据大小为 DWORD(即 32bit)，imag 为虚部在低 16bit，real 为实部在高 16bit。实部和虚部的取值范围都为-32768~32767。

3.1.3.2.2 多类型 FFT 数据（0xC2）

雷达 FFT 数据上报使用 MsgId 为 0xC2 的 hif 消息，该消息的 Payload 定义如下图所示。图中右边为雷达一帧完整的 FFT 数据，其可能被拆分成多个数据分片进行传输，数据分片传输的顺序为从上到下，每个 FFT 数据分片会添加上报数据头部（灰色部分）后，作为消息的 Payload 进行上报，如图中左边所示。

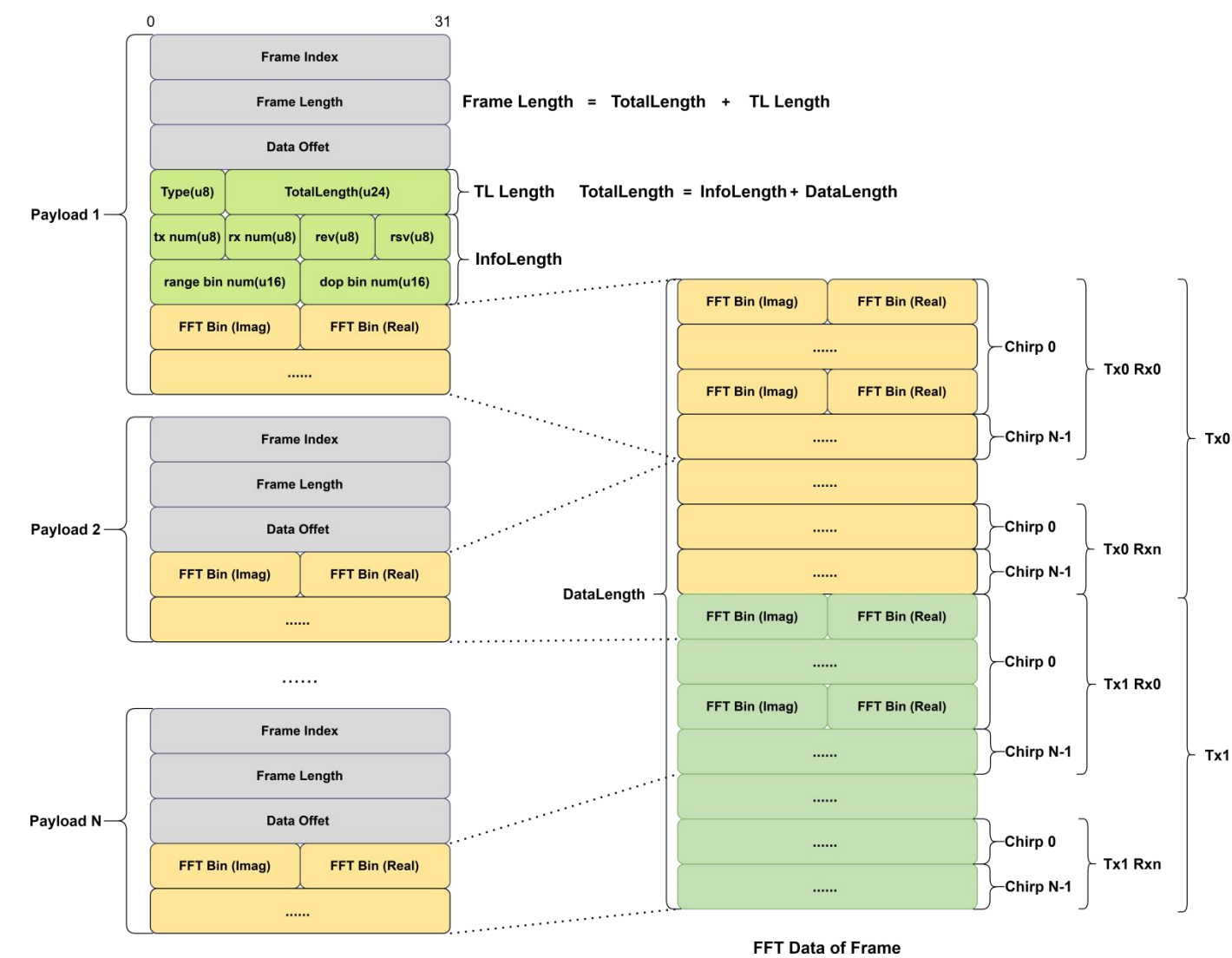


图 3-8 C2 FFT 数据消息

表 3-5 C2 FFT 数据消息 Payload 说明

Frame 参数	参数说明
Frame Index	当前帧序号，同一 Frame 内的多个数据 Payload 的帧序号相同，类型为 32 位无符号整型值。
Frame Length	雷达一帧完整的 FFT 数据总长度，单位 Byte，当相同帧序号的消息 Payload 中的 FFT 数据长度总和（包括 Info 绿色部分，但不包括上报数据头部）与该值相等时表示该帧的所有雷达上报数据传输完毕。
Data Offset	当前数据包的 FFT 数据在总数据中的偏移（包括 Info 绿色部分），单位 Byte。

Frame 参数	参数说明
Type(u8)	数据包的 type （0x0 ~ 0x2: 0~2 通路 datacube 数据）（小端）。
TotalLength(u24)	数据包的 payload 大小，单位 Byte，是 InfoLength 和 DataLength 的长度总和。
Tx_num(u8)	当前雷达配置的发送天线的个数。
Rx_num(u8)	当前雷达配置的接收天线的个数。
Rev(u8)	保留。
Rev(u8)	保留。
Range bin num(u16)	FFT Range num。
Dop bin num(u16)	FFT Dop num。
FFT Bin	FFT bin 数据大小为 DWORD(即 32bit)，imag 为虚部在低 16bit，real 为实部在高 16bit。实部和虚部的取值范围都为-32768~32767。

3.1.3.2.3 点云目标（0xC3）

雷达点云数据上报使用 MsgId 为 0xC3 的 hif 消息，该消息的 Payload 定义如下图所示。图中右边为雷达一帧完整的点云数据，其可能被拆分成多个数据分片进行传输，数据分片传输的顺序为从上到下，每个点云数据分片会添加上报数据头部（灰色部分）后，作为消息的 Payload 进行上报，如图中左边所示。

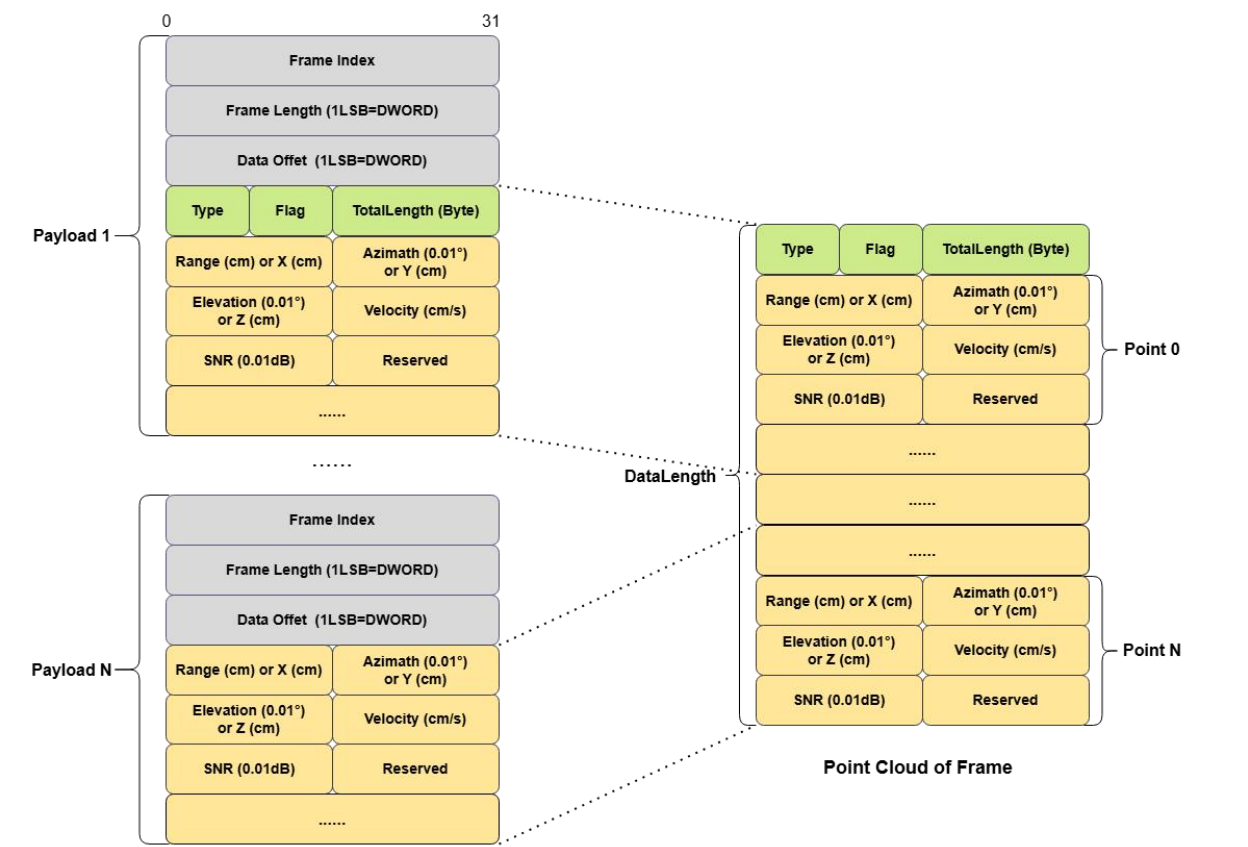


图 3-9 点云数据消息

表 3-6 点云数据消息 Payload 说明

Frame 参数	参数说明
Frame Index	当前帧序号，同一 Frame 内的多个数据 Payload 的帧序号相同，类型为 32 位无符号整型值。
Frame Length	雷达一帧完整的点云数据总长度，单位 DWORD，当相同帧序号的消息 Payload 中的点云数据长度总和（不包括上报数据头部）与该值相等时表示该帧的所有雷达上报数据传输完毕。
Data Offset	当前数据包的点云数据在总数据中的偏移，单位 DWORD。
Type	数据类型，8 位无符号整型值，点云数据固定为 4。
Flag	数据标志，8 位无符号整型值，bit0 为 0 时数据坐标系为极坐标，bit0 为 1 时数据坐标系为笛卡尔坐标。
Total Length	点云数据总长度，16 位无符号整型值，单位 Byte。
Range(cm) or X(cm)	极坐标时为 Range，单位 cm；笛卡尔坐标时为 X，单位 cm。
Azimuth(0.01°) or Y(cm)	极坐标时为 Azimuth，单位 0.01°；笛卡尔坐标时为 Y，单位 cm。
Elevation(0.01°) or Z(cm)	极坐标时为 Elevation，单位 0.01°；笛卡尔坐标时为 Z，单位 cm。
SNR	该点云的信噪比，单位 0.01dB。
Reserved	保留字

3.1.3.2.4 调试数据（0xC6）

SDK 在 HIF 的 Payload 字段内定义了称为“PSIC-调试协议”的子协议（ID：0xC6），专门用于上报灵活的调试信息，配合 RadarDebugTool，实现数据的可视化、存储和数据回放功能。

PSIC-调试协议具有以下关键特性：

- 属于 HIF 协议框架的子协议，开发者免于开发新的应用层协议；
- 支持 byte/short/word/float/double 等多种类型数据的上报，支持数组上报；
- 支持上报定点位宽，并在上位机还原定点位置；
- 支持定义不同的数据组织方法；
- 支持灵活的数据标签，以字符串类型区分每种数据类型；
- 至多支持 4090Bytes 数据上报；
- 支持字符串上报，并在 RadarDebugTool 中显示字符串。

PSIC-调试协议与 HIF 协议 Payload 字段关系如下图所示：

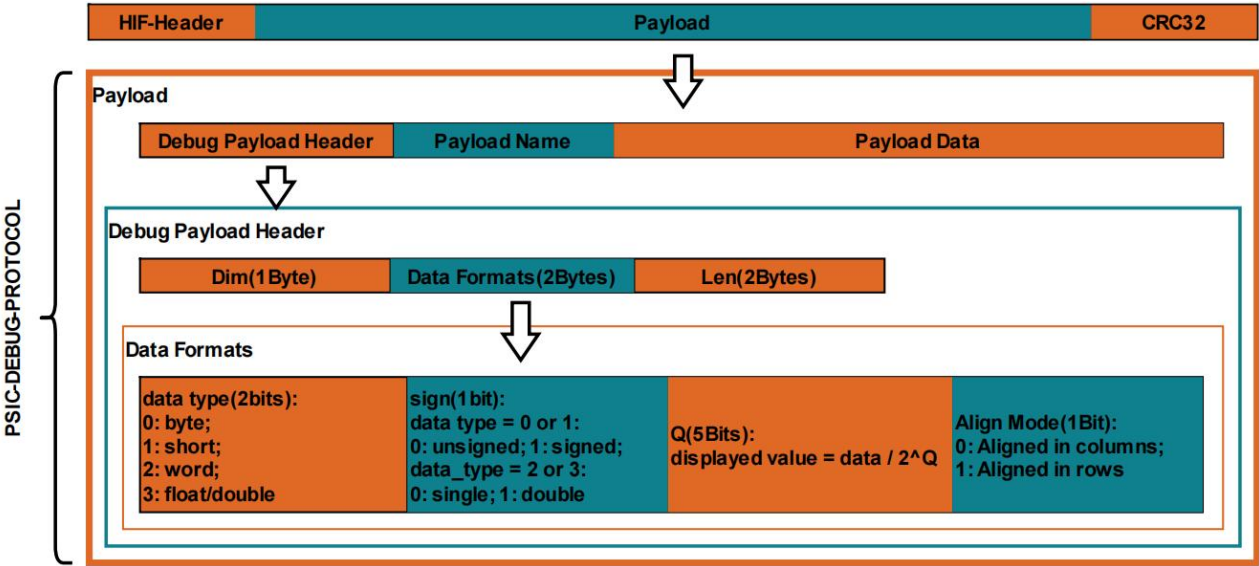


图 3-10 PSIC-调试协议 0xC6

表 3-7 PSIC-调试协议 0xC6 Payload 字段说明

0xC6 Payload 字段/参数	说明
Dim	指示需要上报的数据的维度。当 Dim = 1/2/3 时，RadarDebugTool 支持对数据的可视化。
Len	指示需要上报的多维数据的个数。Dim * Len 为上报数据的总个数。
Align Mode	数据对齐方式。 当 Align Mode=0 时，Payload Data 中数据优先填充 Len 个相同维度的数据，当某个维度数据填满后，再填充下一个维度的 Len 个数据，直到 Dim 维数据全部填充完成； 当 Align Mode = 1 时，Payload Data 中数据优先填充 Dim 个属于相同多维数据的不同维度的数据，再填充下一个多维数据的 Dim 个数据，直到 Len 个多维数据全部填充完成。
Q	定点化位宽： RadarDebugTool 在解析 Payload Data 数据后，会将数据除以 2 ^Q 进行显示和存储。这在上报的数据是定点化的数据时很有用，当上报数据为浮点类型时，一般需要将 Q 设置为 0。
Data Type	数据类型。 指示每一个元素的数据类型，支持的数据类型包括 byte/short/word/float(double)类型；

0xC6 Payload 字段/参数	说明
Sign	符号标志、浮点类型。 当 Data Type 为 byte/short/word 时, Sign=1, 表示 Data Payload 中所有为无符号数, Sign=0, 表示 Data Payload 中所有数据均在上位机被视作无符号数; 当 Data Type 为 float(double)类型时, Sign = 1, 表示上报类型为 double, Sign = 0, 表示上报类型为 float。
Payload Name	Payload Name 指示了该调试数据的标签, 这是一个字符串, 必须以'\0'作为结尾。此标签决定该段数据在 RadarDebugTool 上显示和存储的名称
Payload Data	根据 Align Mode/Dim/Len 填充数据

3.1.3.3 分片

当 MsgHeader 的 frag 字段为 1 时, 意味着这是一个分片帧 Frame, 是由一个大数据包分片得来。Host 需要将所有同属于一个大包的分片帧进行组包。

分片 Frame 的 Payload 结构如下:

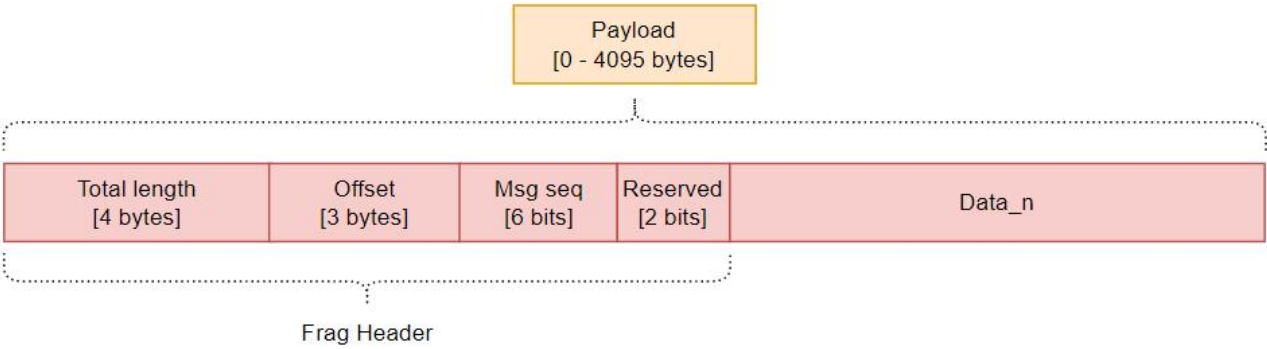


图 3-11 分片 Frame 的 Payload 结构

Frag Header, 即, 分片头, 用于分片控制, 各字段定义如下:

表 3-8 Frag Header 说明

Frag Header 字段	说明
Total length	所有分片数据 (Data_n) 的总长度。各分片的该字段都相同。
Offset	当前分片数据 (Data_n) 的偏移/位置。
Msg seq	消息序列号。各分片的该字段都相同。MsgHeader 的 MsgID 字段和 Msg seq 可共同确定该分片 Frame 的隶属。
reserved	保留位。

3.1.3.3.1 分片示例

假设: HIF 一帧 Frame 的 Payload 的最大长度为 4095Bytes, 待发送数据的长度为 8190Bytes, 则需要将待发送数据分成三片, 每片数据按协议封装成一个 Frame。

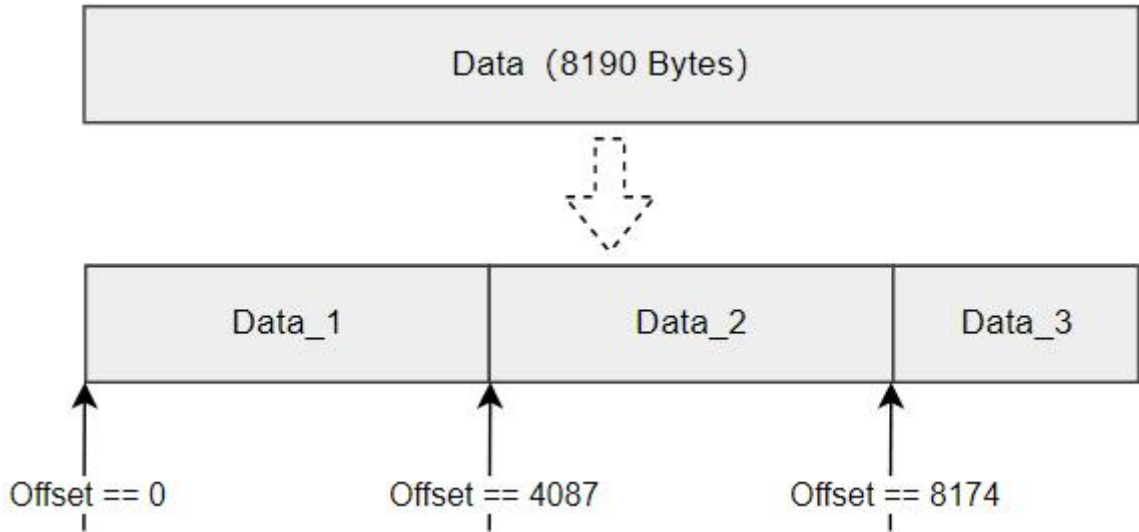


图 3-12 待发送数据分片示例

每个分片 Frame 的 Payload 如下图所示：

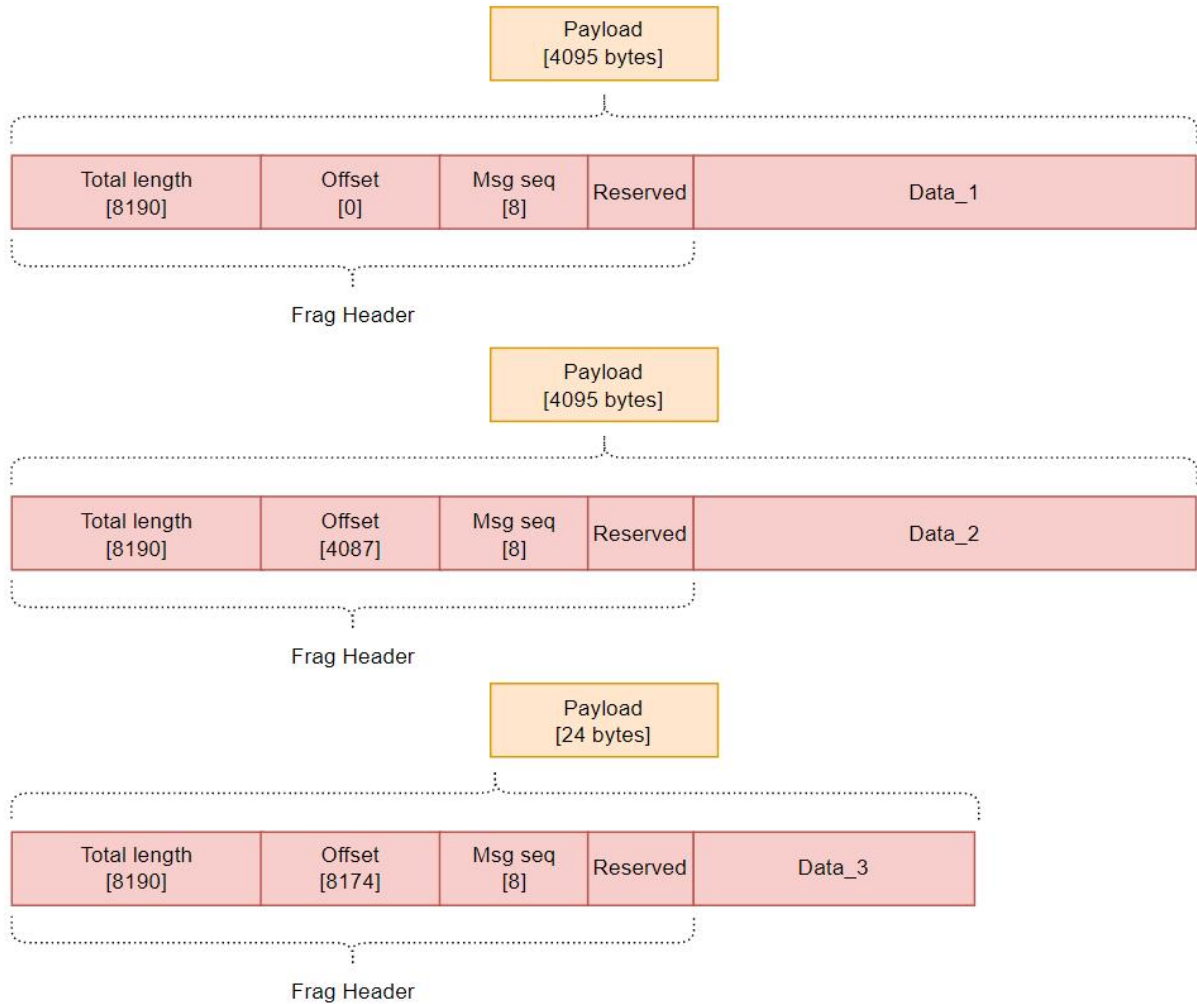


图 3-13 分片 Frame 的 Payload 示例

3.1.4 HIF 协议帧示例

一帧符合 HIF 协议帧格式的报文：

A5 2B 32 C2 0C 2F 00 00 00 0C ... 00 00 01 00 E8 F3 FA FF

Header: A5 2B 32 C2 0C 2F, 字段解析如下:

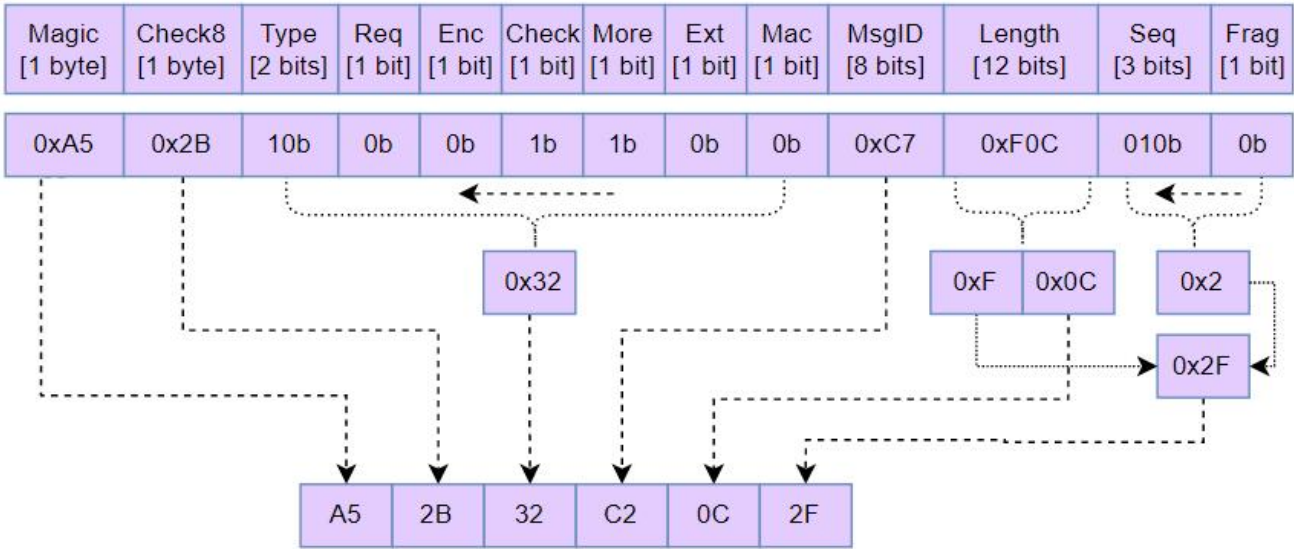


图 3-14 HIF Header 示例 (小端)

Payload: 00 00 00 0C ... 00 00 01 00。

Checksum: E8 F3 FA FF, 转为 uint32, 即, 0xFFFFAF3E8 (小端)。

3.2 Host 与 Device 交互

3.2.1 唤醒与休眠

Host 与 Device 进行 Command-Response 交互之前, Host 不能确定 Device 是否处于休眠状态, 因此需要在交互前唤醒 Device, 即唤醒子阶段; Command-Response 交互完成后需要允许 Device 休眠 (通过 HIF 命令 (MsgID=0x05) 交互实现), 即休眠释放子阶段。

唤醒与休眠交互流程如下:

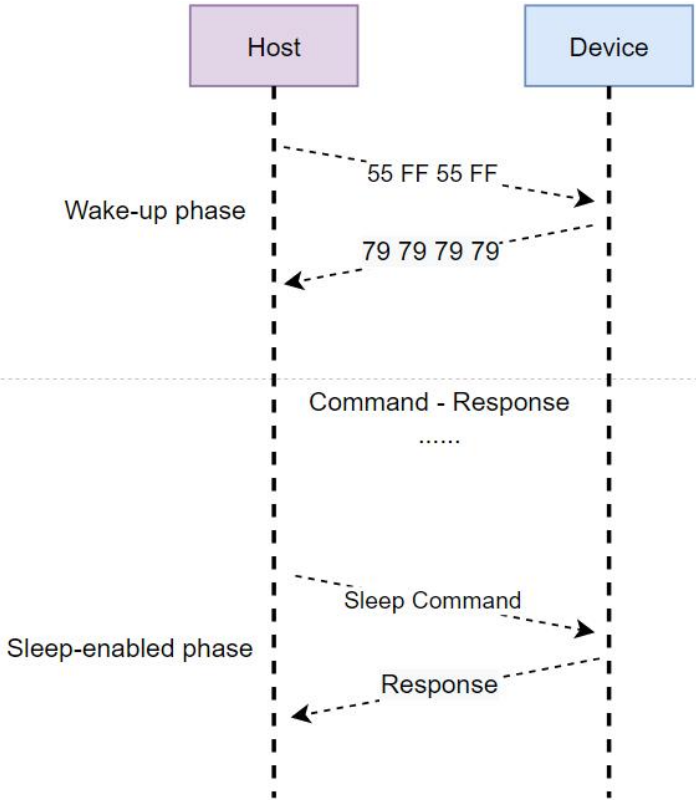


图 3-15 唤醒与休眠交互流程

3.2.1.1 唤醒

Host 通过通信唤醒 Device，唤醒数据如下：

wake sequence：

55 FF 55 FF

Device 被唤醒后，回复 79 79 79 79 给 Host。

ack sequence：

79 79 79 79

由于 Device 处于休眠状态，有概率收不到 Host 发送的唤醒数据，所以可能需要 Host 多尝试几次，直到 Device 回复 79 79 79 79，确认唤醒成功。

3.2.1.2 休眠

休眠 Command：ID - 0x05；Payload（1 Byte）- 0x01，为允许休眠；其他字段参照 HIF 帧格式。例如：

A5 CF 15 05 01 70 01 E9 FA FE 8F

Response：ID - 0x05；Payload（1 Byte）- 执行状态，见前述章节 [Command-Response](#)；其他字段参照 HIF 帧格式。例如：

A5 02 12 05 01 00 00 ED FA FE BF

这里，00 表示执行状态为 Success。

不同的物理通信方式的唤醒和休眠的时序流程如下。

3.2.1.3 SPI

3.2.1.3.1 唤醒

唤醒流程：

- 1. Host 发送 55 FF 55 FF；
- 2. Host 等待 1ms 读 4 个字节数据，4 个字节为 0x79 即为唤醒成功，Device 处于 Active 状态，唤醒流程结束。
- 3. 如果读到的 4 个字节不是 0x79，则等待 10ms 后再次循环 1、2 步骤。

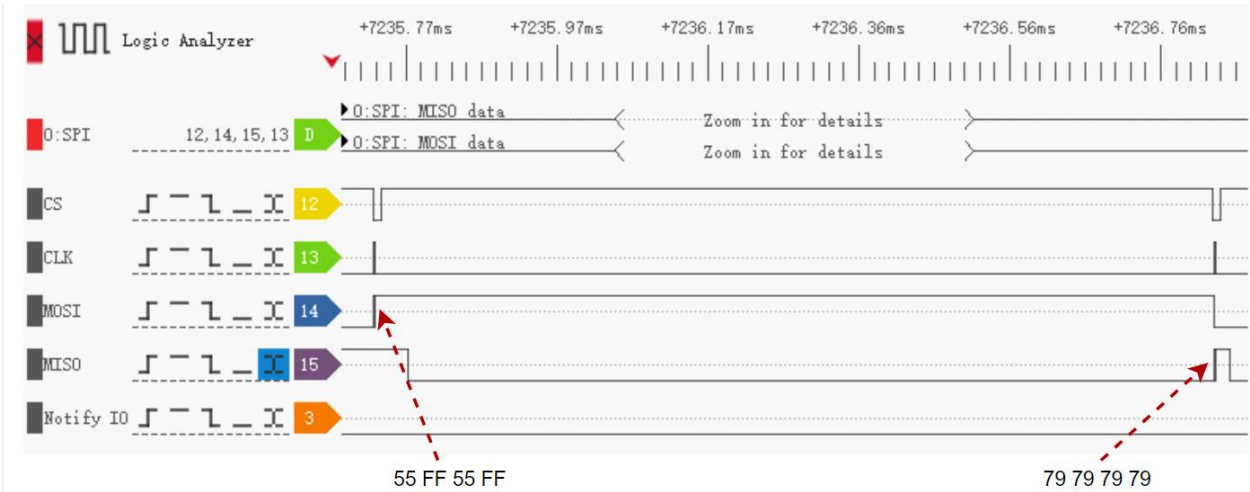


图 3-16 唤醒的 SPI 时序

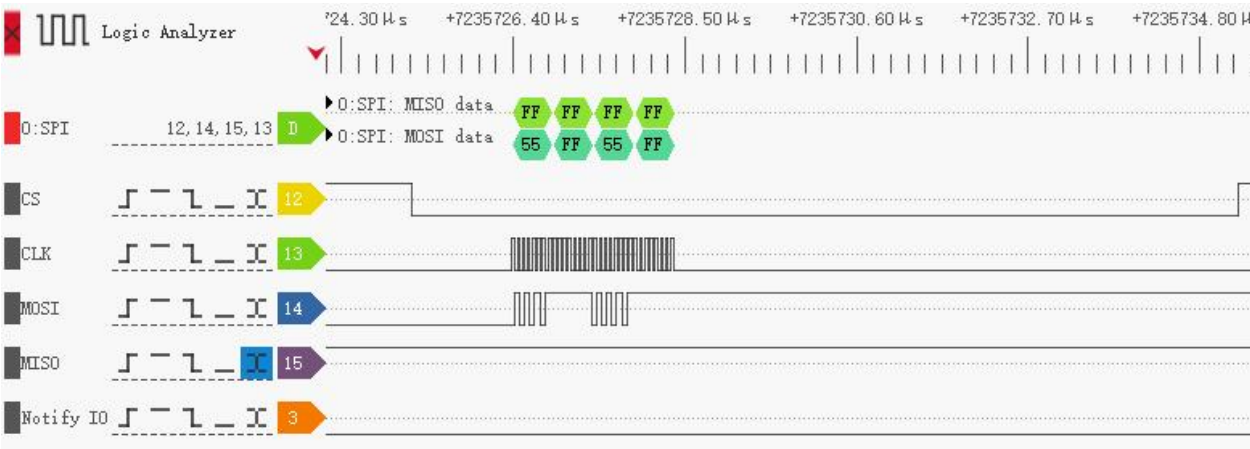


图 3-17 数据 55 FF 55 FF 的 SPI 时序图

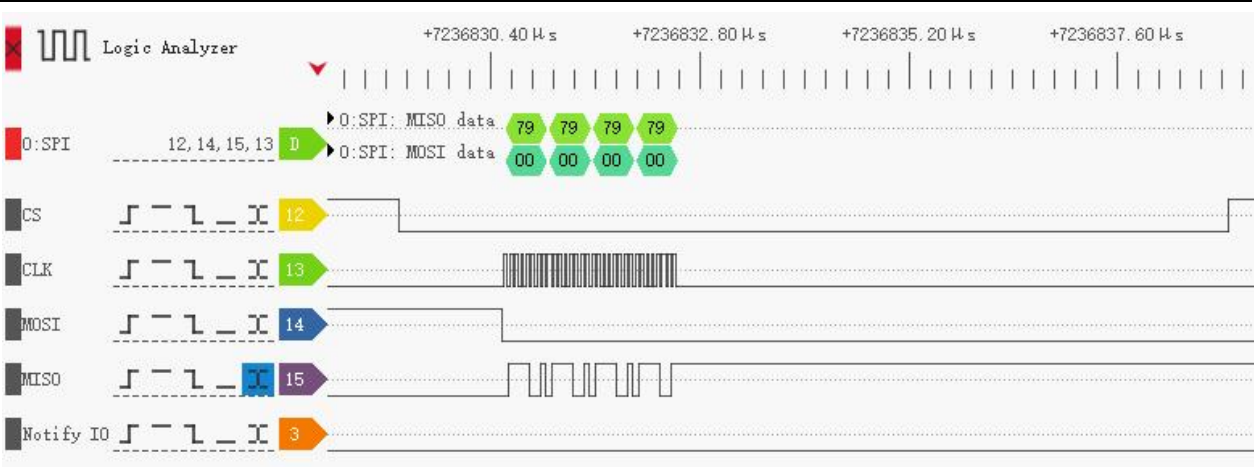


图 3-18 数据 79 79 79 79 的 SPI 时序图

3.2.1.3.2 休眠

休眠流程，其实是 Command-Response 流程。

本章节说明 SPI 在休眠 Command-Response 的数据时序。整体交互流程参考下方 Command-Response 章节。

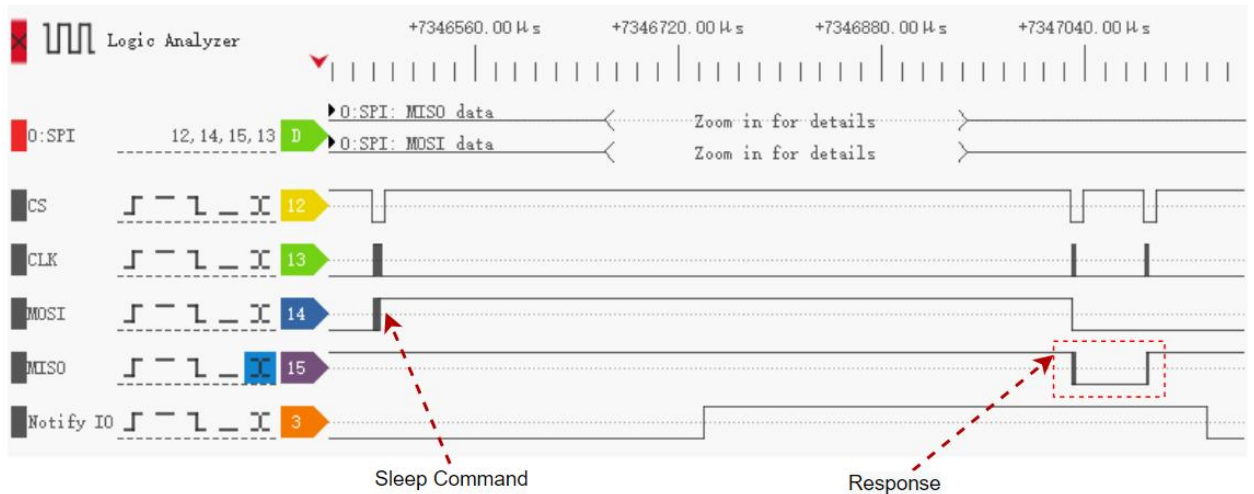


图 3-19 Command-Response 的 SPI 时序图

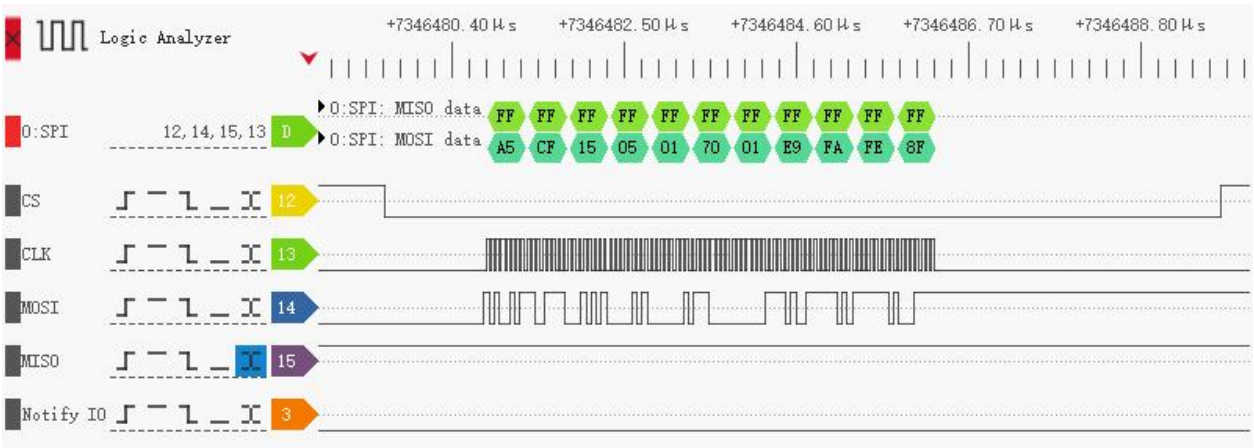


图 3-20 休眠 Command 数据的 SPI 时序

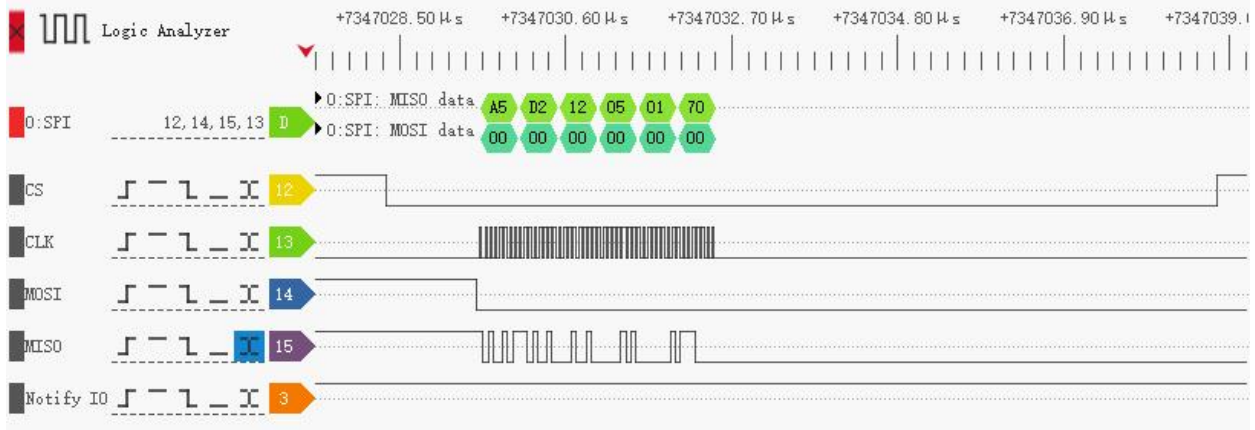


图 3-21 Response 数据 Header 的 SPI 时序

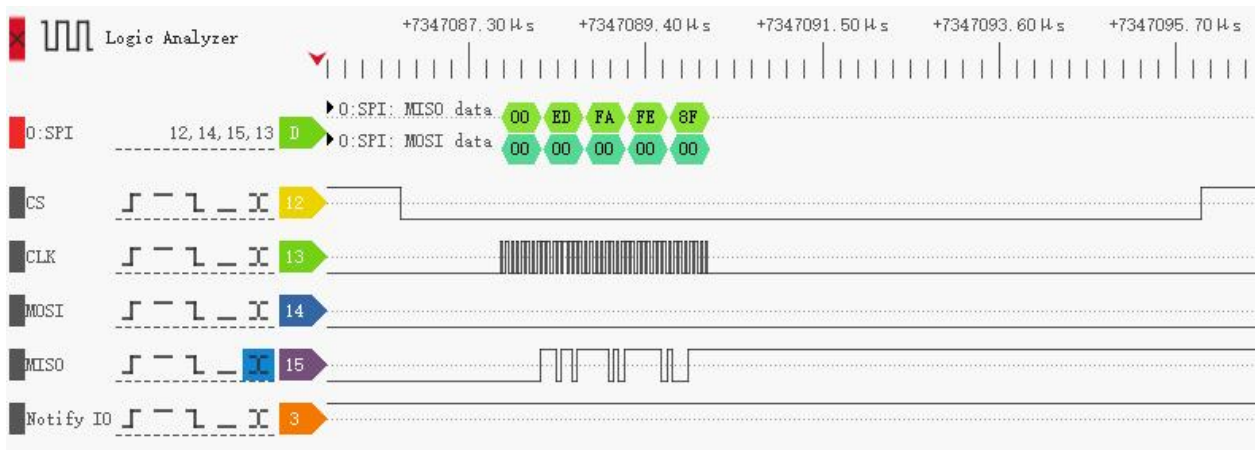


图 3-22 Response 数据 Payload+Checksum 的 SPI 时序

3.2.1.4 UART

3.2.1.4.1 唤醒

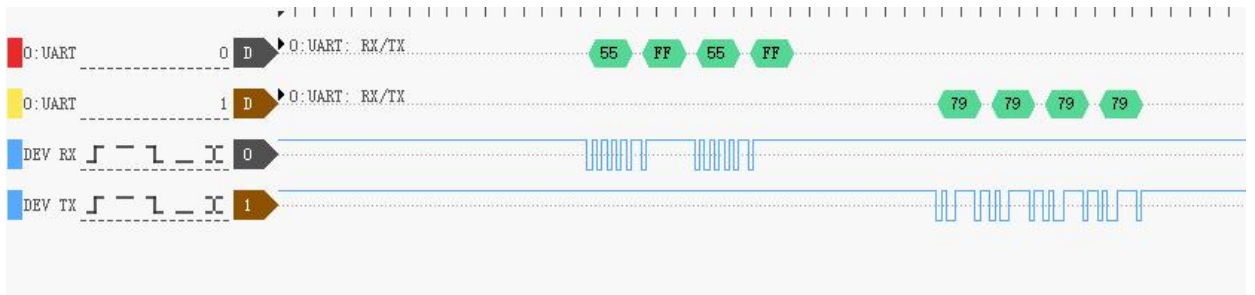


图 3-23 唤醒的 UART 时序

3.2.1.4.2 休眠



图 3-24 Command-Response 的 UART 时序

3.2.1.5 IIC通信

唤醒和休眠流程同 SPI。

3.2.2 Command-Response

前提，Device 处于 Active 状态；如果 Device 处于休眠状态，Host 需先唤醒 Device，见前述章节 [唤醒与休眠](#)。

格式：见前述章节 [HIF 协议帧格式](#) 中的 [Command-Response](#)。

交互：一般，Command-Response 的交互遵循成对出现、一发一回的原则，即，Host 发送一个 Command 给 Device，Device 收到后需要回复 Command 的执行状态和数据。此外，Command 不支持并发，即，Device 一次只会接收并处理一个 Command，只有当前 Command 处理结束且 Response 结束后，才会开始接收下一个 Command。

不同物理通信类型的 Command-Response 时序图说明如下。

3.2.2.1 SPI

其交互流程如下：

- ① Device 接收到 Host 的 Command；
- ② Device 进行数据处理；处理完成后，拉高 IO，通知 Host 可读取 Response 数据；
- ③ Host 读取数据；
- ④ 数据读取结束，Device 拉低 IO。

若第②步后，Host 迟迟未进行读取动作，则在超时（可配置）后，Device 丢弃数据。

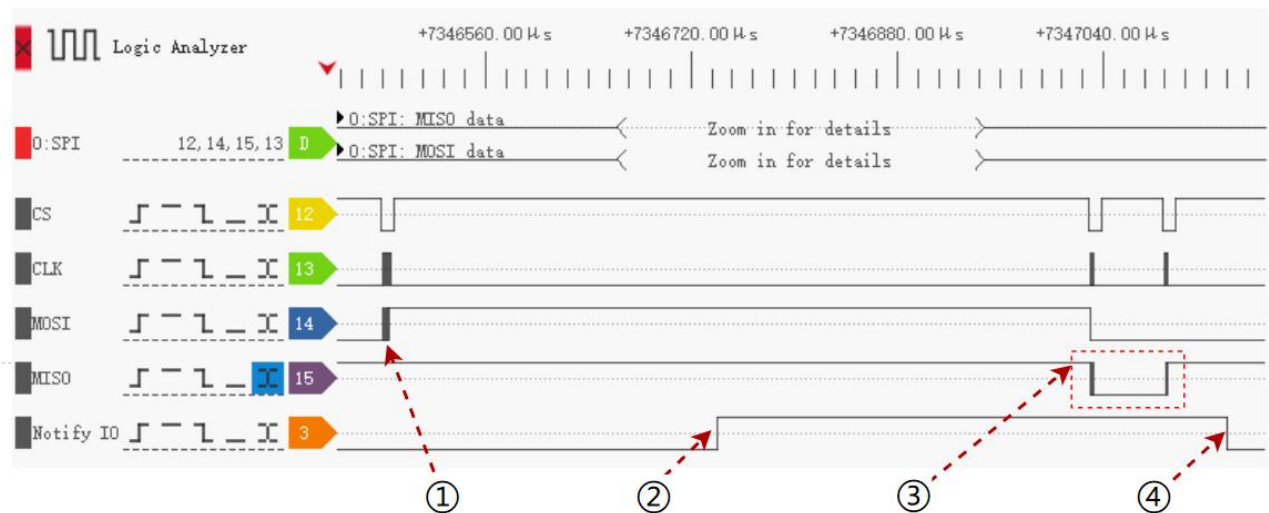


图 3-25 Command-Response 的 SPI 时序

3.2.2.2 UART

交互流程同前述 UART 休眠章节。

3.2.2.3 IIC

流程同 SPI。

3.2.3 数据上报

数据上报分为主动上报和被动上报。

主动上报，适用于主动类型的通信方式，比如：Uart。主动上报时，Device 直接将数据发送给 Host，并无太多交互动作。

被动上报，适用于被动类型的通信方式，比如：SPI、IIC。此外 UART 模式下，也可被动上报。被动上报的流程如下：

1. Device 拉高 Notify IO 唤醒 Host，指示 Device 有数据需要上报；
2. Host 发 Poll Command 给 Device；
3. Device 接收 Host 的 Poll Command 后拉低 Notify IO；
4. Device 准备数据，准备好后拉高 Notify IO，通知 Host 可以取数据；
5. Host 读取一帧数据；Host 检查数据帧的 Header-more 字段和 Notify IO：
 - (1) 若 more 为 1，Notify IO 为高，则重复步骤 5 及其之后的步骤；
 - (2) 若 more 为 1，Notify IO 为低，则返回 error；
 - (3) 若 more 为 0，延时 50us 后，Notify IO 为高，则返回步骤 2；
 - (4) 若 more 为 0，延时 50us 后，Notify IO 为低，则结束传输。

若第 4 步后，Host 迟迟未进行读取动作，则在超时后 Device 丢弃数据。

如果 Device 已将数据全部发送出去，最后会发一个 NullFrame（空包）。

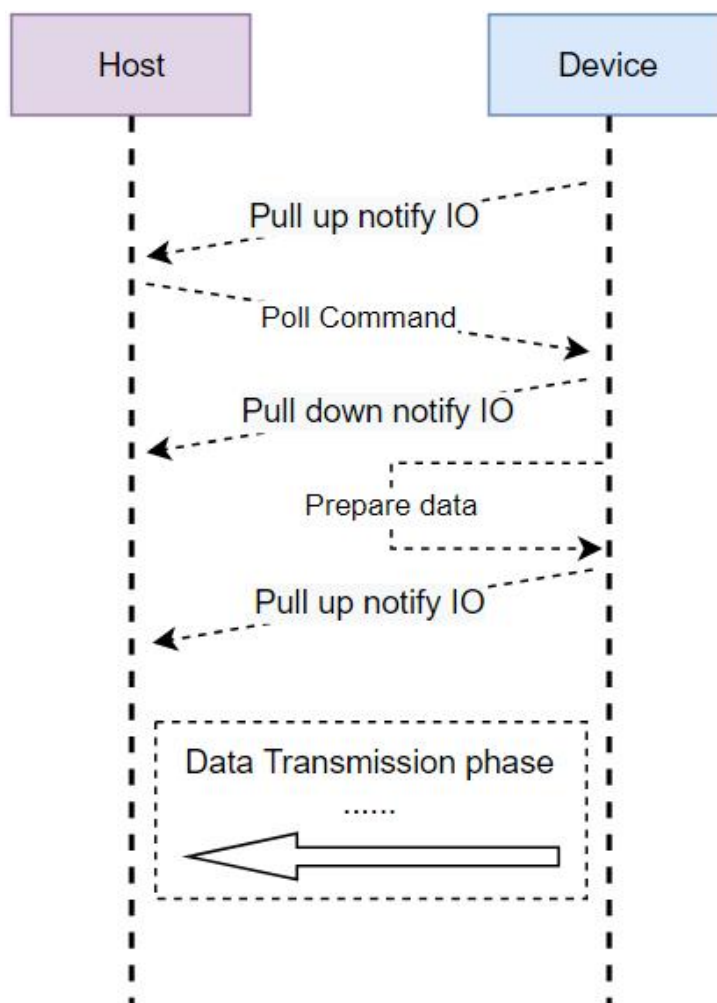


图 3-26 数据上报交互流程

3.2.3.1 SPI

SPI 上报数据时序：

- ① Device 拉高 Notify IO 唤醒 Host，指示 Device 有数据需要上报；
- ② Host 发 Poll Command 给 Device；
- ③ Device 接收 Host 的 Poll Command 后拉低 Notify IO；
- ④ Device 准备数据，准备好后拉高 Notify IO，通知 Host 可以取数据；
- ⑤ Host 读取一帧数据；Host 检查数据帧的 Header-more 字段和 Notify IO；
- ⑥ Device 的数据发送完成，拉低 IO。

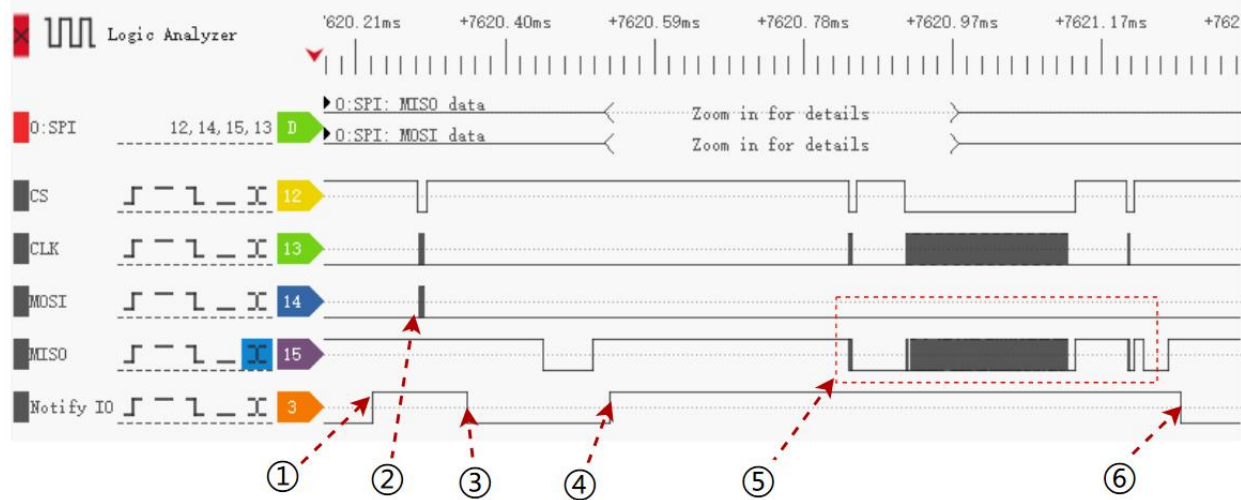


图 3-27 SPI 上报数据时序

3.2.3.2 UART

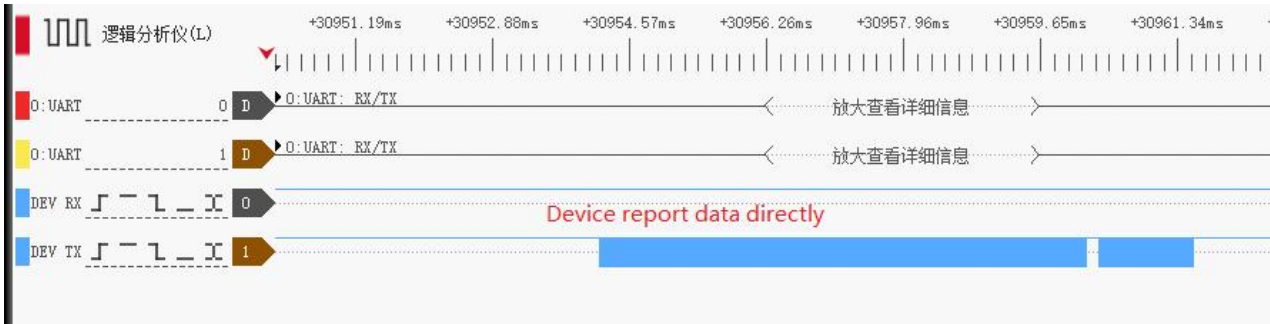


图 3-28 UART 上报数据时序

3.2.3.3 IIC

同 SPI。

3.2.3.4 传输层重传

当物理总线信号不稳定时，为了数据的正确传输，HIF 提供传输层重传功能。

传输层重传需要 Device 和 Host 都支持：Device 开启宏 CONFIG_HIF_RETRY_ENA；Host 开启宏 CFG_TL_RETRY_EN。具体见下述 Host 和 Device 章节。

3.2.4 异常恢复

3.2.4.1 Host接收上报数据时收不到消息头

Host 接收上报数据时，一般来说接收的第一个字节是 0xA5，并且收取的 6 个字节符合消息头的格式规范，但是当不规范的操作导致收取数据头时无法收到正确的消息头时，应该采用连续收取数据的方式进行状态同步，当收到 4KB 的 0xFF 或 0x00 时，认为 Host 与 Device 的状态同步成功。

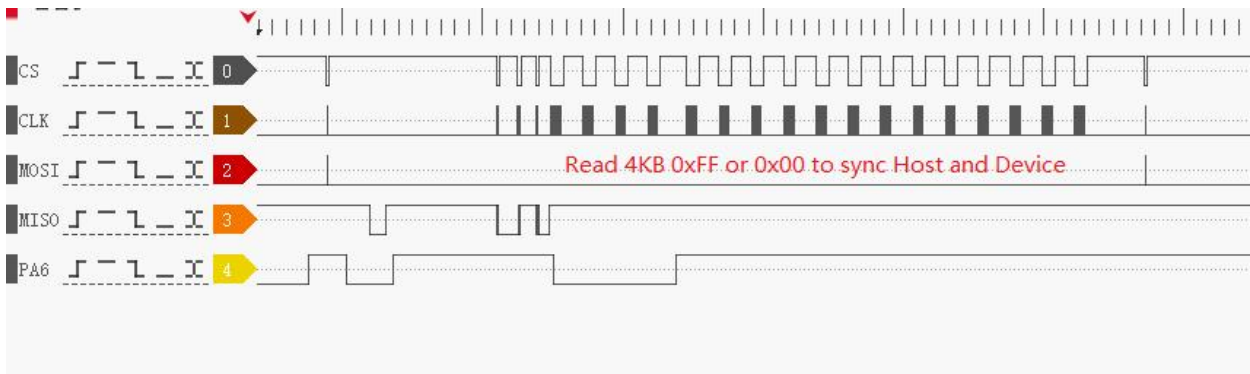


图 3-29 Host 与 Device 进行状态同步

4 软件框架

4.1 协议架构

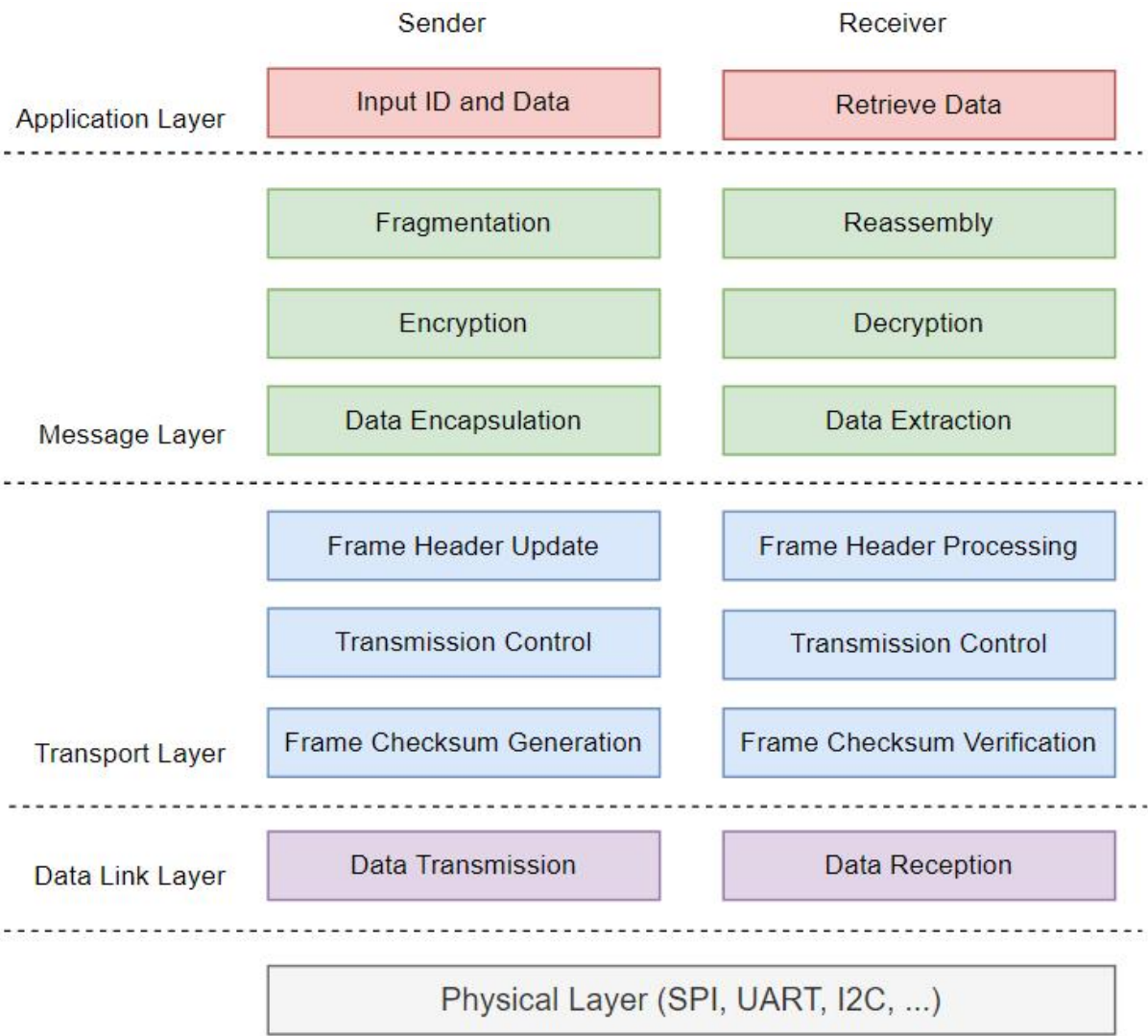


图 4-1 协议架构

4.1.1 消息层

主要功能：

- 向应用层提供消息发送功能：
 - 将 ID 与消息数据封装成 Frame；
 - 消息分片；
 - 加密（2.1.0 暂不支持）；
 - 调用传输层接口将 Frame 传递到传输层。
- 向应用层提供数据接收功能：

- 从传输层接收 Frame；
- 解密（2.1.0 暂不支持）；
- 将分片的消息重组；
- 将消息分发给应用层。

4.1.2 传输层

传输层主要功能是负责 HIF Frame 的传输控制,同时控制 Notify IO,负责唤醒和休眠、Command-Response 等交互。

- 传输 Frame：
 - 接收消息层传递的 Frame；
 - 计算 Seq、Check8、Checksum 等控制和校验字段；
 - 对所有 Frame 进行交互、重传、应答、超时处理等控制；
 - 将 Frame 传递给链路层。
- 接收 Frame
 - 接收链路层传递的 Frame；
 - 校验 CheckSum；
 - 提取 MsgHeader 和 Payload；
 - 将 MsgHeader 和 Payload 传递给消息层。

4.1.3 链路层

主要负责管理、控制物理层，并实现数据的收发。对于接收，链路层会校验 Frame 的 Header。

4.1.4 物理层

当前 SDK 版本的 HIF 支持 SPI、Uart、I2C 等通信方式，对于 SPI，支持 DMA。

4.2 软件架构

4.2.1 Host

HIF Host 端的软件架构如下图所示。分为 4 层，即硬件抽象层（HAL）、链路层（LLC）、中间件（MID）、应用层（APP），**厂商移植驱动实体（HAL 层接口）后即可直接使用 APP 层的接口实现功能。**

HAL 提供操作系统、GPIO、存储介质、总线 COM、日志、休眠的操作函数，其中总线 COM 需要按照一定的格式进行适配，将不同的总线抽象成 LLC 可用的总线控制接口集 ops。

LLC 内部定义了 COM_BUS 和 COM_DEVICE，分别对总线和设备进行管理，最终会将不同的设备抽象成为两种不同通知类型的设备，即事件通知和数据通知，如果设备是被动上报，则是事件通知，如果设备是主动上报，则是数据通知。

MID 基于 LLC 的事件通知和数据通知以及数据发送功能，实现了 HOST HIF 通信及 BROM 下载协议。
APP 整合下层提供的功能函数并简化操作，提供简便的 API 以使用。

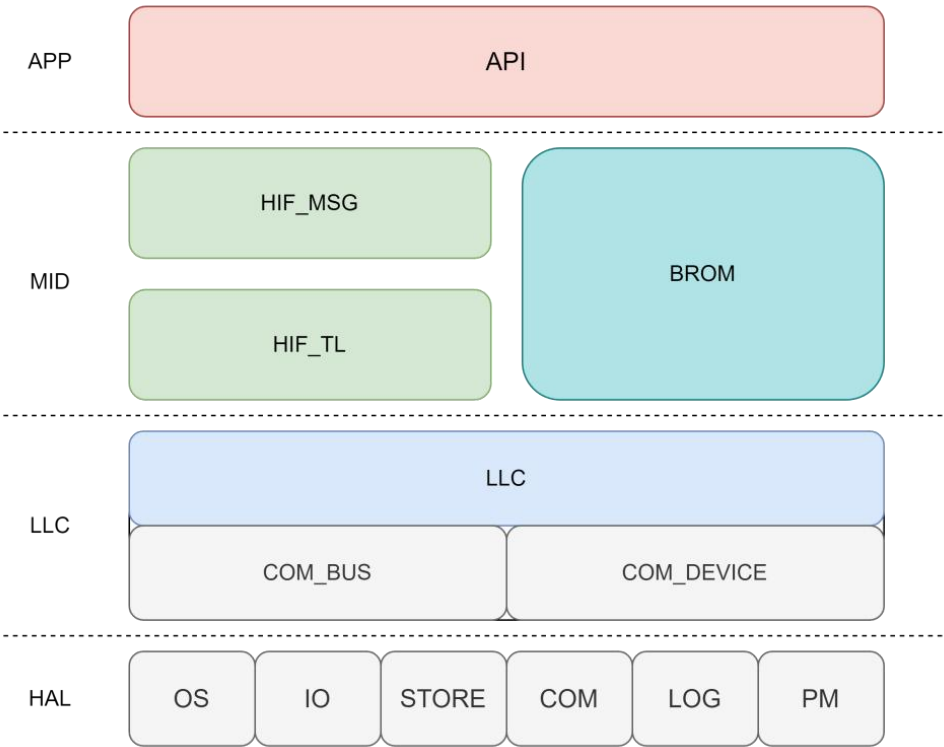


图 4-2 Host 软件架构

4.2.2 Device

HIF Device 端的软件架构如下图所示。各层级或模块的功能基本和协议架构一致，其中 HIF MEMORY 是 Device 端 HIF 的内存资源管理模块。

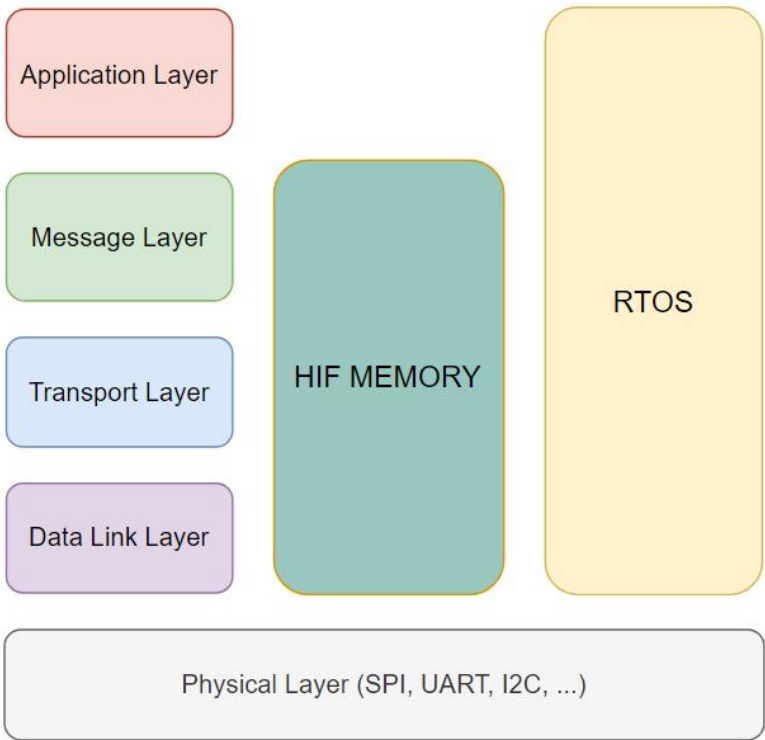


图 4-3 Device 软件架构

5 Host 开发

5.1 Host Driver 移植

Host Driver 能够实现单从机或多从机通信，多从机仅需在单从机的基础上额外注册、配置、打开其它从机便可实现，支持不同总线的从机同时使用。

如果厂商文件夹命名为 AAA，则需要编译如下文件，host_driver 文件夹路径为

“<home>/Software_Kit/02_SDK/psdf_sdk/project/subsys/hif/host_sample/host_driver”，其中路径以 “/” 结尾表示文件夹，否则表示文件。

```
host_driver/host_config.h
host_driver/user_config.h
host_driver/cmd_api/
host_driver/porting/host_port.h
host_driver/porting/include/
host_driver/porting/AAA/
host_driver/protocol/
```

5.1.1 文件结构介绍

```
<home>/Software_Kit/02_SDK/psdf_sdk/project/subsys/hif/host_sample/host_driver
├─ host_config.h           /* host driver 配置文件 */
├─ user_config.h          /* 用户配置文件，能够覆盖 host_config.h 的配置项 */
├─ cmd_api                /* 应用层文件目录 */
│   └─ host_driver.c      /* host driver 初始化、设备注册与开启、通用命令下发、
固件下载等功能 */
│   └─ host_driver.h      /* 声明 */
│   └─ rs6x_7x            /* rs6x_7x 的命令下发与数据上报处理文件目录 */
│       └─ host_mmw_api.h  /* 声明 */
│       └─ host_mmw_cmd_api.c /* 通用命令和 radar analysis 命令 */
│       └─ host_mmw_report_api.c /* C1/C2/C3 数据组包和 C6/C7/C8/CA/CB 数据包回调 */
├─ porting                /* 适配层文件目录 */
│   └─ host_port.h        /* 适配层头文件 */
│   └─ include            /* 适配层接口定义文件目录 */
│       └─ hal_com.h       /* 通信接口集定义 */
│       └─ hal_io.h        /* IO 操作接口定义 */
│       └─ hal_log.h       /* 日志接口 */
```

└─ hal_os.c	/* 部分系统操作接口的虚函数定义 */
└─ hal_os.h	/* 系统操作接口定义 */
└─ hal_store.h	/* 存储介质接口定义 */
└─ psic	/* MRS6130_P1812 移植示例文件目录 */
└─ port.c	/* 提供 porting_get_com_ops 函数实体给 LLC 层使用 */
└─ port_io.c	/* IO 适配层接口实体 */
└─ port_log.h	/* 提供 host_log_print 函数实体 */
└─ port_os.c	/* 提供系统操作接口实体 */
└─ port_store.c	/* 提供存储介质接口实体 */
└─ port_com_no_alloc.c	/* 无内存申请的 COM 适配层，测试用 */
└─ port_i2c.c	/* I2C 的 COM 适配层 */
└─ port_spi.c	/* SPI 的 COM 适配层 */
└─ port_uart_com_irq_thread.c	/* UART 主动上报 COM_IRQ_THREAD 通知类型的 COM 适配层 */
└─ port_uart_com_isr.c	/* UART 主动上报 COM_ISR 通知类型的 COM 适配层 */
└─ port_uart_edge.c	/* UART 被动上报 EDGE 通知类型的 COM 适配层 */
└─ protocol	/* 协议层文件目录 */
└─ burn	/* 下载协议文件目录 */
└─ host_brom.c	/* AS2525 下载协议 */
└─ host_brom.h	/* AS2525 下载协议 */
└─ host_brom_comm_struct.h	/* AS2525 下载协议使用的命令结构定义 */
└─ host_burn.c	/* 固件下载功能的框架 */
└─ host_burn.h	/* 固件下载功能的框架 */
└─ host_img_struct.h	/* 固件结构定义 */
└─ hif	/* HIF 传输层和 HIF 消息层文件目录 */
└─ hif_type.h	/* HIF 消息结构定义 */
└─ host_hif.c	/* HIF 消息层 (MSG) */
└─ host_hif.h	/* HIF 消息层 (MSG) */
└─ host_hif_tl.c	/* HIF 传输层 (TL) */
└─ host_hif_tl.h	/* HIF 传输层 (TL) */
└─ llc	/* 链路层文件目录 */
└─ host_llc.c	/* 链路层 (LLC) */
└─ host_llc.h	/* 链路层 (LLC) */

└─ host_llc_com_bus.c	/* 总线抽象层 */
└─ host_llc_com_bus.h	/* 总线抽象层 */
└─ host_llc_com_device.c	/* 设备抽象层 */
└─ host_llc_com_device.h	/* 设备抽象层 */
└─ utils	/* 基本数据类型、链表、错误码、常用宏函数 */
└─ host_slist.h	/* 链表 */
└─ host_std_includes.h	/* 基本数据类型 */
└─ host_types.h	/* 错误码、常用宏函数 */

host driver 代码包主要有 3 个文件夹和 2 个文件，porting 文件夹是移植相关的内容，对应代码框架中的 HAL，protocol 文件夹是协议相关的内容，对应代码框架中的 LLC 和 MID，cmd_api 文件夹是对厂商开放使用的 API，对应代码框架中的 APP。host_config.h 和 user_config.h 文件是 host driver 的全局配置文件，其中 user_config.h 文件中的配置能够覆盖 host_config.h 的配置，厂商可以在 user_config.h 中进行调整。厂商需要完成 porting 的移植，其中 porting/psic 为 MRS6130_P1812 的移植代码。厂商完成移植后，可以使用 cmd_api 文件夹提供的 API 实现与 Device 的交互。

5.1.2 移植内容

需要移植的接口声明在 porting/include 文件夹中，可在 porting 目录下创建厂商的移植代码文件夹。
如果厂商硬件在使用 DMA 或其它内容时需要内存对齐，可以修改代码包根目录下的文件 host_config.h 的宏 `_ALIGNAS_VARIABLE`，该宏用于变量的内存对齐，如果不需要对齐，请将该宏定义为空白宏。

5.1.2.1 系统接口

5.1.2.1.1 临界区保护

host_os_critical_flag_t 变量需要厂商根据实际情况调整，默认定义为 unsigned long 类型，如无 flag，在函数实体中不使用该参数即可。

表 5-1 host_os_enter_critical

接口	说明
函数	host_os_critical_flag_t host_os_enter_critical(void);
功能	进入临界区
参数	无
注意事项	host_os_critical_flag_t 默认为 unsigned long 类型，可修改
返回值	host_os_critical_flag_t，用于退出临界区时恢复状态

表 5-2 host_os_exit_critical

接口	说明
----	----

函数	void host_os_exit_critical(host_os_critical_flag_t flag);
功能	退出临界区
参数	host_os_critical_flag_t flag 进入临界区的返回值
注意事项	host_os_critical_flag_t 默认为 unsigned long 类型，可修改
返回值	无

5.1.2.1.2 时间戳

表 5-3 host_os_timestamp_us

接口	说明
函数	host_os_time_t host_os_timestamp_us(void);
功能	获取时间戳，单位微秒
参数	无
注意事项	无
返回值	host_os_time_t，时间戳，默认使用 u32，可修改

5.1.2.1.3 内存操作

表 5-4 host_os_malloc

接口	说明
函数	void * host_os_malloc(uint32_t size);
功能	申请内存
参数	uint32_t size 申请的内存大小，单位字节
注意事项	无
返回值	内存地址，无可用内存时返回 NULL

表 5-5 host_os_free

接口	说明
函数	void host_os_free(void *pmem);
功能	释放内存
参数	void *pmem 待释放内存地址

注意事项	无
返回值	无

表 5-6 host_os_memset

接口	说明
函数	<code>void * host_os_memset(void *addr, uint8_t val, uint32_t size);</code>
功能	内存数据设置
参数	<code>void *addr</code> 待设置内存地址 <code>uint8_t val</code> 设置值 <code>uint32_t size</code> 设置的长度，单位字节
注意事项	无
返回值	内存地址，与传入的内存地址一致

表 5-7 host_os_memcpy

接口	说明
函数	<code>void * host_os_memcpy(void *dst, const void *src, uint32_t size);</code>
功能	内存数据拷贝
参数	<code>void *dst</code> 拷贝的目标内存地址 <code>const void *src</code> 拷贝的源内存地址 <code>uint32_t size</code> 拷贝长度，单位字节
注意事项	无
返回值	内存地址，与传入的目标内存地址一致

表 5-8 host_os_memcmp

接口	说明
----	----

函数	<code>int host_os_memcmp(const void *addr1, const void *addr2, uint32_t size);</code>
功能	内存数据比较
参数	<code>const void *addr1</code> 待比较的内存地址 1 <code>const void *addr2</code> 待比较的内存地址 2 <code>uint32_t size</code> 比较长度，单位字节
注意事项	无
返回值	<code>>0</code> 内存地址 1 的数据更大 <code>=0</code> 两个内存地址的数据相等 <code><0</code> 内存地址 2 的数据更大

5.1.2.1.4 线程

表 5-9 host_os_thread_create

接口	说明
函数	<code>host_os_thread_handle_t host_os_thread_create(const char *pname, uint32_t stack_size, host_os_thread_entry_t entry, void *arg, int32_t prio);</code>
功能	创建一个线程
参数	<code>const char *pname</code> 线程名称 <code>uint32_t stack_size</code> 线程栈大小 <code>host_os_thread_entry_t entry</code> 线程入口函数地址，类型为 <code>void (*)(void *)</code> <code>void *arg</code> 线程入口函数的参数

	int32_t prio 线程优先级
注意事项	无
返回值	host_os_thread_handle_t 线程句柄，失败返回 NULL，成功返回其它

表 5-10 host_os_thread_delete

接口	说明
函数	int host_os_thread_delete(host_os_thread_handle_t thread_hdl);
功能	删除一个线程
参数	host_os_thread_handle_t 线程句柄
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

表 5-11 host_os_thread_is_valid

接口	说明
函数	bool host_os_thread_is_valid(host_os_thread_handle_t thread_hdl);
功能	通过线程句柄判断线程是否有效
参数	host_os_thread_handle_t 线程句柄
注意事项	无
返回值	true 有效，false 无效

5.1.2.1.5 信号量

表 5-12 host_os_sem_create

接口	说明
函数	host_os_sem_t host_os_sem_create(uint32_t initial_count, uint32_t limit);
功能	创建一个信号量
参数	uint32_t initial_count 初始值 uint32_t limit 最大值

注意事项	无
返回值	host_os_sem_t 信号量句柄，失败返回 NULL，成功返回其它

表 5-13 host_os_sem_delete

接口	说明
函数	int host_os_sem_delete(host_os_sem_t psem);
功能	删除一个信号量
参数	host_os_sem_t psem 信号量句柄
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

表 5-14 host_os_sem_is_valid

接口	说明
函数	int host_os_sem_is_valid(host_os_sem_t psem);
功能	判断信号量是否有效
参数	host_os_sem_t psem 信号量句柄
注意事项	无
返回值	0 无效，其它有效，推荐返回 1 表示有效

表 5-15 host_os_sem_take

接口	说明
函数	int host_os_sem_take(host_os_sem_t psem, host_os_timeout_t timeout);
功能	获取信号量
参数	host_os_sem_t psem 信号量句柄 host_os_timeout_t timeout 超时时间
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

表 5-16 host_os_sem_give

接口	说明
函数	int host_os_sem_give(host_os_sem_t psem);
功能	释放信号量
参数	host_os_sem_t psem 信号量句柄
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

5.1.2.1.6 延时函数

表 5-17 host_os_delayms

接口	说明
函数	int host_os_delayms(uint32_t ms);
功能	毫秒级延时
参数	uint32_t ms 延时时间，单位毫秒
注意事项	要求为非忙等，允许系统调度
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

表 5-18 host_os_delayus

接口	说明
函数	int host_os_delayus(uint32_t us);
功能	微秒级延时
参数	uint32_t us 延时时间，单位微秒
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

5.1.2.1.7 休眠控制

表 5-19 host_os_pm_lock

接口	说明
函数	void host_os_pm_lock(void);

功能	锁住休眠/关闭休眠/禁止休眠
参数	无
注意事项	直接锁住休眠
返回值	无

表 5-20 host_os_pm_unlock

接口	说明
函数	void host_os_pm_unlock(void);
功能	解锁休眠/开启休眠/允许休眠
参数	无
注意事项	直接解锁休眠
返回值	无

5.1.2.2 IO接口

IO 接口为 GPIO 控制接口，在 host driver 中需要控制复位和升级引脚、读取通知 IO 的电平、响应通知 IO 边沿或电平中断（电平中断暂不支持）。

IO 接口在控制 RST IO 复位 Device 时，默认输出高电平表示使能 Device 运行，输出低电平表示失能 Device 运行，如果需要使用 VCC 复位方案，应确保高电平供电、低电平断电。

如果是低电平供电、高电平断电，可以在适配 host_io_write 接口时，将相应的 RST IO 输出高电平映射成输出低电平、输出低电平映射成输出高电平。

表 5-21 host_io_mode_set

接口	说明
函数	int host_io_mode_set(uint32_t io, host_io_mode_t mode, host_io_pull_t pull, bool is_wkio);
功能	设置 GPIO 模式，需支持输入、输出、中断模式和上拉、下拉、浮空模式
参数	uint32_t io GPIO 编号 host_io_mode_t mode GPIO 模式 host_io_pull_t pull GPIO 上下拉 bool is_wkio

	当前配置的是 WKIO true, 是 false, 否
注意事项	如果需要休眠功能, 必须支持 WKIO, 即 IO 唤醒
返回值	HOST_ERRCODE_SUCCESS 成功, 其它失败

表 5-22 host_io_irq_cfg

接口	说明
函数	int host_io_irq_cfg(uint32_t io, host_io_irq_trig_t trig, host_io_irq_callback_t pcallback, void *arg, bool is_wkio);
功能	设置 GPIO 中断触发方式和回调函数
参数	uint32_t io GPIO 编号 host_io_irq_trig_t trig 中断触发方式 typedef enum { HOST_IO_IRQ_EDGE_RISING, HOST_IO_IRQ_EDGE_FALLING, HOST_IO_IRQ_EDGE_BOTH, HOST_IO_IRQ_LEVEL_HIGH, HOST_IO_IRQ_LEVEL_LOW, } host_io_irq_trig_t; host_io_irq_callback_t pcallback 中断回调函数, 类型为 void (*)(void *) void *arg 函数参数 bool is_wkio 当前配置的是 WKIO true, 是 false, 否
注意事项	如果需要休眠功能, 必须支持 WKIO, 即 IO 唤醒
返回值	HOST_ERRCODE_SUCCESS 成功, 其它失败

表 5-23 host_io_irq_enable

接口	说明
函数	int host_io_irq_enable(uint32_t io, bool is_wkio);
功能	使能 GPIO 中断
参数	uint32_t io 待使能 GPIO 编号 bool is_wkio 当前配置的是 WKIO true, 是 false, 否
注意事项	如果需要休眠功能, 必须支持 WKIO, 即 IO 唤醒
返回值	HOST_ERRCODE_SUCCESS 成功, 其它失败

表 5-24 host_io_irq_disable

接口	说明
函数	int host_io_irq_disable(uint32_t io, bool is_wkio);
功能	失能 GPIO 中断
参数	uint32_t io 待使能 GPIO 编号 bool is_wkio 当前配置的是 WKIO true, 是 false, 否
注意事项	如果需要休眠功能, 必须支持 WKIO, 即 IO 唤醒
返回值	HOST_ERRCODE_SUCCESS 成功, 其它失败

表 5-25 host_io_write

接口	说明
函数	void host_io_write(uint32_t io, host_io_value_t value, bool is_wkio);
功能	设置 GPIO 输出电平
参数	uint32_t io GPIO 编号

	host_io_value_t value 电平值 typedef enum { HOST_IO_VALUE_LOW, HOST_IO_VALUE_HIGH, } host_io_value_t; bool is_wkio 当前配置的是 WKIO true, 是 false, 否
注意事项	无
返回值	无

表 5-26 host_io_read

接口	说明
函数	host_io_value_t host_io_read(uint32_t io, bool is_wkio);
功能	读取 GPIO 输入电平
参数	uint32_t io GPIO 编号
注意事项	如果需要休眠功能，必须支持 WKIO，即 IO 唤醒
返回值	电平值 typedef enum { HOST_IO_VALUE_LOW, HOST_IO_VALUE_HIGH, } host_io_value_t; bool is_wkio 当前配置的是 WKIO true, 是 false, 否

5.1.2.3 存储介质接口

存储介质接口用于 Host 获取 Device 固件。

表 5-27 host_store_open

接口	说明
----	----

函数	<code>int host_store_open(void **hdl, void *path, uint32_t *len);</code>
功能	打开一个固件
参数	<code>void **hdl</code> 固件指针，为输出参数 <code>void *path</code> 固件路径 <code>uint32_t *len</code> 固件大小指针，为输出参数，单位字节
注意事项	如果接口返回 <code>HOST_ERRCODE_SUCCESS</code> ，需要为 <code>hdl</code> 和 <code>len</code> 参数赋值，并且 <code>hdl</code> 不能是 <code>NULL</code>
返回值	<code>HOST_ERRCODE_SUCCESS</code> 成功，其它失败

表 5-28 host_store_close

接口	说明
函数	<code>void host_store_close(void *hdl);</code>
功能	关闭一个固件
参数	<code>void *hdl</code> 固件
注意事项	无
返回值	<code>HOST_ERRCODE_SUCCESS</code> 成功，其它失败

表 5-29 host_store_read

接口	说明
函数	<code>int host_store_read(void *hdl, uint32_t offset, void *pbuff, uint32_t size);</code>
功能	读取固件数据
参数	<code>void *hdl</code> 固件 <code>uint32_t offset</code> 偏移量，单位字节 <code>void *pbuff</code> 接收数据的 buffer

	uint32_t size 数据大小，单位字节
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

5.1.2.4 日志接口

日志接口需要实现格式化打印，建议在头文件中定义宏函数 `host_log_print(fmt, ...)` 实现，然后在文件 `user_config.h` 定义宏 `CFG_HOST_PORT_LOG_FILE`，这个宏应修改为定义/声明 `host_log_print` 函数的文件相对 `porting/include/hal_log.h` 文件的路径。

例如，`host_log_print` 定义在 `porting/psic/port_log.h` 中，则需将宏 `CFG_HOST_PORT_LOG_FILE` 修改为 `"../psic/port_log.h"`。

表 5-30 host_log_print

接口	说明
函数	<code>int host_log_print(const char *, ...);</code>
功能	打印字符串
参数	<code>const char *</code> 待格式化的字符串 ... 格式化参数
注意事项	无
返回值	不使用返回值

5.1.2.5 COM接口

5.1.2.5.1 移植前注意事项

COM 接口即 Host 与 Device 之间的数据交互接口，如 SPI 1/2/4 线、UART、I2C 等，对于不同的数据上报类型和 LLC 通知类型又有区别，下面对这两种类型进行说明。

数据上报类型分为主动上报、被动上报、主动上报（延时），其中主动上报（延时）暂不支持。
主动上报类型的 Device 会在有数据时直接将数据上报给 Host，由于总线的限制，在上述 3 种总线中只有 UART 支持主动上报，**目前主动上报类型的设备独占一条总线，即总线只能挂载一个设备。**

被动上报类型会在有数据时拉高（高电平有效时）通知 IO，Host 需要在通知 IO 拉高期间发送 POLL 指令给 Device，Device 接收到 POLL 指令后会拉低通知 IO 并准备数据，数据准备完成后重新拉高通知 IO，此时 Host 需要读取数据，数据读取完成后，Device 会将通知 IO 拉低，波形示例见 [3.2.3.1 HIF SPI 数据上报](#)。

LLC 通知类型分为边沿通知（EDGE）、COM 中断通知/COM 有数据通知（COM_ISR）、COM 数据到来通知（COM_IRQ_THREAD）。

下载固件只支持 SPI 1 线、I2C、UART，如果使用 SPI 2/4 线作为 HIF COM，需在初始化接口内根据参数 arg 来初始化成 SPI/DSPI/QSPI，如果 arg 为 LLC_BUS_PARAM_SPI，则初始化为 SPI，如果 arg 为 LLC_BUS_PARAM_DSPI，则初始化为 DSPI，如果 arg 为 LLC_BUS_PARAM_QSPI，则初始化为 QSPI。

5.1.2.5.2 选择 LLC 通知类型

如果是被动上报，直接选择 EDGE 类型。

如果是主动上报，可以选择中断读（COM_ISR）或线程读（COM_IRQ_THREAD）数据。

对于 COM_ISR 和 COM_IRQ_THREAD，COM_ISR 需能够设置中断回调函数，如果不能设置，则只能配置为 COM_IRQ_THREAD 类型。无论是这 2 种中的哪一种，都需要在有数据时调用上层注册的中断回调函数，对于 COM_ISR，可以直接将上层注册的函数注册为中断服务函数，对于 COM_IRQ_THREAD 则是在有数据可以读取时调用上层注册的函数。

COM_ISR 和 COM_IRQ_THREAD 可以在接口 interrupt_ctrl 中配置产生中断的阈值，以得到更完整的 CPU 执行时间。

5.1.2.5.3 EDGE 通知类型接收数据

EDGE 通知类型接收上报数据的示意图如下，图中 TL Layer Thread 表示传输层的数据传输线程，LLC Layer 表示链路层，HAL Layer 表示适配层，图中为表明使用了通信口的哪一个控制接口，在相应步骤的括号内添加了使用的接口名，如 com_ops.write。

EDGE 在接收数据过程中回调用 com_ops.write、com_ops.read 接口。

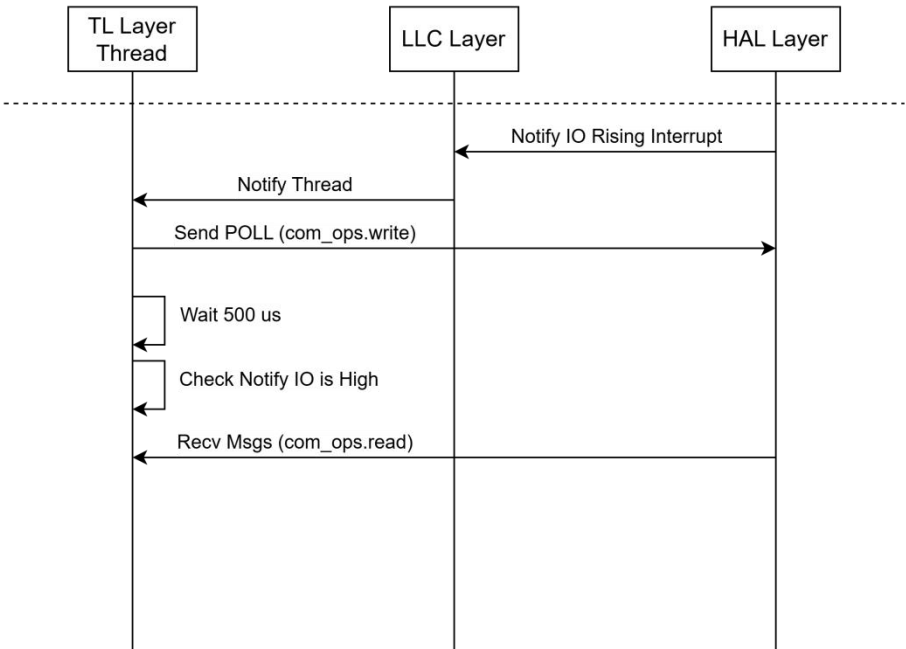


图 5-1 EDGE 通知类型接收上报数据的流程示意图

5.1.2.5.4 COM_ISR 通知类型接收数据

COM_ISR 通知类型是在中断内接收数据，在接收数据的过程中会调用 `com_ops.read`、`com_ops.interrupt` 接口。

COM_ISR 通知类型在接收上报数据时，因线程调用 `LLC_Rcv` 时链路层 Cache 存在已收满和未收满 2 种状态，接收流程会存在些许差异，首先说明线程调用 `LLC_Rcv` 时 Cache 未收满的情况。

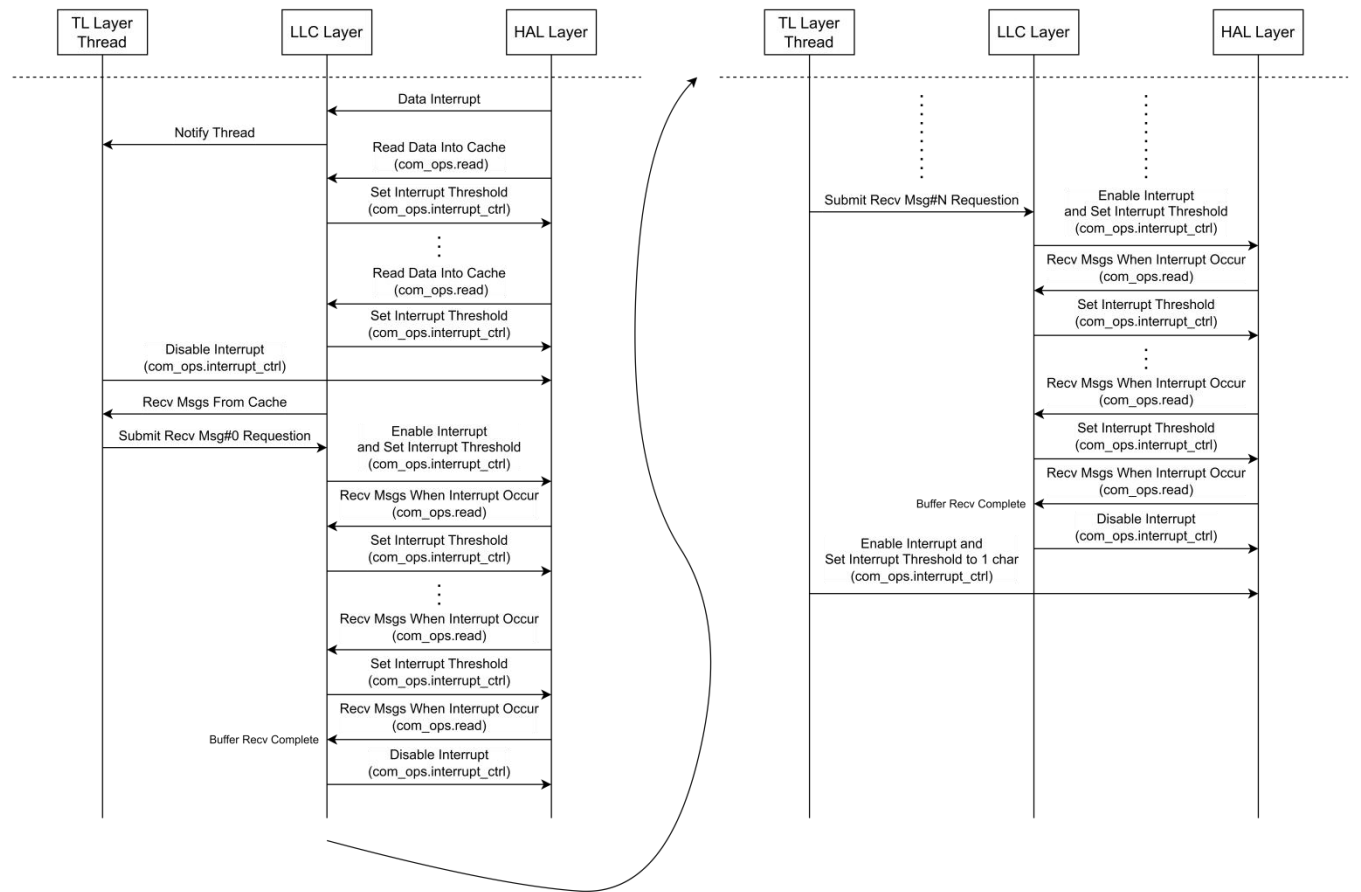


图 5-2 COM_ISR 通知类型接收上报数据的流程示意图（线程在 cache 满之前调用 `LLC_Rcv`）

接下来说明线程调用 `LLC_Rcv` 时 Cache 已收满的情况，在线程调用 `LLC_Rcv` 之前，Cache 已经收满，这时会关闭中断并等待线程调用 `LLC_Rcv` 接收数据。

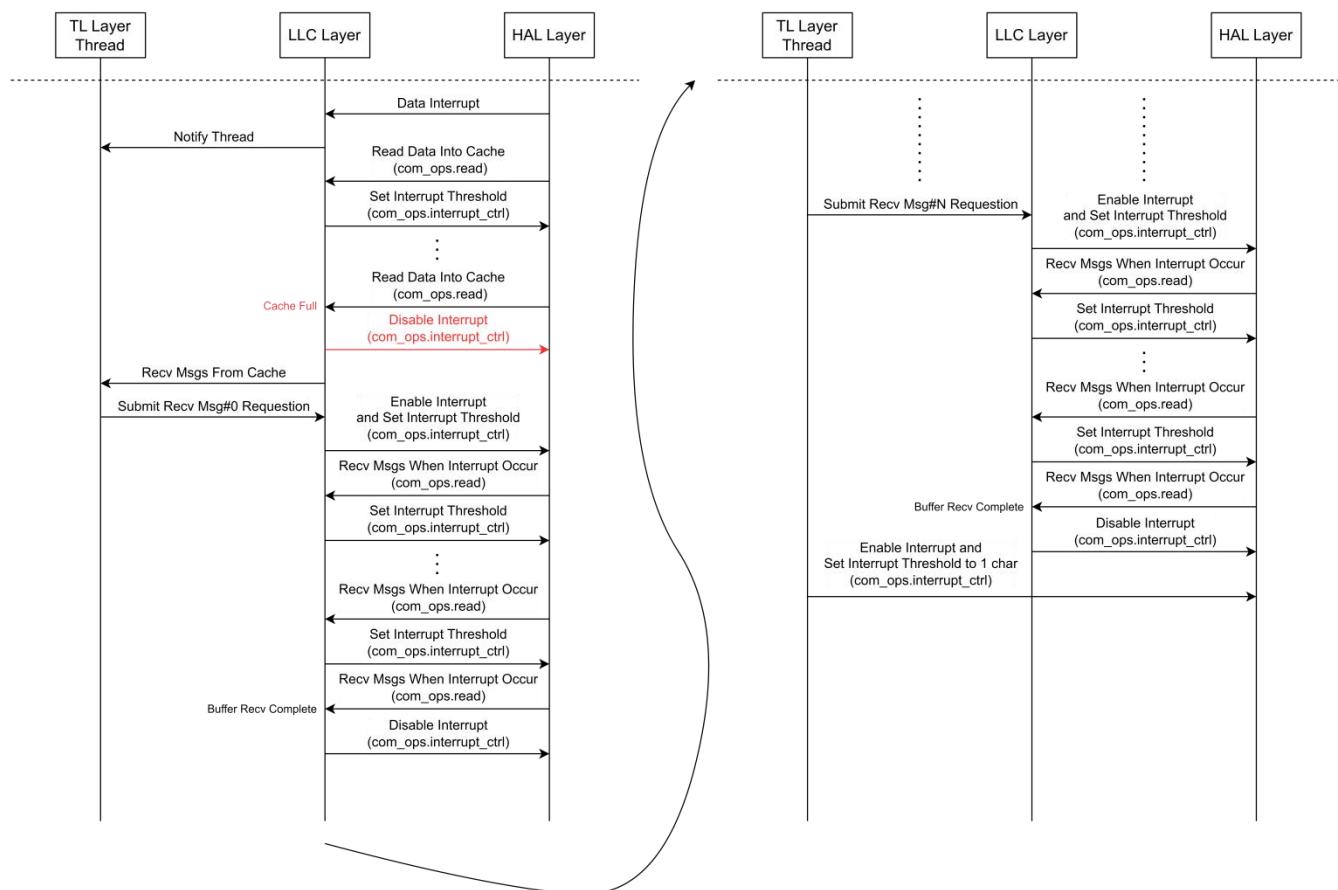


图 5-3 COM_ISR 通知类型接收上报数据的流程图 (线程在 cache 满之后调用 LLC_Recv)

5.1.2.5.5 COM_IRQ_THREAD 通知类型接收数据

这里只介绍 COM_IRQ_THREAD 通知类型不注册 com_ops.interrupt ctrl 接口的情况。

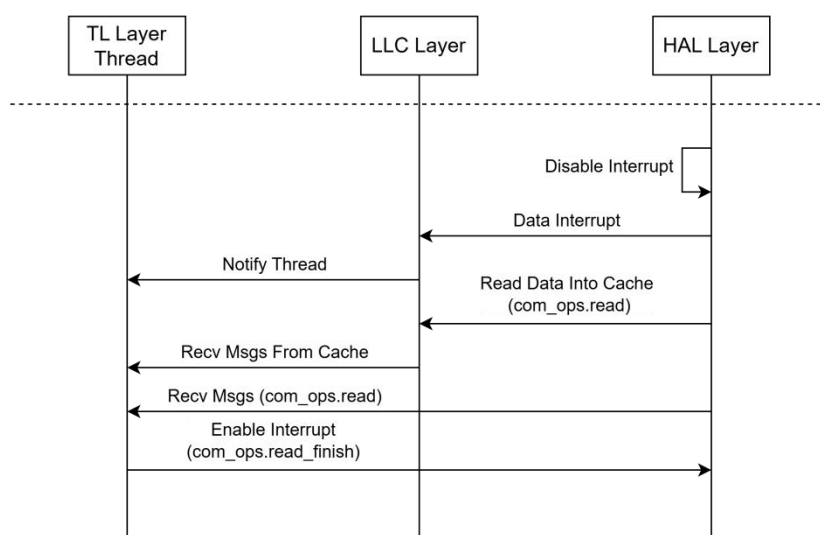


图 5-4 COM_IRQ_THREAD 通知类型接收上报数据的流程示意图

5.1.2.5.6 接口适配需求说明

表 5-31 LLC 不同通知类型的接口需求

	EDGE	COM_ISR	COM_IRQ_THREAD
--	------	---------	----------------

COM 接口 \ LLC 通知类型	通知 IO 边沿通知	HIF COM 数据中断 (中断读)	HIF COM 数据可读 (线程读)
init	✓	✓	✓
deinit	✓	✓	✓
open	✓	✓	✓
close	✓	✓	✓
write	✓	✓	✓
read	✓	✓	✓
read_finish	×	×	✓ 开启中断
interrupt_ctrl	×	✓ 开关 COM 中断, 设置中断阈值	可选, 如果注册该接口, 将会解锁全 cache 缓存数据功能
interrupt_handle_regist	×	✓ 中断触发直接调用 LLC 注册的回调函数	✓ 中断触发先关闭中断再调用 LLC 注册的回调函数
data_callback_regist	×	×	×
特殊要求	无	无	数据包处理用时短

LLC 通知类型为 EDGE 的总线只需提供如下常规接口。

表 5-32 host_com_init_t

接口	说明
函数	typedef void * (*host_com_init_t)(uint8_t id, uint32_t speed, void *arg);
功能	在打开总线时调用，用于初始化与设备无关的内容
参数	uint8_t id 总线 ID uint32_t speed 总线速率，如果是 UART，则表示波特率 void *arg 总线参数（无作用）
注意事项	无
返回值	厂商自定义句柄

表 5-33 host_com_deinit_t

接口	说明
函数	typedef int (*host_com_deinit_t)(void *bus);
功能	在关闭总线时调用，用于反初始化与设备无关的内容
参数	void *bus 厂商自定义句柄
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

表 5-34 host_com_open_t

接口	说明
函数	typedef int (*host_com_open_t)(void *bus, uint32_t param);
功能	在开启设备时调用，用于初始化与设备有关的内容
参数	void *bus 厂商自定义句柄 uint32_t param 设备参数，spi 对应 cs 引脚号、uart 对应挂载的总线 id、i2c 对应设备地址 例如： spi 设备的 cs 引脚号为 0，param 的值为 0 uart 设备挂载的总线 id 为 1，param 的值为 1 i2c 设备地址为 0x3A，param 的值为 0x3A
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

表 5-35 host_com_close_t

接口	说明
函数	typedef int (*host_com_close_t)(void *bus, uint32_t param);
功能	在关闭设备时调用，用于反初始化与设备有关的内容
参数	void *bus 厂商自定义句柄 uint32_t param 设备参数，spi 对应 cs 引脚号、uart 对应挂载的总线 id、i2c 对应设备地址

注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

表 5-36 host_com_write_t

接口	说明
函数	typedef int (*host_com_write_t)(void *bus, uint32_t param, void *pbuff, uint32_t size);
功能	向指定设备写入数据
参数	void *bus 厂商自定义句柄 uint32_t param 设备参数，spi 对应 cs 引脚号、uart 对应挂载的总线 id、i2c 对应设备地址 void *pbuff 待写入 buffer 的地址 uint32_t size buffer 大小
注意事项	无
返回值	>0 写入数据量，其它失败

表 5-37 host_com_read_t

接口	说明
函数	typedef int (*host_com_read_t)(void *bus, uint32_t param, void *pbuff, uint32_t size);
功能	从指定设备读取数据
参数	void *bus 厂商自定义句柄 uint32_t param 设备参数，spi 对应 cs 引脚号、uart 对应挂载的总线 id、i2c 对应设备地址 void *pbuff 接收数据的 buffer 地址 uint32_t size buffer 大小

注意事项	无
返回值	>0 读取数据量，其它失败

如果 LLC 通知类型为 COM_ISR，不仅需要注册上述接口，还需要提供 interrupt_ctrl 和 interrupt_handle_regist 接口。

表 5-38 host_com_interrupt_ctrl_t

接口	说明
函数	typedef int (*host_com_interrupt_ctrl_t)(void *bus, bool en, uint32_t size);
功能	用于控制 COM 中断
参数	void *bus 厂商自定义句柄 bool en true，开启中断 false，关闭中断 uint32_t size 待接收数据量，单位字节，可通过该参数动态设置触发 COM 中断的 FIFO 阈值
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

表 5-39 host_com_interrupt_handle_regist_t

接口	说明
函数	typedef int (*host_com_interrupt_handle_regist_t)(void *bus, host_com_interrupt_handler_t handler, void *arg);
功能	用于通知有数据可接收
参数	void *bus 厂商自定义句柄 host_com_interrupt_handler_t handler 回调函数，类型为 void (*)(void *) void *arg 函数参数
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

如果 LLC 通知类型为 COM_IRQ_THREAD，需要注册常规接口，并额外注册 read_finish 和 interrupt_handle_regist 接口，**触发中断时，应先关闭中断，再回调注册的中断处理函数。**

COM_IRQ_THREAD 通知类型也支持注册 interrupt_ctrl 接口，注册该接口后，会在线程未执行到接收数据的函数前，使用 LLC_HANDLE_S::rx_cache 的全部空间缓存数据。

表 5-40 host_com_read_finish_t

接口	说明
函数	typedef int (*host_com_read_finish_t)(void *bus, uint32_t param);
功能	在完成接收过程时被上层调用，通知 COM 开始等待下一次数据到来
参数	void *bus 厂商自定义句柄 uint32_t param 设备参数，spi 对应 cs 引脚号、uart 对应挂载的总线 id、i2c 对应设备地址
注意事项	开启 COM 中断
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

5.1.2.5.7 总线控制接口集 ops 获取函数定义

厂商移植时需要定义函数 host_com_ops_t *porting_get_com_ops(host_bus_type_t type, uint8_t id)提供给 LLC 层获取总线操作函数接口，其中 type 为总线类型，id 为总线 id，可参考以下方式定义该函数。

```
extern host_com_ops_t *porting_port_spi_get_com_ops(uint8_t id);
extern host_com_ops_t *porting_port_i2c_get_com_ops(uint8_t id);
extern host_com_ops_t *porting_port_uart_get_com_ops(uint8_t id);

/**
 * @brief provided to LLC
 */
host_com_ops_t *porting_get_com_ops(host_bus_type_t type, uint8_t id)
{
    switch (type) {
        case HOST_BUS_TYPE_SPI:
            return porting_port_spi_get_com_ops(id);
        case HOST_BUS_TYPE_I2C:
            return porting_port_i2c_get_com_ops(id);
        case HOST_BUS_TYPE_UART:
            return porting_port_uart_get_com_ops(id);
        default:
            return NULL;
    }
}
```

5.1.2.5.8 添加 SPI 总线示例

SPI 总线支持设置为 EDGE 通知类型，以下为 SPI 总线的驱动伪代码示例。

```
#include "../host_port.h"
#include "../include/hal_com.h"

#include "hal_spi.h"

static void * porting_port_spi_init(uint8_t id, uint32_t speed, void *arg);
static int porting_port_spi_deinit(void *bus);
static int porting_port_spi_open(void *bus, uint32_t cs_pin);
static int porting_port_spi_close(void *bus, uint32_t cs_pin);
static int porting_port_spi_write(void *bus, uint32_t cs_pin, void *pbuff, uint32_t size);
static int porting_port_spi_read(void *bus, uint32_t cs_pin, void *pbuff, uint32_t size);

static const host_com_ops_t spi_ops = {
    .init                = porting_port_spi_init,
    .deinit              = porting_port_spi_deinit,
    .open                = porting_port_spi_open,
    .close               = porting_port_spi_close,
    .write               = porting_port_spi_write,
    .read                = porting_port_spi_read,
    .read_finish         = NULL,
    .interrupt_handle_regist = NULL,
    .data_callback_regist  = NULL,
};

static void * porting_port_spi_init(uint8_t id, uint32_t speed, void *arg)
{}

static int porting_port_spi_deinit(void *bus)
{}

static int porting_port_spi_open(void *bus, uint32_t cs_pin)
{}

static int porting_port_spi_close(void *bus, uint32_t cs_pin)
{}

static int porting_port_spi_write(void *bus, uint32_t cs_pin, void *pbuff, uint32_t size)
{}

static int porting_port_spi_read(void *bus, uint32_t cs_pin, void *pbuff, uint32_t size)
{}

host_com_ops_t *porting_port_spi_get_com_ops(uint8_t id)
```

```
{
    if (id == 0) {
        return (host_com_ops_t *)&spi_ops;
    } else {
        return NULL;
    }
}
```

示例提供了扩展 SPI0 总线的代码，一般来说 id 直接表示总线的 id，即 SPI0 总线的 id 为 0，SPI1 总线的 id 为 1，如果有其它需求，也可以定义其它形式的 id 到 bus 的映射关系。

5.2 Host Driver 使用流程

使用 HostDriver 时，只需要调用 APP 层提供的 API 就能实现数据上报、命令下发、固件下载功能。接下来先介绍总体使用流程，再介绍使用单项功能的流程。

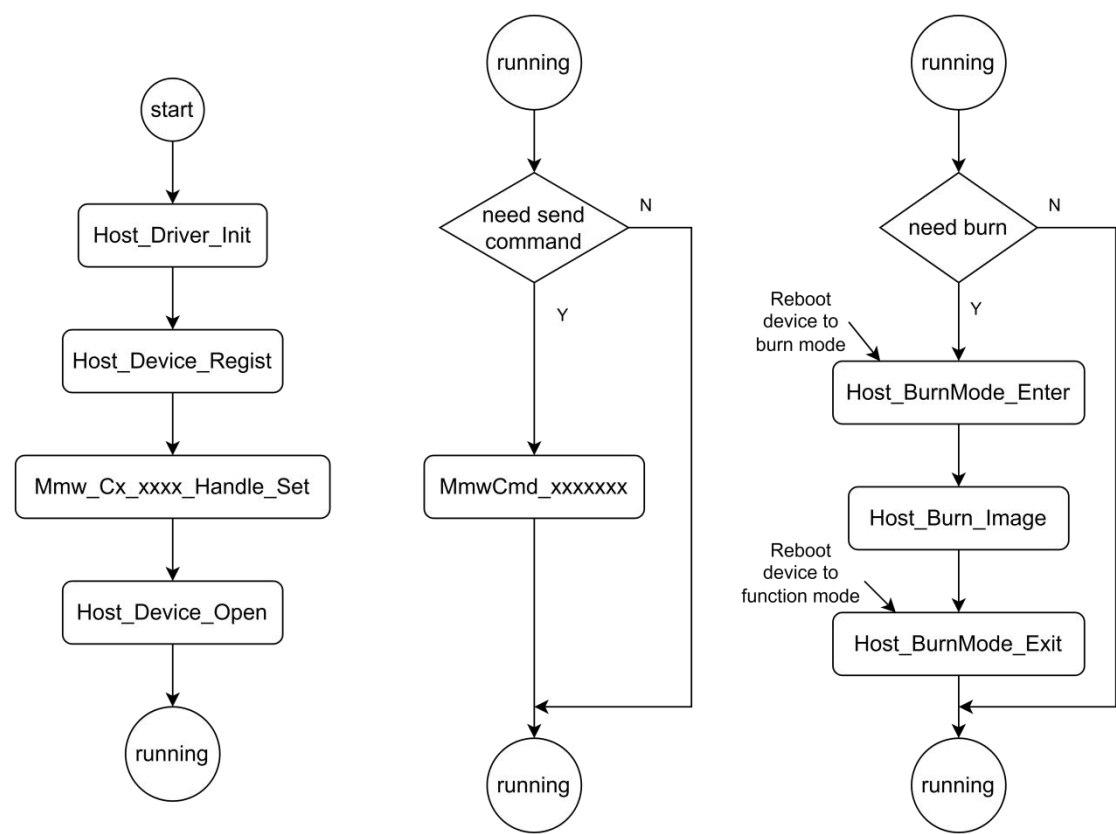


图 5-5 HostDriver 总体使用流程

5.2.1 驱动和设备初始化

5.2.1.1 初始化流程

初始化 HostDriver 和注册一个设备是使用的第一个步骤。

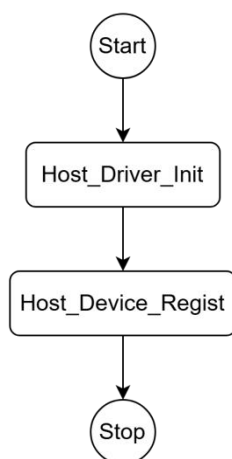


图 5-6 Host Driver 驱动和设备初始化流程

5.2.1.2 设备参数配置

在使用 host driver 之前需要理解数据上报类型和 LLC 通知类型两种概念。

数据上报类型是雷达上报数据时的类型，分为主动上报、被动上报、主动上报（延时）共 3 种，其中主动上报（延时）暂不支持。

如果是 UART 设备，一般是主动上报类型，即雷达有数据时直接将数据发送给主机，UART 也可以是被动上报类型，但是使用不多，这里仅说明 UART 主动上报。

如果是 SPI、I2C，由于其总线特性，Device 只能是被动上报，即 Host 发起数据交互过程。

LLC 通知类型则是 PHY 层（HAL 层）将数据或通知 IO 状态通知给 LLC 层的类型。

对于主动上报，可以选择数据到来通知（COM_IRQ_THREAD）、中断通知/数据可读通知（COM_ISR）。

- COM_IRQ_THREAD: 数据中断到来时，PHY 层关闭中断并回调 host driver 注册的回调函数，host driver 完成数据交互（在 TL 层线程中进行数据交互）后，调用 PHY 层的 read_finish 接口开启中断；
- COM_ISR: 数据中断到来时，PHY 层回调 host driver 注册的回调函数，host driver 在数据交互过程中，调用 PHY 层的 interrupt_ctrl 接口动态调整产生中断的阈值。

对于被动上报，则是将通知 IO 的状态通知给 LLC 的上层，通知类型为边沿通知（EDGE），在这里讨论通知 IO 高电平有效时的状态，EDGE 通知上升沿。

5.2.1.2.1 UART 主动上报 COM_IRQ_THREAD 通知的参数配置

```

DevHw_t devHw = {
    .bus_speed = 921600, // 总线速率
    .bus_type  = LLC_BUS_TYPE_UART, // 总线类型
    .bus_id    = 1, // 总线 ID，当前使用 UART1，因此 ID 为 1
    .upd_io    = 4, // 升级 IO
    .rst_io    = 5, // 复位 IO 或 VCC IO
    .notify_io = 6, // 通知 IO
}
  
```

```
.notify_type = LLC_NOTIFY_TYPE_COM_IRQ_THREAD, // COM_IRQ_THREAD 通知

.upload_type = LLC_UPLOAD_TYPE_ACTIVE, // 主动上报

.param      = 1, // 设备参数，UART 设备参数为挂载的总线 ID
};
```

更推荐的初始化方式：

```
DevHw_t devHw;
DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_UART, 0, 1, 921600, 4, 5, 6, 1);
// 如果是 COM_ISR 通知类型，还需要添加下面这行语句
//devHw.notify_type = LLC_NOTIFY_TYPE_COM_ISR;
```

5.2.1.2.2 SPI 被动上报 EDGE 通知的参数配置

```
DevHw_t devHw = {
    .bus_speed = 8000000, // 总线速率

    .bus_type  = LLC_BUS_TYPE_SPI, // 总线类型

    .bus_id    = 0, // 总线 ID，当前使用 SPI0，因此 ID 为 0

    .bus_param = LLC_BUS_PARAM_SPI, // 1 线 SPI

    .upd_io    = 4, // 升级 IO

    .rst_io    = 5, // 复位 IO 或 VCC IO

    .notify_io = 6, // 通知 IO

    .notify_type = LLC_NOTIFY_TYPE_EDGE, // EDGE 通知

    .upload_type = LLC_UPLOAD_TYPE_PASSIVE, // 被动上报

    .param      = 0, // 设备参数，SPI 设备参数为 CS IO
};
```

更推荐的初始化方式：

```
DevHw_t devHw;
DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);
```

5.2.1.3 API使用说明

在初始化设备之前，需要初始化 host driver，即调用如下 API。

表 5-41 Host_Driver_Init

API	说明
函数	int Host_Driver_Init(void);
功能	初始化 host driver
参数	无

注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

如果需要反初始化 host driver，应在关闭所有设备后调用如下 API。

表 5-42 Host_Driver_Deinit

API	说明
函数	void Host_Driver_Deinit(void);
功能	反初始化 host driver
参数	无
注意事项	无
返回值	无

完成 host driver 初始化之后，可以通过如下 API 注册和注销、打开和关闭设备。

表 5-43 Host_Device_Regist

API	说明
函数	int Host_Device_Regist(DEV_HANDLE *pDevHdl_out, DevHw_t *hw_cfg, HifCfg_t *hif_cfg);
功能	注册一个设备
参数	DEV_HANDLE *pDevHdl_out 设备句柄指针，是输出参数 DevHw_t *hw_cfg 设备的物理配置参数，包含总线信息、总线事件策略、设备信息等内容 HifCfg_t *hif_cfg 设备的 HIF 控制参数
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

DevHw_t 的配置参考上方设备参数配置。

HifCfg_t 可以配置为零，让 host driver 自动将非法配置转换为默认配置，也可以自定义配置，下面对各项配置成员进行说明。

```
typedef struct {
    uint8_t poll_enable; // POLL 命令使能，1 使能，0 失能
    uint8_t app_retry_enable; // 应用层重传使能，1 使能，0 失能
}
```

```
uint8_t  tl_retry_enable; // 传输层重传使能, 1 使能, 0 失能

uint8_t  fragment_enable; // 分片使能, 1 使能, 0 失能

uint8_t  fragment_retry_enable; // 分片重传使能, 1 使能, 0 失能


uint16_t burst_num; // 每次数据接收的 burst 数目

uint16_t cmd_buf_len; // 下发命令使用的 buffer 大小

uint16_t poll_buf_len; // 下发 POLL 使用的 buffer 大小
} HifCfg_t;
```

表 5-44 Host_Device_Unregist

API	说明
函数	void Host_Device_Unregist(DEV_HANDLE DevHdl);
功能	注销一个设备
参数	DEV_HANDLE DevHdl 设备句柄
注意事项	无
返回值	无

表 5-45 Host_Device_Open

API	说明
函数	int Host_Device_Open(DEV_HANDLE DevHdl);
功能	打开一个设备
参数	DEV_HANDLE DevHdl 设备句柄
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功, 其它失败

表 5-46 Host_Device_Close

API	说明
函数	int Host_Device_Close(DEV_HANDLE DevHdl);
功能	关闭一个设备
参数	DEV_HANDLE DevHdl 设备句柄
注意事项	无

返回值	HOST_ERRCODE_SUCCESS 成功，其它失败
-----	------------------------------

5.2.2 数据上报

Host Driver 实现了 2 种方式接收 Device 的上报数据，一种是对数据进行重组后再回调用户处理，如 C1、C2、C3 数据；另一种不做数据处理直接回调用户处理，如 C6、C7、C8、CA、CB。

5.2.2.1 C1/C2/C3数据使用流程

对于 C1/C2/C3 数据（数据定义参考 [3.1.3.2 上报数据结构](#)），Device 上报的数据是分片数据，Host Driver 应用层提供了数据重组功能，即调用 Mmw_C1_CubeData_Handle_Set、Mmw_C2_FFTData_Handle_Set、Mmw_C3_Objects_Handle_Set，注册在这 3 个 API 中的回调函数，会在对应的数据重组完之后被调用。
回调函数设置流程如下：

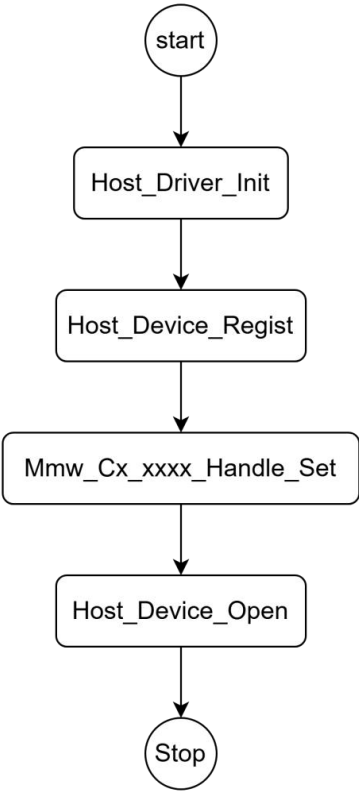


图 5-7 C1/C2/C3 上报数据回调设置流程

如上图所示，执行完该流程后，Host Driver 会在接收到 C1/C2/C3 数据后，通过所设置的回调处理函数把 C1/C2/C3 数据交给用户处理。**需要注意的是，数据存储的 buffer 由 Host Driver 应用层申请，在用户处理函数结束后会被释放，因此，如果在回调结束后，用户仍需要访问这些数据，请拷贝备份数据。**

用户的回调处理函数定义如下：

```
typedef void (*Report_CB)(uint32_t msg_id, uint8_t *data, uint32_t data_len, void *cb_arg);
```

5.2.2.1.1 C1 回调的数据结构

```
typedef struct {
    uint16_t imag;
```

```

uint16_t real;
} Frame_FFT_t;

typedef struct {
    uint32_t frameIdx;
    uint32_t frameLen; /* unit=DWORD */
    uint32_t dataOffset; /* unit=DWORD */
    Frame_FFT_t data[0];
} FrameData_C1_t;
#define HOST_MMW_GET_C1_DATA_LENGTH(data_len) \
    (((data_len) - sizeof(FrameData_C1_t)) / sizeof(Frame_FFT_t))

```

在回调函数中，可参考如下方式对数据进行解析：

```

void C1_Report_CB(uint32_t msg_id, uint8_t *data, uint32_t data_len, void *cb_arg)
{
    FrameData_C1_t *frame = (FrameData_C1_t *)data;
    uint32_t length = HOST_MMW_GET_C1_DATA_LENGTH(data_len);

    for (uint32_t i = 0; i < length; i++) {
        function(&frame->data[i]);
    }
}

```

5.2.2.1.2 C2 回调的数据结构

```

typedef struct {
    uint32_t frameIdx;
    uint32_t frameLen; /* unit=BYTE */
    uint32_t dataOffset; /* unit=BYTE */

    uint32_t type : 8;
    uint32_t total_length : 24;
    uint8_t tx_num;
    uint8_t rx_num;
    uint8_t rev;
    uint8_t rsv;
    uint16_t range_bin_num;
    uint16_t dop_bin_num;

    Frame_FFT_t data[0];
} FrameData_C2_t;
#define HOST_MMW_GET_C2_DATA_LENGTH(data_len) \
    (((data_len) - sizeof(FrameData_C2_t)) / sizeof(Frame_FFT_t))

```

5.2.2.1.3 C3 回调的数据结构

```

typedef struct {
    union {

```

```

        struct {
            uint16_t range;
            uint16_t azimuth;
            uint16_t elevation;
            uint16_t velocity;
            uint16_t snr;
            uint16_t reserved;
        } polar;
        struct {
            uint16_t x;
            uint16_t y;
            uint16_t z;
            uint16_t velocity;
            uint16_t snr;
            uint16_t reserved;
        } coord;
    } type;
} Frame_Point_t;

typedef struct {
    uint32_t frameIdx;
    uint32_t frameLen; /* unit=DWORD */
    uint32_t dataOffset; /* unit=DWORD */

    uint8_t type;
    uint8_t flag;
    uint16_t total_length;

    Frame_Point_t data[0];
} FrameData_C3_t;
#define HOST_MMW_GET_C3_DATA_LENGTH(data_len) \
    (((data_len) - sizeof(FrameData_C3_t)) / sizeof(Frame_Point_t))

```

5.2.2.2 其他数据处理流程

如果需要自定义数据包的重组、解析过程，可以调用 `HIF_Report_Handle_Regist` 设置回调函数，当接收到一包数据或一个完整分片数据后，会回调注册的回调函数。

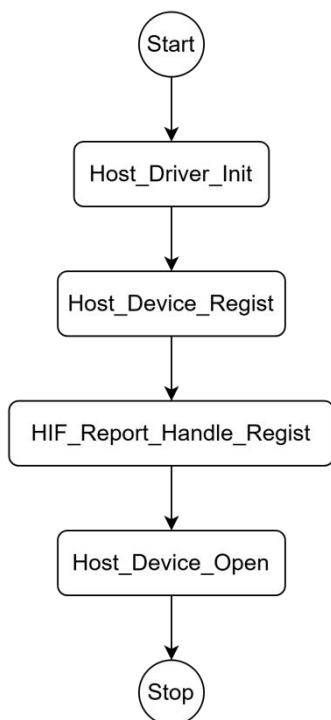


图 5-8 自定义数据处理方案

可以使用 HIF_Report_Handle_Regist 函数注册数据包回调函数，该函数的声明如下：

```
int HIF_Report_Handle_Regist(HIF_HANDLE hdl, uint8_t msg_id, Msg_Report_CB cb, void *arg);
```

回调函数类型如下：

```
typedef void (*Msg_Report_CB)(uint32_t msg_id, uint8_t *payload, uint32_t payload_len, void *arg);
```

其中，msg_id 为注册的消息 ID，payload 为数据包，payload_len 为数据包长度（单位：字节），arg 为注册回调函数时给定的参数 arg。

该函数使用示例如下。

```
static DEV_HANDLE devHandle = NULL;

static void my_report_cb(uint32_t msg_id, uint8_t *data, uint32_t data_len, void *cb_arg)
{
    // 处理和使用数据
}

int main()
{
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);

    Host_Driver_Init();
```

```
Host_Device_Regist(&devHandle, &devHw, &hifCfg);
HIF_Report_Handle_Regist(Host_HIF_Handle_Get(devHandle), 0xE0, my_report_cb, devHandle);
Host_Device_Open(devHandle);

return 0;
}
```

5.2.2.3 API使用说明

如果需要处理上报数据，需要在开启设备之前为设备注册相应消息 ID 的回调函数，回调函数的类型如下。

表 5-47 Report_CB

回调函数	说明
函数	typedef void (*Report_CB)(uint32_t msg_id, uint8_t *data, uint32_t data_len, void *cb_arg);
功能	当收集完一帧/一包数据后，将数据以回调的形式提供给用户
参数	uint32_t msg_id 消息 ID uint8_t *data 数据 uint32_t data_len 数据长度 void *cb_arg 回调参数
注意事项	参数 data 有可能会在回调完成后被释放，对于 C1、C2、C3 是一帧数据回调一次，其它则是一包数据回调一次
返回值	无

host driver 提供了 C1、C2、C3、C6、C7、C8、CA、CB 的数据处理 API，对于这些 Cx 消息处理的 API 定义如下。

表 5-48 Mmw_Cx_FUNCTION_Handle_Set

回调函数	说明
函数	int Mmw_Cx_FUNCTION_Handle_Set(DEV_HANDLE DevHdl, Report_CB cb, void *cb_arg);
功能	设置 Cx 数据回调，其中 C1、C2、C3 是数据帧回调，其它是数据包回调
参数	DEV_HANDLE DevHdl 设备句柄

	Report_CB cb 回调函数 void *cb_arg 回调参数
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

其中，Cx 与 FUNCTION 的对应关系如下。

表 5-49 Cx 与 FUNCTION 的对应关系

消息 ID	FUNCTION
C1	CubeData
C2	FFTData
C3	Objects
C6	GeneralData
C7	PlotData
C8	MotionSensor
CA	MotionSensor
CB	MotionSensor

如果需要设置其它消息 ID 的处理回调，可以使用如下通用消息 ID 回调设置的 API。

表 5-50 Mmw_Report_Handle_Set_General

回调函数	说明
函数	int Mmw_Report_Handle_Set_General(DEV_HANDLE DevHdl, Report_CB cb, void *cb_arg, void *buffer, uint32_t buffer_len, uint8_t msg_id, void *proc_handle);
功能	设置通用消息 ID 的回调，是数据包回调
参数	DEV_HANDLE DevHdl 设备句柄 Report_CB cb 回调函数，记录于消息 ID 控制对象中 void *cb_arg 回调参数，记录于消息 ID 控制对象中 void *buffer 接收数据的 buffer

	uint32_t buffer_len buffer 大小 uint8_t msg_id 消息 ID void *proc_handle 数据包回调函数，类型是 void (*)(uint32_t msg_id, uint8_t *payload, uint32_t payload_len, void *arg)，其中 msg_id 为消息 ID、payload 为数据包的 payload、payload_len 为 payload 的长度、arg 为消息 ID 控制对象
注意事项	每个数据包回调的是 proc_handle，可以通过消息 ID 控制对象获取注册进来的用户回调 cb 和用户回调 cb_arg
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

消息 ID 控制对象结构如下。

```
#define REPORT_FLAG_BUFFER_USER          BIT(0)
#define REPORT_FLAG_BUFFER_HOLD          BIT(1)
typedef struct {
    DEV_HANDLE DevHdl;
    Report_CB  report_cb;
    void      *cb_arg;
    uint8_t    *buffer;
    uint32_t    bufferLen;

    uint8_t     msgId;
    uint8_t     flag;
    uint8_t     drop;
    #if (CFG_APP_RETRY_EN)
        uint8_t     retry_max;
        uint8_t     retry_start;
        uint8_t     retry_end;
        void      *retry_node;
    #else
        uint8_t     reserved;
    #endif
    uint32_t     frameIdx;
    uint32_t     frameLen;    /* unit=byte */
    uint32_t     frameOffset; /* unit=byte */
} Report_Local_t;
```

如果需要释放所有上报数据处理函数及占用资源，可以调用如下 API。

表 5-51 Mmw_Report_Handle_FreeAll

回调函数	说明
函数	void Mmw_Report_Handle_FreeAll(DEV_HANDLE DevHdl);
功能	释放所有上报数据处理函数及占用资源， 只有通过 Mmw_Cx_FUNCTION_Handle_Set 注册后申请的资源会被释放
参数	DEV_HANDLE DevHdl 设备句柄
注意事项	只能释放提供了专用消息 ID 处理函数注册接口的处理函数和占用的资源
返回值	无

如果需要释放自定义的上报数据处理函数及占用资源，可以使用如下代码。

```
Mmw_Report_Handle_Set_General(DevHdl, NULL, NULL, NULL, 0, MSG_ID, NULL);
```

5.2.3 命令下发

5.2.3.1 使用流程

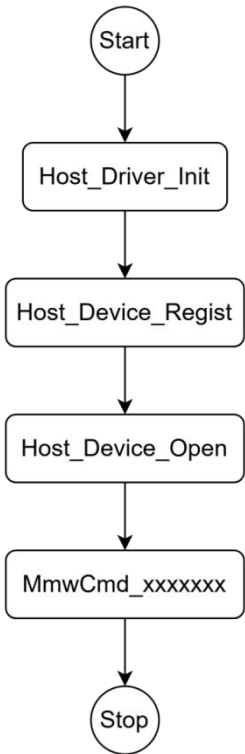


图 5-9 命令下发流程

命令下发需要在开启设备之后进行，接下来以获取 SDK 版本号为例对命令下发流程进行说明。

```
static DEV_HANDLE devHandle = NULL;

int main()
{
```

```
DevHw_t devHw;
HifCfg_t hifCfg = {
    .cmd_buf_len = 1024,
};
DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);

Host_Driver_Init();
Host_Device_Regist(&devHandle, &devHw, &hifCfg);
Host_Device_Open(devHandle);
HostCmd_Version_Get(devHandle, 0);

return 0;
}
```

5.2.3.2 API使用说明

命令控制分为系统命令配置和雷达命令配置，系统命令配置属于通用命令配置，与具体的雷达型号和固件类型无关，而雷达命令配置则需要根据雷达型号和固件类型进行选择。

5.2.3.2.1 系统命令配置

host driver 在 cmd_api/host_driver 提供了获取版本号命令、重启 device 命令和通用命令的 API。

表 5-52 HostCmd_Version_Get

API	说明
函数	int HostCmd_Version_Get(DEV_HANDLE DevHdl, uint32_t user_version);
功能	获取版本号
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t user_version 用户版本号, 用户版本号为 1 时获取的是 custom version, 为 0 时获取的是 code version
注意事项	无
返回值	0 成功, 其它失败

表 5-53 HostCmd_Device_Reboot

API	说明
函数	int HostCmd_Device_Reboot(DEV_HANDLE DevHdl, uint32_t reboot);
功能	重启 device
参数	DEV_HANDLE DevHdl 设备句柄

	uint32_t reboot 重启模式，重启模式为 1 时是软重启，其它不支持
注意事项	无
返回值	0 成功，其它失败

表 5-54 Host_General_Command

API	说明
函数	int Host_General_Command(DEV_HANDLE DevHdl, uint32_t msg_id, void *param, uint32_t param_len, void *resp_buff, uint32_t buffer_len, uint32_t *data_len);
功能	通用命令下发
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t msg_id 消息 ID void *param 命令的 payload uint32_t param_len 命令的 payload 长度 void *resp_buff 接收 resp payload 的 buffer uint32_t buffer_len 接收 resp payload 的 buffer 长度 uint32_t *data_len 接收的 resp payload 的长度，可以传递 NULL 参数，表示不需要知道这个长度信息
注意事项	无
返回值	0 成功，其它失败

在使用通用命令下发时，可以参考如下形式，其中 MSG_ID 为这条命令的消息 ID，X 和 Y 分别为命令的 payload 长度和命令 resp 的 payload 长度，可以通过这种形式实现雷达命令配置。

```
int HostCmd_xxxxx(DEV_HANDLE DevHdl)
{
    int ret = 0;
    uint8_t cmd_payload[X];
```

```
uint8_t resp_payload[Y];
uint32_t resp_data_len;

// set cmd_payload
.....

ret = Host_General_Command(DevHdl, MSG_ID, cmd_payload, X, resp_payload, Y, &resp_data_len);
if (ret != HOST_ERRCODE_SUCCESS) {
    return ret;
} else {
    return resp_payload[0];
}
}
```

5.2.3.2.2 雷达命令配置

host driver 在 cmd_api/host_driver/rs6x_7x 提供了部分雷达命令配置，雷达命令配置基于通用命令下发 API 实现。

如果需要使用雷达分析工具固件（radar_analysis）的命令配置，需要先给雷达分析工具固件下发如下指令开启固件的 HIF 配置和获取雷达参数功能。

```
mmwc hif_regist
```

下面对通用（除了 radar analysis）的雷达命令配置 API 进行说明。

表 5-55 MmwCmd_General_StartCtrl_Cfg

API	说明
函数	int MmwCmd_General_StartCtrl_Cfg(DEV_HANDLE DevHdl, uint32_t enable);
功能	开启或停止雷达
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t enable 开启或停止的控制字，控制字为 1 表示开启、为 0 表示停止
注意事项	无
返回值	0 成功，其它失败

表 5-56 MmwCmd_General_MimoMode_Cfg

API	说明
函数	int MmwCmd_General_MimoMode_Cfg(DEV_HANDLE DevHdl, uint32_t mimo_mode);
功能	配置雷达 MIMO 模式

参数	DEV_HANDLE DevHdl 设备句柄 uint32_t mimo_mode MIMO 模式 <pre> #define HOST_HIF_R3DATABOX_TLV_MIMO_1T1R 0x00 #define HOST_HIF_R3DATABOX_TLV_MIMO_1T3R 0x01 #define HOST_HIF_R3DATABOX_TLV_MIMO_2T3R 0x02 #define HOST_HIF_R3DATABOX_TLV_MIMO_1T4R 0x03 #define HOST_HIF_R3DATABOX_TLV_MIMO_2T4R 0x04 #define HOST_HIF_R3DATABOX_TLV_MIMO_1T2R 0x05 </pre>
注意事项	无
返回值	0 成功, 其它失败

表 5-57 MmwCmd_General_FrameType_Cfg

API	说明
函数	int MmwCmd_General_FrameType_Cfg(DEV_HANDLE DevHdl, uint32_t frame_type);
功能	配置雷达帧类型
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t frame_type 雷达帧类型 <pre> #define HOST_HIF_R3DATABOX_TLV_FRAME_TYPE_1D 0x00 #define HOST_HIF_R3DATABOX_TLV_FRAME_TYPE_2D 0x01 </pre>
注意事项	无
返回值	0 成功, 其它失败

表 5-58 MmwCmd_General_StartFreq_Cfg

API	说明
函数	int MmwCmd_General_StartFreq_Cfg(DEV_HANDLE DevHdl, uint32_t start_freq);
功能	配置雷达起始频率
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t start_freq 起始频率

注意事项	无
返回值	0 成功，其它失败

表 5-59 MmwCmd_General_TriggerRange_Cfg

API	说明
函数	int MmwCmd_General_TriggerRange_Cfg(DEV_HANDLE DevHdl, uint32_t trigger_range);
功能	配置雷达触发距离
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t trigger_range 触发距离
注意事项	无
返回值	0 成功，其它失败

表 5-60 MmwCmd_General_RangeResolution_Cfg

API	说明
函数	int MmwCmd_General_RangeResolution_Cfg(DEV_HANDLE DevHdl, uint32_t range_resolution);
功能	配置雷达距离分辨率
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t range_resolution 距离分辨率
注意事项	无
返回值	0 成功，其它失败

表 5-61 MmwCmd_General_MaxVelocity_Cfg

API	说明
函数	int MmwCmd_General_MaxVelocity_Cfg(DEV_HANDLE DevHdl, uint32_t max_velocity);
功能	配置雷达最大检测速度
参数	DEV_HANDLE DevHdl 设备句柄

	uint32_t max_velocity 最大检测速度
注意事项	无
返回值	0 成功，其它失败

表 5-62 MmwCmd_General_VelResolution_Cfg

API	说明
函数	int MmwCmd_General_VelResolution_Cfg(DEV_HANDLE DevHdl, uint32_t vel_resolution);
功能	配置雷达速度分辨率
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t vel_resolution 速度分辨率
注意事项	无
返回值	0 成功，其它失败

表 5-63 MmwCmd_General_FramePeriod_Cfg

API	说明
函数	int MmwCmd_General_FramePeriod_Cfg(DEV_HANDLE DevHdl, uint32_t period_ms);
功能	配置雷达帧周期
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t vel_resolution 帧周期
注意事项	无
返回值	0 成功，其它失败

雷达分析工具命令 API 如下。

表 5-64 MmwCmd_RadarAnalysis_Start

API	说明
函数	int MmwCmd_RadarAnalysis_Start(DEV_HANDLE DevHdl);
功能	开启雷达

参数	DEV_HANDLE DevHdl 设备句柄
注意事项	无
返回值	0 成功，其它失败

表 5-65 MmwCmd_RadarAnalysis_Stop

API	说明
函数	int MmwCmd_RadarAnalysis_Stop(DEV_HANDLE DevHdl);
功能	停止雷达
参数	DEV_HANDLE DevHdl 设备句柄
注意事项	无
返回值	0 成功，其它失败

表 5-66 MmwCmd_RadarAnalysis_Mode_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_Mode_Cfg(DEV_HANDLE DevHdl, uint32_t txrx, uint32_t mode);
功能	配置雷达 MIMO 模式和帧类型
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t txrx MIMO 模式 uint32_t mode 帧类型
注意事项	无
返回值	0 成功，其它失败

表 5-67 MmwCmd_RadarAnalysis_Freq_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_Freq_Cfg(DEV_HANDLE DevHdl, uint32_t start_freq, uint32_t max_freq);
功能	配置雷达起始频率和最大频率

参数	DEV_HANDLE DevHdl 设备句柄 uint32_t start_freq 起始频率 uint32_t max_freq 最大频率
注意事项	无
返回值	0 成功，其它失败

表 5-68 MmwCmd_RadarAnalysis_Range_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_Range_Cfg(DEV_HANDLE DevHdl, uint32_t range, uint32_t resol);
功能	配置雷达探测距离和距离分辨率
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t range 探测距离 uint32_t resol 距离分辨率
注意事项	无
返回值	0 成功，其它失败

表 5-69 MmwCmd_RadarAnalysis_Veloc_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_Veloc_Cfg(DEV_HANDLE DevHdl, uint32_t veloc, uint32_t resol);
功能	配置雷达检测速度和速度分辨率
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t veloc 检测速度

	uint32_t resol 速度分辨率
注意事项	无
返回值	0 成功，其它失败

表 5-70 MmwCmd_RadarAnalysis_Frame_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_Frame_Cfg(DEV_HANDLE DevHdl, uint32_t period, uint32_t num);
功能	配置雷达帧周期和发帧数目
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t period 帧周期 uint32_t num 发帧数目，发帧数目为 0 表示持续发帧
注意事项	无
返回值	0 成功，其它失败

表 5-71 MmwCmd_RadarAnalysis_Intv_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_Intv_Cfg(DEV_HANDLE DevHdl, uint32_t period, uint32_t intv_num);
功能	配置雷达 interval 周期和数目
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t period interval 周期 uint32_t intv_num interval 数目
注意事项	仅在 1DFFT 模式时使用
返回值	0 成功，其它失败

表 5-72 MmwCmd_RadarAnalysis_ChirpNum_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_ChirpNum_Cfg(DEV_HANDLE DevHdl, uint32_t num);
功能	配置雷达每个 interval 的 chirp 数目
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t num 每个 interval 的 chirp 数目
注意事项	无
返回值	0 成功，其它失败

表 5-73 MmwCmd_RadarAnalysis_Report_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_Report_Cfg(DEV_HANDLE DevHdl, uint32_t report_type, uint32_t report_num);
功能	配置上报数据类型和数目
参数	DEV_HANDLE DevHdl 设备句柄 uint32_t report_type 数据类型，为 0 表示 CUBE 数据、为 4 表示点云数据、为 255 表示关闭上报 uint32_t report_num 数据数目，表示上报的帧数
注意事项	无
返回值	0 成功，其它失败

表 5-74 MmwCmd_RadarAnalysis_DopFft_Cfg

API	说明
函数	int MmwCmd_RadarAnalysis_DopFft_Cfg(DEV_HANDLE DevHdl, uint32_t flag);
功能	在 2DFFT 模式时开启或关闭 DOP FFT
参数	DEV_HANDLE DevHdl 设备句柄

	uint32_t report_type 开关标志，开关标志为 1 时开启、为 0 时关闭
注意事项	无
返回值	0 成功，其它失败

5.2.4 固件下载

5.2.4.1 使用流程

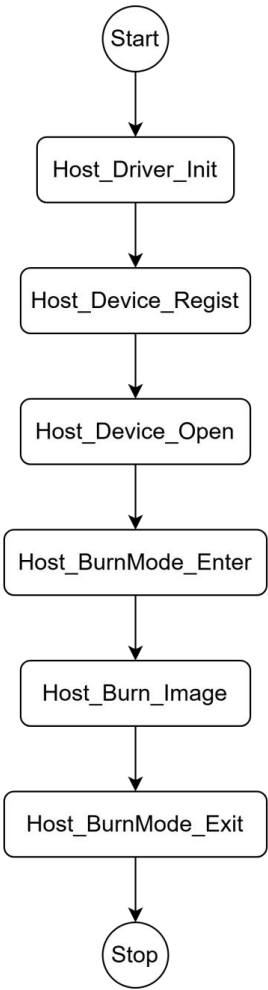


图 5-10 下载固件流程

下载固件需要在开启设备之后进行，下载固件示例如下：

```
static DEV_HANDLE devHandle = NULL;

int main()
{
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
}
```

```
DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);

Host_Driver_Init();
Host_Device_Regist(&devHandle, &devHw, &hifCfg);
Host_Device_Open(devHandle);
Host_BurnMode_Enter(devHandle);
Host_Burn_Image(devHandle, "image1");
Host_BurnMode_Exit(devHandle);

return 0;
}
```

其中字符串"image1"会传递给存储介质接口 int host_store_open(void **hdl, void *path, uint32_t *len)的 path 参数，该接口会返回固件句柄和固件长度。

5.2.4.2 API使用说明

如果需要为雷达下载固件，需要先使用 host driver 打开一个设备对象，然后调用如下 API 将雷达进入下载模式。

表 5-75 Host_BurnMode_Enter

API	说明
函数	int Host_BurnMode_Enter(DEV_HANDLE DevHdl);
功能	将一个设备进入下载模式
参数	DEV_HANDLE DevHdl 设备句柄
注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

进入下载模式后，调用如下 API 进行固件下载，其中参数 path 与移植的 hal_store 中的接口有关，属于厂商自定义参数，可以根据该参数选择不同的固件。

表 5-76 Host_Burn_Image

API	说明
函数	int Host_Burn_Image(DEV_HANDLE DevHdl, void *path);
功能	给从机下载固件
参数	DEV_HANDLE DevHdl 设备句柄 void *path 固件

注意事项	无
返回值	HOST_ERRCODE_SUCCESS 成功，其它失败

固件下载完成后，通过如下 API 将雷达退出下载模式并进入功能模式。

表 5-77 Host_BurnMode_Exit

API	说明
函数	void Host_BurnMode_Exit(DEV_HANDLE DevHdl);
功能	给一个从机退出下载模式
参数	DEV_HANDLE DevHdl 设备句柄
注意事项	无
返回值	无

需要注意的是，**host driver** 规定只能有一个雷达处于下载模式。

5.3 配置和编译

代码包根目录下的文件 host_config.h 管理 host driver 全局配置，user_config.h 覆盖 host_config.h 的配置宏，可修改这些配置达到定制化目的。

5.3.1 功能裁剪

全局宏支持裁剪代码/功能，可裁剪传输层重传、消息层分片、消息层重传、应用层重传 4 项功能，如果裁剪消息层分片功能，则必须裁剪消息层重传功能。

表 5-78 用于裁剪功能的全局宏

宏	说明
CFG_TL_RETRY_EN	传输层重传使能， 可用于 CodeSize 裁剪
CFG_MSG_FRAGMENT_EN	消息层分片使能， 可用于 CodeSize 裁剪
CFG_MSG_FRAGMENT_RETRY_EN	消息层分片重传使能， 可用于 CodeSize 裁剪，如果开启此功能，必须开启消息层分片功能
CFG_APP_RETRY_EN	应用层重传使能， 可用于 CodeSize 裁剪

5.3.2 适配层功能开关

适配层功能开关宏可以配置日志、存储介质、通信接口、OS、休眠、IO 功能的开启与关闭，日志功能还配置有日志等级和打印日志的接口原型/声明路径。

表 5-79 用于开关适配层功能的全局宏

宏	说明
CFG_HOST_PORT_LOG_EN	日志使能
CFG_HOST_PORT_LOG_FILE	host_log_print 函数定义/声明与文件 porting/include/hal_log.h 的相对路径
CFG_HOST_LOG_LEVEL	日志等级, HOST_LOG_LEVEL_xxx
CFG_HOST_PORT_STORE_EN	存储介质使能
CFG_HOST_PORT_COM_EN	COM 使能, 必须使能
CFG_HOST_PORT_COM_SPI_EN	SPI COM 使能
CFG_HOST_PORT_COM_I2C_EN	I2C COM 使能
CFG_HOST_PORT_COM_UART_EN	UART COM 使能
CFG_HOST_PORT_OS_EN	OS 使能, 必须使能
CFG_HOST_PORT_PM_EN	休眠使能
CFG_HOST_PORT_IO_EN	IO 使能, 必须使能

5.3.3 固件下载功能

下载功能的详细配置宏, 可配置读、写 Flash, 读、写 SRAM, 设置 Device PC 指针功能, 可开启回读校验功能 (只支持 UART 下载时开启回读校验), 可以配置固件下载相关指令的重试次数和读回复包的重试次数, 能够配置是否校验 checksum。

表 5-80 固件下载功能的详细配置

宏	说明
CFG_HOST_BURN_DOWNLOAD_READ_CHECK_EN	回读校验使能, 开启后, UART 下载固件时, 会回读校验
CFG_HOST_BURN_FLASH_EN	下载固件到 flash 使能
CFG_HOST_BURN_FLASH_READ_EN	读 Device flash 使能
CFG_HOST_BURN_SRAM_EN	下载固件到 sram 使能, SDK2.1.0 暂不支持
CFG_HOST_BURN_RUN_EN	下载模式时设置 Device PC 指针使能, SDK2.1.0 暂不支持
CFG_HOST_BURN_RETRY_CNT	下载模式时的指令重传次数
CFG_HOST_BURN_RSP_RETRY_CNT	下载模式时的读回复包的重试次数
CFG_HOST_BURN_CHECKSUM_EN	下载模式时校验 checksum 使能

CFG_HOST_DRIVER_SPI_SPEED_IN_DOWNLOAD	SPI 下载固件时使用的速率，需低于 8MHz
---------------------------------------	-------------------------

5.3.4 消息层功能

消息层的详细功能配置宏，能够配置消息层分片的异常检测、保留的最大 buffer 大小等内容。

表 5-81 用于配置消息层的全局宏

宏	说明
SUPPORT_HOST_HIF_FRAGMENT_PENDING_INFO_ABNORMAL_DETECT	消息层分片的异常检测功能使能，用于清除无效分片
CFG_HOST_HIF_FRAGMENT_PENDING_INFO_ABNORMAL_BURST_THR	消息层分片的异常检测阈值，如果一个分片流未接收完成，并且后续连续接收到该宏指定个数的非该分片流的 burst 后，将对这个分片流执行清除操作
SUPPORT_HOST_HIF_FRAGMENT_PENDING_INFO_ABNORMAL_DETECT_TIMELY	及时清除异常分片使能，如果使能，清除操作会在接收数据过程中进行，如果不使能，清除操作会在数据处理过程中进行，为保证数据接收的效率，默认不使能
ABNORMAL_FRAGMENT_INFO_LOG	清除操作执行前的回调宏，类似于析构函数
CFG_HOST_HIF_FRAGMENT_FREE_INFO_NUM_MAX	每个设备的每个消息 ID 最多保留的空闲分片流控制体数目，能够改善内存碎片现象，提高分片数据流的接收效率 -1 表示不限制最大数目，0 表示不保留，>0 表示保留的数目
CFG_HOST_HIF_FRAGMENT_HOLD_BUFFER_SIZE_MAX	每个设备的每个消息 ID 能够保留的分片流使用过的 buffer 的最大大小，分片流使用完成后，会比较分片流的 buffer 和保留的 buffer 的大小，如果分片流的 buffer 更大，并且保留的 buffer 此时未被使用，将会释放原来保留的 buffer，并保留分片流的 buffer，该宏会限制这个过程中替换 buffer 的动作，如果分片流的 buffer 满足替换条件，但是超过了该宏限制的大小，将不进行替换操作 0 表示不限制大小，>0 表示最大保留大小

5.3.5 其它

表 5-82 其它配置宏

宏	说明
_ALIGNAS_VARIABLE	变量内存对齐，如果对变量的内存对齐有要求，可以修改该宏
_WEAK_FUNCTION	虚函数定义
HOST_OS_TIMEOUT_FOREVER	OS 永久等待

HOST_OS_TIMEOUT_NO_WAIT	OS 不等待
HOST_OS_THREAD_PRIO_NORMAL	线程普通优先级
HOST_OS_THREAD_PRIO_LOW	线程低优先级
HOST_OS_THREAD_PRIO_HIGH	线程高优先级
HOST_HIF_POLL_ACK_TLV_IS_VALID	POLL-ACK 的 TLV 有效判断宏，不要修改该宏
CFG_LLC_DEVICE_RX_CACHE_SIZE	cache 大小，单位字节，当通知方式为 COM_ISR 或 COM_IRQ_THREAD 时，会创建 cache 用于缓存数据，COM_ISR 通知类型会使用全部的 cache 缓存数据，COM_IRQ_THREAD 只有注册 interrupt_ctrl 接口时才会使用全部的 cache 缓存数据
HOST_DEVICE_STATE_INFO	获取设备状态及信息使能
HOST_HIF_CASE_GENERAL_EN	通用雷达指令使能
HOST_HIF_CASE_RADAR_ANALYSIS_EN	radar analysis 指令使能

5.4 简单的移植测试代码

移植完成 HAL 之后，需要编写简单的测试代码来验证移植是否成功，下面以业务代码的形式说明如何使用 UART 设备和 SPI 设备。

5.4.1 数据上报

5.4.1.1 UART

首先是 UART 设备，COM_IRQ_THREAD 通知类型。

```
#include "host_driver.h"

static DEV_HANDLE devHandle = NULL;

static void Msg_C6_Report_CB(uint32_t msg_id, uint8_t *payload, uint32_t payload_len, void *arg)
{
    static uint64_t counter = 0;
    HOST_LOG_INF("[%llu] device(%p) recv 0x%02X msg\n", counter++, arg, msg_id);
    HOST_LOG_INF("payload len = %u\n", payload_len);
    HOST_LOG_INF_HEX(payload, ((payload_len > 16) ? 16 : payload_len), "PAYLOAD");
}

int main()
{
```

```

int status = HOST_ERRCODE_SUCCESS;
DevHw_t devHw;
HifCfg_t hifCfg = {
    .cmd_buf_len = 1024,
};
DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_UART, 0, 1, 921600, 4, 5, 6, 1);

HOST_LOG_PRINT("\n\n\n");
HOST_LOG_INF("start to test application\n\n\n");

status = Host_Driver_Init();
if (status != HOST_ERRCODE_SUCCESS) {
    HOST_LOG_ERR("init host driver fail(%d)\n", status);
    goto out;
}
HOST_LOG_INF("init host driver succ\n");
status = Host_Device_Regist(&devHandle, &devHw, &hifCfg);
if (status != HOST_ERRCODE_SUCCESS) {
    HOST_LOG_ERR("regist device(%p) fail(%d)\n", devHandle, status);
    goto out;
}
HOST_LOG_INF("regist device(%p) succ\n", devHandle);
status = HIF_Report_Handle_Regist(Host_HIF_Handle_Get(devHandle), 0xC6, Msg_C6_Report_CB,
devHandle); // 修改第二个参数 0xC6 为 0xC1 就可以接收 0xC1 数据
if (status != HOST_ERRCODE_SUCCESS) {
    HOST_LOG_ERR("regist 0xC6 report handle fail(%d)\n", status);
    goto out;
}
HOST_LOG_INF("regist 0xC6 report handle succ\n");
status = Host_Device_Open(devHandle);
if (status != HOST_ERRCODE_SUCCESS) {
    HOST_LOG_ERR("open device(%p) fail(%d)\n", devHandle, status);
    goto out;
}
HOST_LOG_INF("open device(%p) succ\n", devHandle);

out:
HOST_LOG_PRINT("\n\n\n");
HOST_LOG_INF("test application done\n\n\n");
}

```

5.4.1.2 SPI

1 线 SPI 设备，以 EDGE 通知类型为例。

```

#include "host_driver.h"

static DEV_HANDLE devHandle = NULL;

```

```
static void Msg_C6_Report_CB(uint32_t msg_id, uint8_t *payload, uint32_t payload_len, void *arg)
{
    static uint64_t counter = 0;
    HOST_LOG_INF("[%llu] device(%p) recv 0x%02X msg\n", counter++, arg, msg_id);
    HOST_LOG_INF("payload len = %u\n", payload_len);
    HOST_LOG_INF_HEX(payload, ((payload_len > 16) ? 16 : payload_len), "PAYLOAD");
}

int main()
{
    int status = HOST_ERRCODE_SUCCESS;
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);

    HOST_LOG_PRINT("\n\n");
    HOST_LOG_INF("start to test application\n\n");

    status = Host_Driver_Init();
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("init host driver fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("init host driver succ\n");
    status = Host_Device_Regist(&devHandle, &devHw, &hifCfg);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("regist device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("regist device(%p) succ\n", devHandle);
    status = HIF_Report_Handle_Regist(Host_HIF_Handle_Get(devHandle), 0xC6, Msg_C6_Report_CB,
    devHandle); // 修改第二个参数 0xC6 为 0xC1 就可以接收 0xC1 数据
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("regist 0xC6 report handle fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("regist 0xC6 report handle succ\n");
    status = Host_Device_Open(devHandle);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("open device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("open device(%p) succ\n", devHandle);
}
```

```
out:
    HOST_LOG_PRINT("\n\n");
    HOST_LOG_INF("test application done\n\n");
}
```

正常运行的 LOG 示例如下。

```
[I][0] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[I][1] device(10505708) recv 0xC6 msg
[I]payload len = 120
[I]PAYLOAD (16)
03 03 01 08 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E

[I][2] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[I][3] device(10505708) recv 0xC6 msg
[I]payload len = 72
[I]PAYLOAD (16)
03 03 01 04 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E

[I][4] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[I][5] device(10505708) recv 0xC6 msg
[I]payload len = 84
[I]PAYLOAD (16)
03 03 01 05 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E

[I][6] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[INF][7] device(10505708) recv 0xC6 msg
[INF]payload len = 72
[INF]PAYLOAD (16)
03 03 01 04 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E
```

```
[I][8] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72
```

```
[I][9] device(10505708) recv 0xC6 msg
[I]payload len = 84
[I]PAYLOAD (16)
03 03 01 05 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E
```

5.4.2 配置下发

5.4.2.1 UART

UART 设备，COM_IRQ_THREAD 通知类型。

```
#include "host_driver.h"
#include "host_mmw_api.h"

static DEV_HANDLE devHandle = NULL;

int main()
{
    int status = HOST_ERRCODE_SUCCESS;
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_UART, 0, 1, 921600, 4, 5, 6, 1);

    HOST_LOG_PRINT("\n\n");
    HOST_LOG_INF("start to test application\n\n");

    status = Host_Driver_Init();
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("init host driver fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("init host driver succ\n");
    status = Host_Device_Regist(&devHandle, &devHw, &hifCfg);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("regist device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("regist device(%p) succ\n", devHandle);
    status = Host_Device_Open(devHandle);
    if (status != HOST_ERRCODE_SUCCESS) {
```

```
        HOST_LOG_ERR("open device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("open device(%p) succ\n", devHandle);

    host_os_delayms(20);

    HOST_LOG_INF("start to test command...\n");
    status = HostCmd_Version_Get(devHandle, 0);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("get version fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("end test command!\n");

out:
    HOST_LOG_PRINT("\n\n");
    HOST_LOG_INF("test application done\n\n");
}
```

正常运行的 LOG 示例如下。

```
[I]start to test command...
[I][app_code] ir_type:0 ir_minor:0
im_minor:0 im_project:0 im_number:0
iv_major:2 iv_minor:0 iv_revision:7
[I]end test command!
```

5.5 典型调试过程

5.5.1 数据通路

如果适配层移植完成后，发现无数据上报或命令下发，可以检查初始化流程、LLC 的发送和接收函数、传输层线程执行流程。

5.5.1.1 初始化流程

在初始化过程中，Host_Device_Open 函数最终会调用 com_ops.init 和 com_ops.open，即通信口的初始化和打开，可以在这两个接口中添加 LOG，并观察调用 Host_Device_Open 时，是否调用这两个接口。

5.5.1.2 LLC_Send和LLC_Recv通路

LLC_Send 和 LLC_Recv 分别对接 com_ops.write 和 com_ops.read，即通信口的写和读接口，可以在这两个接口中添加 LOG，并观察数据上报时，是否调用这两个接口。

5.5.2 数据上报

5.5.2.1 HIF_TL_DataNotify_CB

无论主动上报还是被动上报，都会调用 HIF_TL_DataNotify_CB 以通知 TL 层接收数据，可以在该函数中添加 LOG。如果使用的是主动上报，向 Host 的通信口发送数据，Host 在接收到数据时会调用该函数。如果使用的是被动上报，为 Host 连接 Device 通知 IO 的引脚提供上升沿，该函数会被调用。

5.5.2.2 com_ops.interrupt_ctrl

如果出现了线程调度慢等情况，并且是 COM_ISR 通知类型，可能是中断阈值设置失败或设置得过小，可查看 com_ops.interrupt_ctrl 接口在被调用时的 size 参数是否一致是 1 或 0，一般情况下，size 参数应该是由大到小变化的。

例如在接收 3000 字节的 payload 时，size 参数可能的变化情况如下 2967、2933、2899、……、56、24、8、6、4、2、1。

5.5.2.3 协议逻辑

对于主动上报类型，在接收数据的过程中，一般不需要 Host 发送数据给 Device。

对于被动上报类型，接收数据时，需要符合数据上报协议，例如 [SPI 数据上报](#)。如果没有发送 POLL 命令，可观察 HIF_TL_Transport_Task 线程是否调用 HIF_TL_SendPoll 函数，如果 Host 不接收数据，可观察 HIF_TL_Transport_Task 线程是否调用 HIF_TL_RecvMsg 函数，如果有数据接收，但用户设置的回调函数没有收到数据，可观察 HIF_TL_Report_Task 线程是否被执行。

5.5.3 命令下发

5.5.3.1 命令发送请求到HIF_TL_Transport_Task线程的通路

发送命令的函数 Host_General_Command 会调用 HIF_Cmd_Request 向 HIF_TL_Transport_Task 线程发送一个发送命令的请求，如果调用应用层提供的发送命令的 API 之后，Host 没有接收 Device 发送的 Response，可检查该通路是否正常。

如果是主动上报类型，应优先使用逻辑分析仪检查 Device 是否发送 Response。如果是被动上报类型，应优先使用逻辑分析仪检查通知 IO（默认为 PA6）是否拉高。

确认上一步之后，再确认 [HIF_TL_DataNotify_CB](#) 被正确调用。

5.5.3.2 协议逻辑

如果命令正常发送，但是 Device 未能正确响应命令，应优先检查发送的命令是否正确，Device 回复的 Response 的状态码是否为 0，格式参考附录中常用 Command-Response 的介绍。

5.5.4 固件下载

5.5.4.1 同步失败

在使用 Host_BurnMode_Enter 函数将 Device 进入下载模式时，会对设备进行同步，如果同步失败，应确认 RST IO 和 UPD IO 时序是否正确、RST_IO 输出高电平时是否对应 Device 允许、Host 发送 0x7E 后 Device 是否回复 0x59 (AS2525) 或 0x79 (AS2531)，如果回复的是 0x79，Host 是否继续发送 0x55 0xFF，Device 再回复 0x79。

5.6 注意事项

5.6.1 应用层的 C1、C2、C3 自动组包

应用层自动组包时，会保留上一帧数据使用的 Buffer，并在接收下一帧的数据时使用，因此需要在下一帧数据到来之前使用数据或对数据进行备份。

5.6.2 休眠功能

如果需要开启休眠功能，应确保在回调 WKIO 的中断回调函数之前，恢复所有需要使用外设等内容。

6 Device 开发

6.1 代码目录结构

inc 和 src 文件夹中包含 HIF 协议的主要代码，新建工程需要添加这两个文件夹中的代码。

cmd 文件夹中包含一些命令，比如：执行任意函数、read/write memory 等，可根据需要选择添加。

```
<home>/psdf_sdk/subsys/hif
├── cmd                                /* 命令代码目录 */
│   ├── hif_cmd.c                     /* 命令注册/初始化 */
│   ├── hif_cmd.h                     /* 命令声明、配置 */
│   ├── cmd                           /* 命令目录 */
│   │   ├── hif_cmd_0x23_fexec.c      /* 执行任意函数 */
│   │   ├── hif_cmd_0x24_readMem.c    /* read memory */
│   │   └── hif_cmd_0x25_writeMem.c   /* write memory */
├── inc                               /* HIF 协议代码头文件目录 */
│   ├── hif_checksum.h                /* Check Sum 计算 API */
│   ├── hif_config.h                  /* 功能、资源裁剪配置和默认参数宏定义 */
│   ├── hif_dll.h                     /* 数据链路层，以 Frame 为单位处理数据收发 */
│   ├── hif_io.h                      /* IO 控制，Device 通知或唤醒 Host，Host 通过 IO 唤醒 Device */
│   ├── hif_log.h                     /* 调试，分级别打印控制，Trace IO 调试方式控制 */
│   ├── hif_mem.h                     /* 内存池管理，链表操作接口 */
│   ├── hif_ml.h                      /* Msg Level，消息层，主要包含分片功能 */
│   ├── hif_msg.h                     /* HIF 格式、ID、状态码等定义 */
│   ├── hif_phy_can.h                 /* PHY 层 CAN 通信接口，需查看对应版本是否支持 */
│   ├── hif_phy_dma.h                 /* PHY 层 DMA 接口 */
│   ├── hif_phy_i2c.h                 /* PHY 层 I2C 通信接口，需查看对应版本是否支持 */
│   ├── hif_phy_spi.h                 /* PHY 层 SPI 通信接口，需查看对应版本是否支持 */
│   ├── hif_phy_uart.h                /* PHY 层 UART 通信接口，需查看对应版本是否支持 */
│   ├── hif_phy.h                     /* PHY 层公共数据类型及对上层 DLL 接口定义 */
│   ├── hif_pm.h                      /* 休眠唤醒管理 */
│   ├── hif_priv.h                    /* 内部共用数据类型和接口定义 */
│   └── hif_tl.h                      /* frame 传输超时控制 */
```

```

├──hif_type.h          /* 数据类型及错误码定义 */
├──hif.h              /* 对外配置数据结构，接口定义 */
├──src                /* 源文件目录 */
│   ├──hif_checksum.c
│   ├──hif_dll.c
│   ├──hif_io.c
│   ├──hif_mem.c
│   ├──hif_ml.c
│   ├──hif_msg.c      /* Msg 注册、分配以及公共 Msg 处理实现 */
│   ├──hif_phy_can.c
│   ├──hif_phy_dma.c
│   ├──hif_phy_i2c.c
│   ├──hif_phy_spi.c
│   ├──hif_phy_uart.c
│   ├──hif_phy.c
│   ├──hif_pm.c
│   ├──hif_tl.c
│   └──hif.c          /* HIF 主体逻辑实现及对外接口封装 */

```

6.2 使用流程

基本使用方式：

1. 建立工程，将上述文件加入工程；
2. 初始化 HIF；
3. 注册接收回调函数（按需可选）。当 Device 接收到 Host 的数据后，根据 ID 调用相应的回调函数。
4. 上报数据。

例如：

```

/**
 * @brief Message(ID: 0x18) 接收回调 callback
 * @param msg Message Header
 * @return int
 */
int hif_msghdl_0x18_callback(HIF_MsgHdr_t *msg)
{
    /* ack status, 可选状态见 API: HIF_MsgResp */
    uint8_t cmd_status = HIF_CMD_STATUS_SUCCESS;
    .....
}

```

```

        return HIF_MsgResp(msg, respDataTestLen, cmd_status);
    }

int main(void)
{
    int status = HAL_STATUS_OK;

    /* 2. 初始化 */
    HIF_CfgParam_t hifCfgParam;
    HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_SPI, 56000000);

    status = HIF_Init(&hifCfgParam);
    if (status != HAL_STATUS_OK) {
        LOG_PRINT("HIF_Init FAIL (%d)\n", status);
        return status;
    }

    /* 3. 注册接收回调 */
    status = HIF_MsgHdl_Regist(0x18, hif_msghdl_0x18_callback);
    if (status != HAL_STATUS_OK) {
        LOG_PRINT("HIF_MsgHdl_Regist 0x18 FAIL (%d)\n", status);
    }

    /* 4. 发送数据, pData 为数据地址, dataLen 为数据长度, NULL 说明发送回调函数为空 */
    status = HIF_MsgReport(0xC6, pData, dataLen, NULL);

    return 0;
}

```

6.3 配置说明

HIF 以结构体配置/控制 HIF 的功能，结构体为 HIF_CfgParam_t。

默认配置在 hif_config.h 文件。

6.3.1 配置结构体和全局变量

全局变量 HIF_CfgParam_t hifCfg；hifCfg 保存 HIF 的实时配置参数，其中的参数可通过默认、静态、动态进行配置。

结构体 HIF_CfgParam_t 的定义如下：

```

typedef struct {
    /* com
    */
    HIF_ComType_t      comType;
    uint32_t           comId;
    uint32_t           comRate;
    /* uart :
    *         start bit, data bit, stop bit, parity

```

```

*
* i2c :
*     address
* spi :
*     line:
*         0: stand
*         1: dual
*         2: quad
*/
uint32_t          comParam;

HIF_ReportMode_t  reportMode;

/* device notify host
*/
HIF_NotifyType_t   notifyType;
HIF_IOLevel_t     notifyIoLevel;
uint32_t          notifyIoNum;

/* host wakeup device
*/
HIF_WakeType_t     wakeType;
HIF_IOLevel_t     wakeIoLevel;
uint32_t          wakeIoNum;
uint32_t          wakeComParam;

/* timeout
*/
/* begin receive frame magic to all frame
*/
uint32_t          rcvFrameTo;
/*
* from the start of sending to successful sending.
*/
uint32_t          sendFrameTo;
/* configuration phase
* after waking up the device but before sending any cmd,
* if a timeout occurs, this is marked as a false wake-up.
*/
uint32_t          sendDelay;
/* configuration phase
* a series of cmd were sent, but the sleep cmd was not sent for a long time.
* after this timeout, the system will auto enter sleep mode.
*
* calc start from the resoinse to the last cmd.
*/
uint32_t          sleepTo;

uint32_t          splitSize;
uint32_t          splitTotalSize;
uint32_t          splitEna;
uint32_t          splitRetryEna;

```

uint32_t	reportInitStatus;
uint32_t	retryEna;
uint32_t	burstEna;
uint32_t	sendMode;
uint32_t	recvMode;
uint32_t	cmdrspBuffSize;
uint32_t	frameNodeSize;
uint32_t	dataNodeSize;
uint32_t	semNodeSize;
uint32_t	taskPrio;
uint32_t	taskStackSize;
uint32_t	cascadeThresh;
uint32_t	msgNodeSize;
uint32_t	sendNodeLevel;
} HIF_CfgParam_t;	

6.3.2 默认配置

即，hif_config.h 文件中的 HIF 默认配置。

6.3.2.1 基本配置

表 6-1 HIF 基本配置

配置项	说明	Default
CONFIG_HIF	为 0 则关闭 HIF 功能，代码不编译；为 1 则开启 HIF 功能	0
CONFIG_HIF_MODE	HIF 模式配置： 0 - 极低功耗模式； 1 - 标准模式。 (以下配置说明基于标准模式)	1
CONFIG_HIF_TASK_STACK_SIZE	HIF Task 栈配置。	1024
CONFIG_HIF_TASK_PRIO	HIF Task 优先级	14
CONFIG_HIF_MEM_CMD_RSP_SIZE	接收 command Buffer size，最大接收命令长度，单位 Byte。	512

配置项	说明	Default
CONFIG_HIF_MSG_PAYLOAD_MAX_SIZE	上报 Frame 的 Payload 最大长度，最大有效值 4095。	3839
CONFIG_HIF_MEM_FRAME_NODE_SIZE	Frame 内存池大小。	16
CONFIG_HIF_MEM_DATA_NODE_SIZE	Data Node 内存池大小。	64
CONFIG_HIF_MEM_SEM_NODE_SIZE	Sem 信号量内存池大小（用于阻塞式上报数据）。	4
CONFIG_HIF_PHY_TYPE	支持的物理通信方式，bit 0 表示 UART，bit 1 表示 IIC，bit 2 表示 SPI，bit 3 表示 CAN。对应 bit 为 1 则表示支持此通信方式，如值为 5 表示支持 UART 和 SPI 通信方式。	0x07

6.3.2.2 功能配置

本章节对 HIF 的一些可选功能的配置项进行说明。

表 6-2 HIF 功能配置项

配置项	说明	Default
CONFIG_HIF_RETRY_ENA	传输层重传使能与否： 0 - 不支持传输层重传，1 - 支持传输层重传。	0
CONFIG_HIF_SPLIT_ENA	分片功能使能与否： 0 - 不支持分片，1 - 支持分片。	0
CONFIG_HIF_MEM_MSG_NODE_SIZE	HIF Msg 内存池大小。用于分片功能，在分片功能使能下有效。	4
CONFIG_HIF_SEND_SPLIT_RETRY_ENA	分片重传使能与否： 0 - 不支持分片重传，1 - 支持分片重传。 分片重传的前提是使能分片功能。	0
CONFIG_HIF_PM	HIF PM 使能与否： 0 - 不支持 PM，1 - 支持 PM。 该配置跟随 CONFIG_PM，即系统支持 PM 则 HIF 也支持。	CONFIG_PM

6.3.3 其他配置

更细节的功能配置，比如：UART 的发送使用 DMA 或是中断传输，IIC 的默认通信地址，Notify IO 的默认 GPIO 及电平等等。建议不修改。

6.3.4 配置与使用方法

6.3.4.1 使用默认配置

即，按照前述章节的默认配置初始化/使用 HIF。这是最简单、直接的使用方式。

例如，SPI、Uart、IIC 使用默认配置的初始化方式如下：

```
/* 1. 使用默认配置以“SPI+56Mhz”初始化 HIF */
HIF_CfgParam_t hifCfgParam;
HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_SPI, 56000000);

status = HIF_Init(&hifCfgParam);
if (status != HAL_STATUS_OK) {
    LOG_PRINT("HIF_Init FAIL (%d)\n", status);
    return status;
}

/* 2. 使用默认配置以“UART+921600baud”初始化 HIF */
HIF_CfgParam_t hifCfgParam;
HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_UART, 921600);

status = HIF_Init(&hifCfgParam);
if (status != HAL_STATUS_OK) {
    LOG_PRINT("HIF_Init FAIL (%d)\n", status);
    return status;
}

/* 3. 使用默认配置以“IIC+400Khz”初始化 HIF */
HIF_CfgParam_t hifCfgParam;
HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_IIC, 400000);

status = HIF_Init(&hifCfgParam);
if (status != HAL_STATUS_OK) {
    LOG_PRINT("HIF_Init FAIL (%d)\n", status);
    return status;
}
```

6.3.4.2 工程静态配置

不建议修改 hif_config.h 中的内容。

对于不同的 Project，如果需要 HIF 的不同功能或配置，比如：重传、分片、HIF 内存池大小配置等等，则可以在工程的 prj_config.h 中通过相同的宏名重新定义该配置，工程的 prj_config.h 中的宏会覆盖 hif_config.h 中的对应宏，从而达到修改 HIF 部分配置以实现不同需求或功能的目的。

例如：

如下 prj_config.h 中重新定义了 HIF 的一些配置：

```
#ifndef _PRJ_CONFIG_H
#define _PRJ_CONFIG_H
```

```

#define CONFIG_XIP 1

#define CONFIG_PM 1
/* 使能 HIF */
#define CONFIG_HIF 1
/* 修改 hif_Task 的优先级 */
#define CONFIG_HIF_TASK_PRIO 8
/* 修改 hif_Task 的堆栈大小 */
#define CONFIG_HIF_TASK_STACK_SIZE 2048
/* 修改上报 Frame 的 Payload 的最大长度 */
#define CONFIG_HIF_MSG_PAYLOAD_MAX_SIZE 1024
/* 修改接收 Buffer 的大小（容纳 Frame 的 Header、Payload、Checksum） */
#define CONFIG_HIF_CMD_RSP_SIZE_MIN 512
/* 分片使能 */
#define CONFIG_HIF_SPLIT_ENA 1

#endif /* _PRJ_CONFIG_H */

```

之后，该工程会按照以上配置项和 hif_config.h 的其他配置项初始化 HIF。

例如：

```

HIF_CfgParam_t hifCfgParam;
HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_SPI, 56000000);

status = HIF_Init(&hifCfgParam);
if (status != HAL_STATUS_OK) {
    LOG_PRINT("HIF_Init FAIL (%d)\n", status);
    return status;
}

```

6.3.4.3 动态配置

在 HIF 初始化后，仍可通过接口 HIF_ExtControl() 修改某项参数，适合运行时参数调整的场景。

例如：

```

/* 修改发送超时为 50ms */
uint32_t sendTo = 50;
status = HIF_ExtControl(HIF_SET_SEND_FRAME_TO, &sendTo, sizeof(uint32_t));
if (status != 0) {
    return status;
}

```

6.4 API 使用说明

6.4.1 状态码

表 6-3 状态码

Status	值	含义
HIF_ERRCODE_SUCCESS	0	命令执行成功
HIF_ERRCODE_NOT_READY	-1	状态错误，未准备好
HIF_ERRCODE_INVALID_PARAM	-2	无效参数
HIF_ERRCODE_NO_BUFFER	-3	没有足够 buffer
HIF_ERRCODE_IO_ERROR	-4	传输错误
HIF_ERRCODE_TIMEOUT	-5	超时
HIF_ERRCODE_ABORTED	-6	强制终止
HIF_ERRCODE_TOO_LONG	-7	数据长度溢出
HIF_ERRCODE_FAIL	-8	通用失败

6.4.2 API

6.4.2.1 HIF_DefaultInitParam

接口	说明
函数	<pre>int HIF_DefaultInitParam(HIF_CfgParam_t * cfgParam, HIF_ComType_t comType, uint32_t comRate);</pre>
功能	设置 HIF 参数默认配置。
参数	HIF_CfgParam_t * cfgParam HIF 参数配置结构体。 HIF_ComType_t comType 通信类型，比如：SPI、Uart 等。 uint32_t comRate 物理接口的通信速率。

接口	说明
注意事项	无。
返回值	若为零则成功，非零失败，详见状态码。

6.4.2.2 HIF_Init

接口	说明
函数	int HIF_Init(HIF_CfgParam_t *pInitParam);
功能	初始化 HIF。
参数	HIF_CfgParam_t * pInitParam HIF 参数配置结构体。
注意事项	pInitParam 可以为 NULL，若为 NULL 需要在此函数前调用 HIF_ExtControl 函数的 HIF_SET_CFG_PARAM 将参数配置进入 HIF，否则初始化失败。 若 pInitParam 非 NULL，则无论是否调用 HIF_ExtControl 配置过参数均以 pInitParam 参数初始化。
返回值	若为零则成功，非零失败，详见状态码。

6.4.2.3 HIF_ExtControl

接口	说明
函数	int HIF_ExtControl(HIF_Attr_t attr, void * arg, uint32_t argSize);
功能	获取或设置 HIF 配置参数。
参数	HIF_Attr_t attr 配置类型，HIF_SET_CFG_PARAM 和 HIF_GET_CFG_PARAM 是获取或配置整个参数，其他是获取或配置单个参数。 void * arg 配置参数 buff。 uint32_t argSize 配置参数长度
注意事项	HIF_SET_CFG_PARAM 和 HIF_GET_CFG_PARAM 的参数长度为 sizeof(HIF_Attr_t) 其他类型，参数长度固定为 4。 此函数可在 HIF_Init 前或后调用。

接口	说明
返回值	若为零则成功，非零失败，详细见状态码。

6.4.2.4 HIF_MsgReport_ListPushData

接口	说明
函数	<pre>int HIF_MsgReport_ListPushData(HIF_Data_ListHead_t * pDataListHead, void *data, uint16_t len);</pre>
功能	将数据压入链表中，压入位置为链表头。
参数	HIF_Data_ListHead_t * pDataListHead 数据链表。 void *data 数据地址。 uint16_t len 数据长度。
注意事项	一个上报第一个节点，双头链表 pDataListHead 需要初始化，即 pDataListHead->head 和 pDataListHead->tail 都需要初始化为 NULL。 若失败，应使用 HIF_MsgReport_ListPopData() 释放 HIF 内部资源。
返回值	若为零则成功，非零失败，详细见状态码。

6.4.2.5 HIF_MsgReport_ListPopData

接口	说明
函数	<pre>uint8_t *HIF_MsgReport_ListPopData(HIF_Data_ListHead_t * pDataListHead);</pre>
功能	弹出链表头节点中的数据地址，同时，链表头结点后移。
参数	HIF_Data_ListHead_t * pDataListHead 数据链表。
注意事项	无。
返回值	用户数据 buffer。若为 NULL 表示当前链表已空。

6.4.2.6 HIF_MsgReport_ListStart

接口	说明
函数	<pre>int HIF_MsgReport_ListStart(uint8_t msgId, HIF_Data_ListHead_t * pDataListHead, HIF_MsgTrans_Callback sentCb);</pre>
功能	启动链表式数据发送。
参数	uint8_t msgId 消息 ID。 HIF_Data_ListHead_t * pDataListHead 上报数据双头链表。不可为 NULL。 HIF_MsgTrans_Callback sentCb 发送回调函数。 若为 NULL，为阻塞式发送，此接口在发送结束（成功或失败）后返回。 若非 NULL，此接口立即返回，在数据发送结束后调用 sentCb 。 在 sentCb 中，调用 HIF_MsgReport_ListPopData() 可弹出发送数据地址。
注意事项	● sentCb == NULL 时，返回后，需要使用 HIF_MsgReport_ListPopData() 释放 hif buffer。 ● sentCb != NULL 时： ■ 返回失败时，需要使用 HIF_MsgReport_ListPopData() 释放 hif buffer。 ■ 返回成功时，在 sentCb 中，调用 HIF_MsgReport_ListPopData() 可弹出发送数据地址。
返回值	若为零则成功，非零失败，详细见状态码。

6.4.2.7 HIF_MsgReport

接口	说明
函数	<pre>int HIF_MsgReport(uint8_t msgId, void * pData, uint32_t len, HIF_MsgTrans_Callback sentCb);</pre>
功能	非链表上报数据。

接口	说明
参数	uint8_t msgId 消息 ID。 void *data 用户数据。 uint32_t len 数据长度。 HIF_MsgTrans_Callback sentCb 发送回调函数。 若为 NULL，为阻塞式发送，此接口在发送结束（成功或失败）后返回。 若非 NULL，此接口立即返回，在数据发送结束后调用 sentCb 。 在 sentCb 中，调用 HIF_MsgReport_ListPopData() 可弹出发送数据地址。
注意事项	当 sentCb != NULL 时，如果返回值为成功，则在回调未执行前，不可修改用户数据或释放 *data。
返回值	若为零则成功，非零失败，详细见状态码。

6.4.2.8 HIF_MsgHdl_Regist

接口	说明
函数	int HIF_MsgHdl_Regist(uint8_t msgId, HIF_MsgHdl_t msgHdl);
功能	消息处理函数注册。
参数	uint8_t msgId 消息 ID。 HIF_MsgHdl_t msgHdl 消息处理函数指针。
注意事项	无。
返回值	若为零则成功，非零失败，详细见状态码。

6.4.2.9 HIF_MsgResp

接口	说明
----	----

接口	说明
函数	int HIF_MsgResp(HIF_MsgHdr_t *pMsg, uint16_t dataLen, uint8_t status);
功能	用于在消息处理函数中回复 host。
参数	HIF_MsgHdr_t *pMsg 消息头，Cmd ID 需与 cmd 保持一致。 uint16_t dataLen Payload 长度，不包括 status。 uint8_t status Cmd 执行状态。 <pre> /! @brief General Ack Data */ #define HIF_CMD_STATUS_SUCCESS 0 /**< Cmd Success */ #define HIF_CMD_STATUS_UNSUPPORT 1 /**< Unsupport Cmd */ #define HIF_CMD_STATUS_VERSION 2 /**< Version not match */ #define HIF_CMD_STATUS_TOOLONG 3 /**< Message too long */ #define HIF_CMD_STATUS_CHECK_FAILED 4 /**< Head or Data check failed */ #define HIF_CMD_STATUS_PARAM 5 /**< Invalid Param */ #define HIF_CMD_STATUS_REPLAY 6 /**< Replay message */ #define HIF_CMD_STATUS_AUTH 7 /**< Auth failed */ #define HIF_CMD_STATUS_LATE 8 /**< Result need report later */ #define HIF_CMD_STATUS_BUSY 9 /**< Device is busy now */ #define HIF_CMD_STATUS_STATE 10 /**< Invalid Device State to process Cmd */ #define HIF_CMD_STATUS_SYSERR 11 /**< System error in Cmd Handler */ #define HIF_CMD_STATUS_IO 12 /**< General error */ </pre>
注意事项	无。
返回值	若为零则成功，非零失败，详细见状态码。

6.4.2.10 HIF_InitStatusReport

接口	说明
函数	int HIF_InitStatusReport(uint8_t status);
功能	上报系统启动状态
参数	uint8_t status 系统启动状态。

接口	说明
注意事项	需在 HIF_Init 前调用。
返回值	若为零则成功，非零失败，详细见状态码

6.4.2.11 HIF_MsgReport_Lock

接口	说明
函数	void HIF_MsgReport_Lock(void);
功能	阻止系统上报数据。
参数	无
注意事项	在配置参数时，阻止数据上报，配置完成后解除阻止。
返回值	无

6.4.2.12 HIF_MsgReport_Unlock

接口	说明
函数	void HIF_MsgReport_Unlock(void);
功能	解除对系统上报数据的阻止。
参数	无
注意事项	在配置参数时，阻止数据上报，配置完成后解除阻止。
返回值	无

6.4.2.13 HIF_PM_WakeupDevice

接口	说明
函数	void HIF_PM_WakeupDevice(void);
功能	唤醒 HIF。
参数	无
注意事项	无
返回值	无

6.4.2.14 HIF_PM_SuspendDevice

接口	说明
函数	void HIF_PM_SuspendDevice(void);
功能	挂起 HIF
参数	无
注意事项	无
返回值	无

6.4.3 数据上报

数据上报有两类接口：链表式数据上报和非链表式数据上报。

- 链表式数据上报：当需要将几个分散的数据组合成一个包发送出去时，通常做法是将数据复制到一个发送 Buffer 进行组包，但这样会增加 Data Size 和复制数据的时间，且当多个包排队时情况会更严重。链表式数据上报将待发送的数据用链表串起来，减少发送 buffer 的备份和复制时间。

示例场景一：

有三块地址不连续的 Buffer：Buffer_1、Buffer_2、Buffer_3，需要将这三个 Buffer 组成一个数据作为一帧 HIF Frame 的 Payload 发送出去，可用链表式数据上报。



图 6-1 链表式数据上报示例场景一

场景示例二：

BufferA 分为 3 块：BufferA_1、BufferA_2、BufferA_3，在每块 Buffer 之前需要添加一些数据头：Header_1、Header_2、Header_3，组成如下数据结构，并且将这个数据作为一帧 HIF Frame 的 Payload 发送出去，可用链表式数据上报。

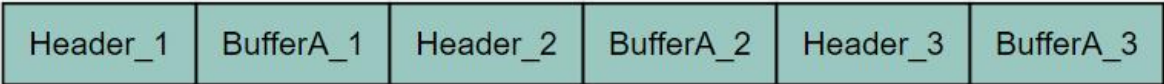


图 6-2 链表式数据上报示例场景二

- 非链表式数据上报：所有需要上报的数据存在同一块 buffer，可以直接将地址内的数据上报。

上报方式分阻塞式和回调式。

- 阻塞式：调用上报接口上报数据时，不传入回调函数（NULL）；接口在将数据传入 HIF 内部后，不会返回，直到 HIF 将数据发送结束。

- 回调式/非阻塞式：调用上报接口上报数据时，可传入回调函数；接口在将数据传入 HIF 内部后，会直接返回；当 HIF 将数据发送结束时，会调用回调函数。这种方式适用多并发场景。

6.4.3.1 链表阻塞式发送

相关接口：[*HIF_MsgReport_ListPushData*](#)、[*HIF_MsgReport_ListPopData*](#)、[*HIF_MsgReport_ListStart*](#)。

注意事项：

- 如果 `HIF_MsgReport_ListPushData` 向链表中添加数据失败，需停止并调用 `HIF_MsgReport_ListPopData` 释放链表中的 HIF 内部资源。
- `HIF_MsgReport_ListStart` 返回后，需调用 `HIF_MsgReport_ListPopData` 释放链表中的 HIF 内部资源。
- 其他注意事项见相关 API 说明。

示例：

```
int status = 0;
/* 初始化上报数据链表，注意：一定要将链表头和尾赋值 NULL */
HIF_Data_ListHead_t dataList = {NULL, NULL};

do {
    /* 将上报数据 pRptDataA 加入发送队列 */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataA, rptLenA);
    if (status != 0) {
        /* 一定要判断返回值，若失败，应停止添加并调用 HIF_MsgReport_ListPopData 释放链表中的 HIF 内部资源，否则会造成内存泄漏 */
        break;
    }

    /* 将上报数据 pRptDataB 加入发送队列 */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataB, rptLenB);
    if (status != 0) {
        break;
    }

    /* 将上报数据 pRptDataC 加入发送队列 */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataC, rptLenC);
    if (status != 0) {
        break;
    }

    /* 启动上报数据传输。若第三个参数是 NULL 则表示阻塞发送，即传输完成才会返回 */
    status = HIF_MsgReport_ListStart(0xC6, &dataList, NULL);
} while (0);

/* 一定要进行资源回收，否则会泄漏内存 */
uint8_t *pBuff = NULL;
do {
```

```
/* 循环弹出 buff，在弹出过程中会释放 HIF 内部资源 */
pBuff = HIF_MsgReport_ListPopData(&dataList);
if (pBuff == NULL) {
    /* 为空，则表示链表中的所有地址均已弹出 */
    break;
}

/* 应用自行管理 pBuff，比如 free 等 */
.....
} while (1);
```

6.4.3.2 链表回调式发送

相关接口：[HIF_MsgReport_ListPushData](#)、[HIF_MsgReport_ListPopData](#)、[HIF_MsgReport_ListStart](#)。

注意事项：

- 如果 HIF_MsgReport_ListPushData 向链表中添加数据失败，需停止并调用 HIF_MsgReport_ListPopData 释放链表中的 HIF 内部资源。
- 如果 HIF_MsgReport_ListStart 返回失败，需调用 HIF_MsgReport_ListPopData 释放链表中的 HIF 内部资源。
- 其他注意事项见相关 API 说明。

示例：

```
int status = 0;
/* 初始化上报数据链表，注意：一定要将链表头和尾赋值 NULL */
HIF_Data_ListHead_t dataList = {NULL, NULL};

do {
    /* 将上报数据 pRptDataA 加入发送队列 */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataA, rptLenA);
    if (status != 0) {
        /* 一定要判断返回值，若失败，应停止添加并调用 HIF_MsgReport_ListPopData 释放链表中的 HIF 内部资源，否则会造成内存泄漏 */
        break;
    }

    /* 将上报数据 pRptDataB 加入发送队列 */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataB, rptLenB);
    if (status != 0) {
        break;
    }

    /* 将上报数据 pRptDataC 加入发送队列 */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataC, rptLenC);
    if (status != 0) {
        break;
    }
}
```

```

    }

    /* 启动上报数据传输。若第三个参数是回调函数 report_callback 则表示回调式发送 */
    status = HIF_MsgReport_ListStart(0xC6, &dataList, report_callback);
} while (0);

/* 失败时，需要释放资源 */
if (status != 0) {
    /* 一定要进行资源回收，否则会泄露内存 */
    uint8_t *pBuff = NULL;
    do {
        /* 循环弹出 buff，在弹出过程中会释放 HIF 内部资源 */
        pBuff = HIF_MsgReport_ListPopData(&dataList);
        if (pBuff == NULL) {
            /* 为空，表示链表中的所有地址均已弹出 */
            break;
        }

        /* 应用自行管理 pBuff，比如 free */
        .....
    } while (1);
}

/**
 * @brief 回调函数
 * @param msgId Message ID
 * @param pDataListHead 调用 HIF_MsgReport_ListStart 时传入的链表
 * @param status 数据发送状态
 */
void report_callback(uint8_t msgId, HIF_Data_ListHead_t * pDataListHead, int8_t status)
{
    /* 进行资源释放。
       若不需要对链表 pDataListHead 中的应用层 buffer 进行管理，也可以不添加以下释放代码，
       HIF 驱动会自动回收 HIF 的内部资源 */
    uint8_t *pBuff = NULL;
    do {
        pBuff = HIF_MsgReport_ListPopData(&dataList);
        if (pBuff == NULL) {
            break;
        }

        /* 应用自行管理 pBuff，比如 free 等 */
        .....
    } while (1);
}

```

6.4.3.3 非链表阻塞式发送

相关接口：[HIF_MsgReport](#)。

示例：

```
/* 回调函数为 NULL，阻塞式发送 */
status = HIF_MsgReport(0xC6, pData, dataLen, NULL);
```

6.4.3.4 非链表回调式发送

相关接口：[HIF_MsgReport](#)。

示例：

```
/* 回调函数为 report_callback，回调/非阻塞式发送 */
status = HIF_MsgReport(0xC6, pData, dataLen, report_callback);

/**
 * @brief 回调函数
 * @param msgId Message ID
 * @param pDataListHead 调用 HIF_MsgReport_ListPopData(pDataListHead) 可弹出 HIF_MsgReport
   传入的数据地址
 * @param status 数据发送状态
 */
void report_callback(uint8_t msgId, HIF_Data_ListHead_t * pDataListHead, int8_t status)
{
    /* 链表 pDataListHead 有且仅有一个节点——头结点，该节点中保存有 HIF_MsgReport 传入的数
    据地址，调用 HIF_MsgReport_ListPopData(pDataListHead) 可弹出该地址。
    下方弹出数据地址的代码可选择是否添加。*/
    uint8_t *pBuff = NULL;
    do {
        pBuff = HIF_MsgReport_ListPopData(&dataList);
        if (pBuff == NULL) {
            break;
        }

        /* 应用自行管理 pBuff，比如 free 等 */

        .....
    } while (1);
}
```

6.4.4 Command-Response

对于 Device，从 Host 接收消息就是接收 Command。Device 通过 `HIF_MsgHdl_Regist` 注册接收消息处理回调函数（在处理函数中需要对该消息进行回复）。Device 收到相应 ID 的数据时会调用相应的回调函数。

一般，开发流程如下：

1. 创建消息 (ID) 的处理回调函数 Callback;
2. Callback 中获取 Host 发送的数据, 并进行处理与回复;
3. 注册 Callback。

6.4.4.1 消息处理回调函数原型

```
/**
 * @brief Received-message handler type
 * @param msg Message header
 * @attention The address of user data is (msg + 1)
 */
typedef int (* HIF_MsgHdl_t)(HIF_MsgHdr_t *msg);
```

6.4.4.2 Response说明

消息回复接口: HIF_MsgResp。

回复消息遵循 HIF 帧格式见前述章节 [HIF 协议帧格式](#) 中的 [Command-Response](#)。

HIF_MsgResp 接口中的第二个参数 dataLen 指的是“回复数据”的长度, 第三个参数 status 指的是“Ack Status”; 第一个参数可沿用消息处理函数的入参。

```
/**
 * @brief Call this function to respond to host in a message-received handler
 * @param pMsg Message handler
 * @param dataLen The length of data responded to host
 * @param status The ack status responded to host
 * @return int
 */
int HIF_MsgResp(
    HIF_MsgHdr_t *pMsg,
    uint16_t dataLen,
    uint8_t status);
```

6.4.4.3 接收与回复示例

```
/**
 * @brief Message(ID: 0x18) callback
 * @param msg Message Header
 * @return int
 */
int hif_msghdl_0x18_callback(HIF_MsgHdr_t *msg)
{
    /* ack status, 可选状态见 API: HIF_MsgResp */
    uint8_t cmd_status = HIF_CMD_STATUS_SUCCESS;

    /* 接收到的数据的地址 (HIF 帧结构中的 payload 地址) */
    uint8_t *payloadAddr = NULL;

    /* 回复的数据长度 */
    uint16_t respDataTestLen = 0;

    for (uint8_t i = 0; i < RESP_DATA_TEST_LEN; i++) {
```

```

        respDataTest[i] = i;
    }

    /* 计算 payload 地址 */
    payloadAddr = (uint8_t*)(msg + 1);
    /* 支持扩展头时，需要偏移扩展头的长度 */
    if (msg->flag & HIF_MSG_FLAG_EXTEND_BIT) {
        payloadAddr += sizeof(HIF_MsgExtHdr_t);
    }

    /* 状态检查，这里是回复数据的长度校验（其他检验或状态，用户可自行定义）。
       “+1” 是因为 ack status 需要占据一个字节 */
    if ((RESP_DATA_TEST_LEN + 1) > CONFIG_HIF_CMD_PAYLOAD_MAX_SIZE) {
        respDataTestLen = 0;
        cmd_status = HIF_CMD_STATUS_TOOLONG;
    } else {
        /* “Ack Status” 占据 HIF 帧格式中 payload 的第一个字节位置，所以回复数据 respDataTest
        的起始地址为 payloadAddr + 1 */
        memcpy(payloadAddr + 1, respDataTest, RESP_DATA_TEST_LEN);
        respDataTestLen = RESP_DATA_TEST_LEN;
    }

    /* 第一个参数（如果无特殊需求，比如修改 Message header 中的某些字段）可沿用消息处理
    函数的入参；
       第二个参数 dataLen 指的是“回复数据”的长度；
       第三个参数 status 指的是“Ack Status”。*/
    return HIF_MsgResp(msg, respDataTestLen, cmd_status);
}

.....

/* 注册 0x18 的消息处理函数 hif_msghdl_0x18_callback。
   注册成功后，当 HIF 接收到 ID 为 0x18 的消息时，会调用 hif_msghdl_0x18_callback。 */
status = HIF_MsgHdl_Regist(0x18, hif_msghdl_0x18_callback);

```

6.5 雷达数据上报流程

本章节说明雷达数据在调用 HIF API 之前的流程，对于 HIF 的上报流程，参考前述章节。

雷达原始数据一般流程如下：

1. 雷达数据由雷达 IP 生成并预处理；
2. 雷达中断触发，产生雷达数据 Ready 事件；
3. 雷达线程处理雷达数据 Ready 事件，执行应用层注册的雷达数据处理回调函数（用户注册）；

4. 雷达数据处理回调函数进行数据获取、算法处理等操作，得到不同类型的数据（0xC2、0xC3、0xC6 等），然后通过 HIF 接口上报给 Host。

相关接口：

- mmw_callback_handle - 雷达中断；
- mmw_ctrl_task - 雷达线程；
- mmw_data_handle - 雷达数据处理回调函数，通过接口 mmw_ctrl_callback_cfg 注册，例如：
mmw_ctrl_callback_cfg(radar_framework_2d_frame_cb, MMW_DATA_TYPE_2DFFT, NULL)。

6.5.1 0xC2 雷达 FFT 数据

0xC2 是雷达 FFT 数据在 HIF 协议中的 MsgID。0xC2 的结构和各字段信息见前述章节 [HIF 协议帧格式](#)。

0xC2 上报流程：

1. 雷达线程调用雷达数据处理回调函数：radar_framework_2d_frame_cb；
2. 回调函数中获取雷达数据并进行处理，然后调用接口 radar_framework_report_cube；
3. radar_framework_report_cube 对雷达数据进行组包（0xC2），然后调用 radar_framework_report_cube_split 向 Host 上报雷达数据。

6.5.2 0xC3 雷达点云数据

0xC3 是雷达点云数据在 HIF 协议中的 MsgID。0xC3 的结构和各字段信息见前述章节 [HIF 协议帧格式](#)。

0xC3 上报流程：

1. 雷达线程调用雷达数据处理回调函数：radar_framework_2d_frame_cb；
2. 调用接口 radar_framework_motion_point_process（微动点云）、mmw_presence_point_process（存在点云）获取点云数据；
3. 调用接口 radar_framework_report_point_cloud_data 将点云数据按照 0xC3 的结构组成数据包：
 - (1) 0xC3 的帧头 MMW_FRAME_UPLOAD：frame_idx、frame_len、offset 等；
 - (2) 微动点云数据 ptr_point_cloud_buffer；
 - (3) 存在点云数据 ptr_presence_point_cloud_buffer；
4. 然后调用 HIF_MsgReport 向 Host 上报 0xC3 数据。

6.5.3 0xC6 “PSIC-调试协议” 数据

0xC6 是“PSIC-调试协议”数据在 HIF 协议中的 MsgID。0xC6 的结构和各字段信息见前述章节 [HIF 协议帧格式](#)。

0xC6 上报流程：

1. 雷达线程调用雷达数据处理回调函数：radar_framework_2d_frame_cb；
2. 调用接口 radar_framework_report_frameInfo：

- (1) 获取雷达数据：txrx_mode、work_mode、range_mm、resol_mm、velocity_mm、veloc_resol、period_ms、frame_num、rfft_num、dfft_num；
 - (2) 调用 radar_framework_report_debug_data；
3. radar_framework_report_debug_data 根据 0xC6 的数据结构构造数据包：
 - (1) Debug Payload Header：Dim、Data Formats、Len；
 - (2) Payload Name；
 - (3) Payload Data；
4. 完成 0xC6 数据包构造后，将该数据包通过 HIF 发送给 Host。

7 自定义 Msg ID

如果用户需要自定义上报数据或雷达 HIF 命令，可以通过本章节了解如何实现。

7.1 允许自定义的 Msg ID 范围

HIF 支持增加自定义 MsgID，包括自定义命令 ID 和自定义上报 ID 两个部分。HIF 向用户开放的命令 ID 范围为 0x90~0x9F，上报 ID 范围为 0xE0~0xEF。

7.2 自定义命令

自定义命令 ID 需要 Host 端和 Device 端分别支持和定义。Host 需要创建相应的 MsgID 发送指令，填充完成 payload 信息后调用 Host_General_Command 函数即可将命令发送。Device 需要创建接收到相应 MsgID 的回调处理函数，在 msg 初始化后调用 HIF_MsgHdl_Regist 注册相应的 MsgID 回调函数。

7.2.1 Host 端开发步骤

示例：创建一个 MsgID 为 0x90，payload_length 为 3，payload[0] = 0x10，payload[1] = 0x11，payload[2] 为参数，返回值 resp_length 为 1 的配置命令，device 接收成功返回值为 0x00。即发送 3 个字节的 payload，接收 1 个字节的返回 resp。

- A：配置 msgid 以及 payload 信息；
- B：调用 Host_General_Command 函数；
- C：判断返回值以及 Device resp；
- D：声明该命令函数。

在 host_mmw_cmd_api.c 中创建该函数。

```
/* DevHdl 为设备句柄，在 Host_Device_Open 时获取*/
int MmwCmd_R3_User_Define_Cfg(DEV_HANDLE DevHdl, uint8_t user_value)
{
    int ret = 0;
    uint32_t msg_id = 0x90;    //配置命令 msgid = 0x90;
    uint32_t param_len = 3;    //配置 payload_length = 3;
    uint32_t buffer_len = 1;   //配置 resp_length = 1;
    uint8_t param[3];          //payload buffer
    uint8_t resp_buff[1];      //resp buffer
    param[0] = 0x10;           //赋值 payload[0]
    param[1] = 0x11;           //赋值 payload[1]
    param[2] = user_value;     //赋值 payload[2]

    /* Host 通过 Host_General_Command 接口将命令发送到 device */
    ret = Host_General_Command(DevHdl, msg_id, param, param_len, resp_buff, buffer_len, NULL);
    if (ret != HOST_ERRCODE_SUCCESS) {
```

```

        return ret;           //判断是否发送成功，不成功则返回状态值
    } else {
        return resp_buff[0]; //发送成功则返回 resp
    }
}

```

在 host_mmw_api.h 中声明该函数。

```
int MmwCmd_R3_User_Define_Cfg(DEV_HANDLE DevHdl, uint8_t user_value);
```

7.2.2 Device 端开发步骤

示例：创建一个 MsgID 为 0x90，接收 payload_length 为 3。当 payload[0] = 0x10，payload[1] = 0x11 时，将 payload[2]打印出来的命令。若打印成功返回 HIF_CMD_STATUS_SUCCESS（0），若失败则返回 HIF_CMD_STATUS_UNSUPPORT（1）。

- A：创建 MsgID 命令回调函数；
- B：获取 payload 信息；
- C：添加用户自定义逻辑代码；
- D：调用 HIF_MsgResp 回复 Host；
- E：调用 HIF_MsgHdl_Regist 注册该回调函数。

```

int r3_user_define_handler(HIF_MsgHdr_t *msg)
{
    int ret = HIF_CMD_STATUS_SUCCESS;
    if (msg->length != 3) {           //判断接收到的 payload_length 是否为 3
        return HIF_CMD_STATUS_PARAM;
    }
    uint8_t *pId = (uint8_t *)(msg + 1); //获取 payload
    if (pId[0] == 0x10 && pId[1] == 0x11) { //判断 payload 是否满足条件
        printf("payload[2]:0x%02x\n", (pId[2]));
    } else {
        ret = HIF_CMD_STATUS_UNSUPPORT; //接收失败则返回 HIF_CMD_STATUS_UNSUPPORT
    }
    return HIF_MsgResp(msg, 0, ret); //接收成功则返回 HIF_CMD_STATUS_SUCCESS
}

HIF_MsgHdl_Regist(0x90, r3_user_define_handler); //注册 0x90 回调函数

```

若命令 ID 不够，可以采用 subid（子 ID）的方式来扩展控制，即将 payload 第一个字节当作子 ID，在相应的 MsgID 回调处理函数中分别处理。

7.3 自定义上报数据

自定义上报 ID 需要 Host 端和 Device 端分别支持。Host 端创建回调处理函数，然后通过 HIF_Report_Handle_Regist 注册回调，Device 端调用 HIF_MsgReport 将数据发出。

7.3.1 Host 端开发步骤

参考 [自定义数据处理方案](#)。

7.3.2 Device 端开发步骤

示例：创建一个 MsgID 为 0xE0 的上报 ID 处理函数。

参考前述章节 [Device 开发](#) 中的 [数据上报](#)。

8 Q&A

8.1 通信问题

8.1.1 HOST 发送 payload 大于等于 1024 字节的消息时出现 checksum 校验问题

SDK V2.1.0 版本仅支持向雷达（device）发送消息总长度（包括消息头、payload、checksum32 等）小于 512 字节的 HIF 消息，如果大于或等于 512 字节（可配置），该消息将被雷达丢弃。

雷达接收命令时判断为同一条消息的依据是处于同一次使能 CS 期间发送的数据内容。

8.1.2 为什么使用 host driver 主动上报 COM_IRQ_THREAD 通知时会接收一包数据但连续遗漏多包数据

COM_IRQ_THREAD 通知类型会在线程中接收数据，如果每一包数据的回调占用时间过多，会导致数据接收不及时，遗漏数据，因此产生了连续遗漏多包数据的现象。

可以选择在 COM_IRQ_THREAD 的基础上，额外注册 interrupt_ctrl 接口，注册该接口后，会使用全部的 LLC_HANDLE_S::rx_cache 空间缓存数据。interrupt_ctrl 接口需要支持开、关中断，设置中断阈值，即接收到多少个字节的数据才会触发中断。

也可以换用 COM_ISR 通知类型，该通知类型支持使用全部的 LLC_HANDLE_S::rx_cache 空间缓存数据。

8.1.3 无法接收到通过 checksum 校验的上报数据包

如果 Host 开启了 cache，有可能是在对 cache 的 invalid 和 clean 操作时，意外操作了其它内存地址。

可以修改移植的 host_os_malloc 和 host_os_free，让其申请的内存满足 N 字节对齐，其中 N 根据 Host 而定，例如 Host 的 cache 每次至少操作 32 字节，N 就为 32。并且 host_os_malloc 的返回值需要是 32 的倍数，实际申请的长度也要是 32 的倍数，如申请 33 字节，实际申请的地址可能是 0x10507715，host_os_malloc 应返回地址 0x10507720，申请的内存空间末尾的地址是 0x1050775F。

简单来说，如果开启 cache、N 为 32，应满足：

- host_os_malloc 返回值 32 字节对齐；
- 申请的内存空间末尾的地址+1 应 32 字节对齐。

8.1.4 指令与回复

指令一般指 Host 向 Device 发送的指令。

回复一般指 Device 对接收到的指令的回复。是否回复，由指令中 Header 的 Req 字段决定。

Device 接收、处理指令，**不支持并发**。即，在将上一个指令处理完毕或回复后，Device 才会接收下一个指令。

8.2 配置问题

8.2.1 通过 Host 修改工具/上位机对应的配置

通过 Host 修改配置有两种方法，分别是上位机修改参数辨别 TLV 和直接查看 TLV 说明文档。

在上位机修改一项参数并观察修改前后的 TLV 码流，找到不同的那一组码流即可判断这一项参数对应哪一个 TLV，然后再到 TLV 的说明文档中查看该 TLV 的作用以及是否存在多项配置，即一个 TLV 配置多个参数。

直接查看 TLV 说明文档能够更准确地了解所有 TLV 对应的功能及如何配置。

8.3 雷达工作异常

8.3.1 radar analysis sample 的帧周期设置得很小时，雷达突然不上报了

当帧周期设置得很小时，有可能还没有接收完数据，雷达的探测中断就触发了，触发该中断后，会尝试申请大量内存空间用于计算、保存结果等工作，如果前一帧数据没有接收完成，雷达就不会释放这些数据占用的内存空间，从而导致申请内存失败而挂死。

最小帧周期的计算可以参考公式：

$$T = t_{\text{radar}} + t_{\text{report}} + t_{\text{delay}}$$

其中， t_{radar} 是从雷达发波到产生中断的时间，可近似认为是发波时间，单位：毫秒；

t_{report} 是数据上报时间，单位：毫秒；

t_{delay} 是上报延时时间，单位：毫秒。

t_{radar} 的计算公式如下：

$$t_{\text{radar}} = \text{ceiling}\left(\frac{150}{V_{\text{res}} f_{\text{start}}}\right)$$

其中， $\text{ceiling}(x)$ 表示对 x 向上取整；

V_{res} 是速度分辨率，单位：米/秒；

f_{start} 是起始频率，单位：GHz。

SPI 上报的 t_{report} 计算公式如下：

$$t_{\text{report}} = \text{ceiling}\left(\frac{85R_{\text{max}}V_{\text{max}}N_{\text{antCh}}}{65536R_{\text{res}}V_{\text{res}}}\right)$$

其中， N_{antCh} 是天线通道数，计算公式为 $N_{\text{antCh}} = N_{\text{tx}}N_{\text{rx}}$ ，即发射通道和接收通道的乘积；

R_{max} 是最大测量距离，单位：米；

V_{max} 是最大检测速度，单位：米/秒；

R_{res} 是距离分辨率，单位：米。

t_{report} 是以 85ms 传输 256kB 数据为例，需根据实际情况进行换算。值得注意的是，单个字节的传输时间并不是 SPI 或 UART 传输一个字节的的时间，而是整个 256kB 数据传输过程的时间平均下来的，通常会大于 SPI 或 UART 传输一个字节的的时间。

UART 上报的 t_{report} 计算公式如下：

$$t_{report} = ceiling \left(\frac{48000 N_{antCh} R_{max} V_{max}}{baudrate \times R_{res} V_{res}} \right)$$

其中， $baudrate$ 为串口波特率。

t_{delay} 在主动上报和被动上报中都为 0，仅在主动上报（延时）中有值。

附录

现有 Msg ID 定义

MSG ID	含义说明
0x00	获取芯片型号/ID，固件版本号
0x01	获取设备状态信息
0x02	获取特性支持信息
0x03	保留使用
0x04	设备重启
0x05	设备休眠/唤醒
0x06	设备杂项控制
0x07	雷达工作/停止
0x08	保留使用
0x09	HIF 硬件接口的模式参数
0x0A	HIF 的交互模式配置
0x0B	主机状态同步
0x0C	传输控制
0x0D	保留使用
0x0E	固件下载控制
0x0F	固件数据包传输
0x10	GPIO 控制
0x11	存储使能控制
0x12	EEPROM 写
0x13	EEPROM 读
0x14	Flash ID 获取
0x15	Flash 擦除
0x16	Flash 写

MSG ID	含义说明
0x17	Flash 读
0x18	eFuse 参数配置
0x19	eFuse 参数获取
0x1A	信息状态获取
0x1B	image 数据写入
0x1C	请求命令
0x1D	Shell 功能
0x1E	Debug 控制配置
0x1F	Debug 信息读取
0x20-0x2F	内部保留使用
0x30	产测模式切换
0x31	系统电源
0x32	系统 IO
0x33	雷达性能评估
0x34	雷达校准
0x35	雷达场景功能
0x36	雷达场景功耗
0x37	校准参数保存
0x38	校准参数获取
0x39-0x3F	保留使用
0x40	雷达参数读取
0x41	雷达参数配置
0x42	AS2528 雷达参数获取
0x43-0x4F	保留使用
0x50-0x5F	保留使用
0x60	下发配置

MSG ID	含义说明
0x61	数据获取
0x62-0x64	保留使用
0x65	人体存在传感器控制设备端
0x66-0x6F	保留使用
0x70-0x7F	保留使用
0x80-0x8F	保留使用
0x90-0x9F	客户命令 ID 配置注册
0xA0-0xAF	保留使用
0xB0-0xBF	保留使用
0xC0	ADC 数据
0xC1	FFT 数据
0xC2	多类型 FFT 数据
0xC3	点云/目标
0xC4	R7 手势数据
0xC5	低功耗人存数据上报
0xC6	Motion Sensor
0xC7	Motion Sensor Plot
0xC8、0xCA-0xCB	R4 上报距离、方位角、俯仰角、uid、snr
0xC9	MRS2 上报距离
0xCC-0xCF	保留使用
0xD0-0xDF	保留使用
0xE0-0xEF	客户上报 ID 配置注册
0xF0	系统打印
0xF1	系统压缩打印
0xF2-0xFD	保留使用
0xFE	系统心跳

MSG ID	含义说明
0xFF	系统启动上报

常用 Command-Response 数据格式

● VERSION [0x00]

获取 Device 版本号。

MsgID: 0x00

Payload: 1 Byte, 0x40 表示客户配置文件 customer-cfg.img 版本, 0x42 表示 Code 和客户配置文件 customer-cfg.img 版本。

Code 版本:

```
struct code_version_t {
    uint8_t  ir_type;
    uint16_t ir_minor;

    uint16_t im_minor;
    uint8_t  im_project;
    uint16_t im_number;

    uint8_t  iv_major;
    uint8_t  iv_minor;
    uint16_t iv_revision;
};
```

客户配置文件 customer-cfg.img 版本:

```
struct version_t {
    uint8_t major;
    uint8_t minor;
    uint16_t revision;
    uint32_t build_num; // 表示 cfg 文件生成日期
};
```

32 bit build_num 日期格式如下

秒: [0:4] 2 秒为单位。Sec = (build_num & 0x1F) * 2

分: [5:10]。Min = (build_num >> 5) & 0x3F

时: [11:15]。Hour = (build_num >> 11) & 0x1F

日: [16:20]。Day = (build_num >> 16) & 0x1F

月: [21:24]。Mon = (build_num >> 21) & 0xF

年: [25:31],以 2000 年为起始年。Year = (build_num >> 25) & 0x7F + 2000
 build_num = ((Year - 2000) << 25) | (Mon << 21) | (Day<<16) | (Hour << 11) | (Min <<5) | (Sec/2)

客户配置文件版本:

CMD:

A5 24 15 00 01 20 40 AA FF FE DF

00 为获取版本号, 40 为获取 customer-cfg.img 版本。

RSP:

A5 1F 12 00 09 20 00 01 04 05 00 AD 7C 6A 33 BA 51 76 70

00 为获取版本号, 00 为返回状态, 01 04 05 00 AD 7C 6A 33 为版本号。

Code 和客户配置文件版本:

CMD:

A5 04 15 00 01 40 42 A8 FF FE BF

00 为获取版本号, 42 表示获取 app code 和 customer-cfg.img 版本。

RSP:

A5 03 12 00 15 30 00 00 00 00 00 00 00 02 00 03 00 01 04 05 00 AD 7C 6A 33 BA 4F 6A 5D

00 为获取版本号, 00 为返回状态, 00 00 00 00 00 00 00 00 02 00 03 00 01 04 05 00 AD 7C 6A 33 为版本号。

● TICK [0x01]

获取设备的心跳时钟

CMD:

A5 D3 15 01 01 70 40 AA FE FE 8F

01 是表示获取设备信息, 40 代表获取具体内容为 Tick。

RSP:

A5 22 12 01 05 20 00 2F F7 51 04 E9 CF 03 8E

01 是表示获取设备信息, 00 表示状态码, 2F F7 51 04 表示 tick 值 0x0451F72F。

● REBOOT[0x04]

设置 radar 复位重启 payload 为 1。

CMD

A5 40 15 04 01 00 01 E9 FB FE FF

04 为配置雷达复位重启, 01 为雷达复位重启。

RSP

```
A5 E3 12 04 01 60 00 ED FB FE 9F
```

04 为配置雷达参数， 00 状态 success。

● POWER_STATE_SET [0x05]

Host 下发或获取配置阶段，Device 被唤醒后不休眠，直到收到此命令后，Device 允许休眠。

CMD

```
A5 CF 15 05 01 70 01 E9 FA FE 8F
```

05 为设置设备状态， 01 为允许休眠。

RSP

```
A5 02 12 05 01 00 00 ED FA FE BF
```

05 表示获取设备配置信息， 00 表示状态码。

● START_CTRL(0x07)

设置开始 radar 检测 payload 为 1，停止雷达检测进入配置模式 payload 为 0。

CMD

```
A5 3D 15 07 01 00 01 E9 F8 FE FF
```

07 为配置雷达检测， 01 为开始 radar 检测。

RSP

```
A5 10 12 07 01 30 00 ED F8 FE CF
```

07 为配置雷达参数， 00 状态 success。

● POLL_CMD [0x0C]

当通信口为被动上报，notify io（一般是 PA6）拉高后，Host 需要向 Device 下发 poll 指令。

CMD:

```
A5 35 15 0C 04 00 02 00 01 00 E8 F3 FA FF
```

● CFG_GET [0x42]

Host 获取 radar 配置通过此命令（部分固件使用其它 ID），下发的配置为 TLV 格式。

radar 获取配置 TLV 见表 8-1 radar 配置 TLV。

CMD

```
A5 FF 15 42 04 00 12 02 00 00 D8 BB FB FF
```

42 为获取雷达参数， 12 为 type， 02 为 length， 00 00 为 Value，此 TLV 表示获取雷达工作的起始频率。

RSP

```
A5 01 12 42 05 00 00 12 02 78 E6 07 AB F8 87
```

42 为获取雷达参数，00 状态 success，12 为 type，02 为 length，78 E6 为 Value，此 TLV 表示获取到雷达的起始频率为 E678（59000）。

● CFG_SET [0x60]

Host 下发 radar 配置都通过此命令，下发的配置为 TLV 格式。

表 8-1 radar 配置 TLV（部分）

Type	Length	Value	含义说明	有效值范围
0x10	1	1	配置雷达天线工作模式（选择相应模式时下发相应参数， 1T1R:0， 1T3R:1，2T3R:2，1T4R:3，2T4R:4，1T2R:5）6130 系列仅支持 1T3R	0-5
0x11	1	1	配置雷达工作的帧模式（1D 帧为 0，2D 帧为 1）	0/1
0x12	2	59000	配置雷达工作的起始频率 unit：MHz（兆赫兹）	[57-61]GHz
0x13	4	8000	配置雷达最大检测距离 unit：mm（毫米）	[1.3-32]m
0x14	2	40	配置雷达距离分辨率 unit：mm（毫米）	[0.02-0.5]m
0x15	2	1600	配置最大模糊速度 unit：mm/s（毫米/秒）	[0.1-20]m/s
0x16	2	100	配置速度分辨率 unit：mm/s（毫米/秒）	[0.01-10]m/s
0x17	2	500	配置雷达工作帧周期 unit：ms（毫秒）	[0.05-1]s

CMD

A5 D1 15 60 04 10 17 02 2C 01 D3 9D CF EE

60 为配置雷达参数，17 为 type，02 为 length，2C 01 为 Value，此 TLV 表示设置雷达工作帧周期为 300ms。

RSP

A5 E7 12 60 01 00 00 ED 9F FE FF

60 为配置雷达参数，00 状态 success

免责声明和著作权声明

本司在本文档中提供的信息力求准确，但并不保证内容完全无误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或不遵守本文档使用说明而进行的任何不当使用行为（包括但不限于如超压，超频，超温使用）所造成的损失，本司概不负责。

本文中的内容，包括但不限于供参考的 URL 地址，由于产品版本升级或其他原因，与实际内容存在差异，如有变更，恕不另行通知。文档“按现状”提供，不负任何担保责任，包括对适销性、适用于特定用途或非侵权性的任何担保，和任何提案、规格或样品在他处提到的任何担保。

本文档不负任何责任，包括使用本文档内信息产生的侵犯任何专利权行为的责任。本文档在此未以禁止反言或其他方式授予任何知识产权使用许可，不管是明示许可还是暗示许可。

版权所有©2026 珠海正和微芯科技有限公司。保留所有权利。